

STAR RX72N - rx_comm_manager Library
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 libs/rx_comm_manager/inc Directory Reference	5
3.2 libs Directory Reference	5
3.3 libs/rx_comm_manager Directory Reference	6
3.4 libs/rx_comm_manager/src Directory Reference	6
4 File Documentation	7
4.1 libs/rx_comm_manager/inc/rx_comm_manager.h File Reference	7
4.1.1 Detailed Description	9
4.1.1.1 Overview	9
4.1.1.2 System Architecture	10
4.1.1.3 Channel Configuration	10
4.1.1.4 Communication Flow State Machine	11
4.1.1.5 Hardware Requirements	12
4.1.1.6 Performance Characteristics	13
4.1.1.7 Memory Usage	13
4.1.1.8 Thread Safety Analysis	14
4.1.1.9 Integration Example	14
4.1.2 Data Structure Documentation	18
4.1.2.1 struct rx_comm_event_entry_t	18
4.1.2.2 struct rx_comm_heartbeat_state_t	18
4.1.2.3 struct rx_comm_manager_config_t	19
4.1.2.4 struct rx_comm_manager_t	25
4.1.2.5 struct rx_comm_send_params_t	29
4.1.3 Typedef Documentation	30
4.1.3.1 rx_comm_frame_callback_t	30
4.1.3.2 Callback Contract	30
4.1.3.3 rx_comm_link_status_callback_t	32
4.1.4 Enumeration Type Documentation	32
4.1.4.1 rx_comm_channel_t	32
4.1.4.2 rx_comm_event_retry_constants_t	35
4.1.4.3 rx_comm_heartbeat_constants_t	36
4.1.4.4 rx_comm_link_status_t	36
4.1.4.5 rx_comm_manager_constants_t	37
4.1.5 Function Documentation	38
4.1.5.1 rx_comm_manager_channel_name()	38
4.1.5.2 rx_comm_manager_channel_ready()	40

4.1.5.3 rx_comm_manager_deinit()	43
4.1.5.4 rx_comm_manager_event_send()	45
4.1.5.5 rx_comm_manager_init()	47
4.1.5.6 rx_comm_manager_link_status()	52
4.1.5.7 rx_comm_manager_poll()	53
4.1.5.8 rx_comm_manager_process_retransmits()	57
4.1.5.9 rx_comm_manager_respond()	58
4.1.5.10 rx_comm_manager_send()	61
4.1.5.11 rx_comm_manager_set_auto_retransmit()	66
4.1.5.12 rx_comm_manager_stream_send()	67
4.2 rx_comm_manager.h	68
4.3 libs/rx_comm_manager/src/rx_comm_manager.c File Reference	84
4.3.1 Detailed Description	86
4.3.1.1 Overview	86
4.3.1.2 Implementation Architecture	86
4.3.1.3 Polling State Machine	86
4.3.1.4 Performance Characteristics	87
4.3.1.5 Memory Usage	88
4.3.1.6 Algorithm Details	89
4.3.1.7 Thread Safety Analysis	90
4.3.1.8 Integration Example	91
4.3.1.9 NASA Power of 10 Compliance	92
4.3.1.10 SOLID Principles	92
4.3.1.11 Module Dependencies	93
4.3.2 Enumeration Type Documentation	94
4.3.2.1 internal_constants_t	94
4.3.2.2 rx_comm_sim_time_t	96
4.3.3 Function Documentation	96
4.3.3.1 internal_check_heartbeat()	96
4.3.3.2 internal_get_time_ms()	98
4.3.3.3 internal_handle_frame()	98
4.3.3.4 internal_output_decoded()	101
4.3.3.5 internal_poll_spi()	103
4.3.3.6 internal_poll_usb()	105
4.3.3.7 internal_process_event_queue()	108
4.3.3.8 rx_comm_manager_channel_name()	109
4.3.3.9 rx_comm_manager_channel_ready()	111
4.3.3.10 rx_comm_manager_deinit()	115
4.3.3.11 rx_comm_manager_event_send()	117
4.3.3.12 rx_comm_manager_init()	118
4.3.3.13 rx_comm_manager_link_status()	122
4.3.3.14 rx_comm_manager_poll()	123

4.3.3.15 rx_comm_manager_process_retransmits()	126
4.3.3.16 rx_comm_manager_respond()	127
4.3.3.17 rx_comm_manager_send()	130
4.3.3.18 rx_comm_manager_set_auto_retransmit()	135
4.3.3.19 rx_comm_manager_stream_send()	136
4.3.4 Variable Documentation	137
4.3.4.1 s_tag	137
4.3.4.2 s_zero_mgr	137
4.4 rx_comm_manager.c	138

Chapter 1

Directory Hierarchy

1.1 Directories

inc	5
rx_comm_manager.h	7
libs	5
rx_comm_manager	6
inc	5
rx_comm_manager.h	7
src	6
rx_comm_manager.c	84
rx_comm_manager	6
inc	5
rx_comm_manager.h	7
src	6
rx_comm_manager.c	84
src	6
rx_comm_manager.c	84

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

libs/rx_comm_manager/inc/ rx_comm_manager.h	
Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols	7
libs/rx_comm_manager/src/ rx_comm_manager.c	
Unified Multi-Channel Communication Manager Implementation	84

Chapter 3

Directory Documentation

3.1 libs/rx_comm_manager/inc Directory Reference

Directory dependency graph for inc:

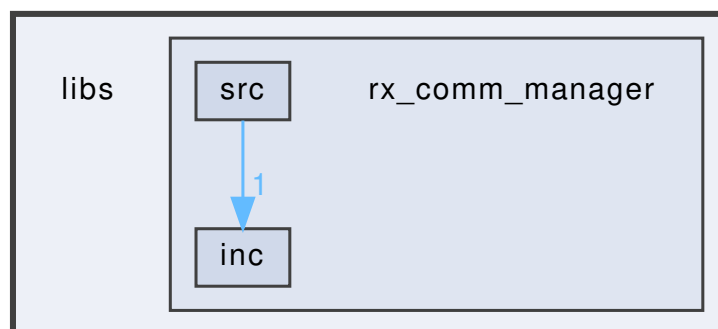


Files

- file [rx_comm_manager.h](#)
Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

3.2 libs Directory Reference

Directory dependency graph for libs:

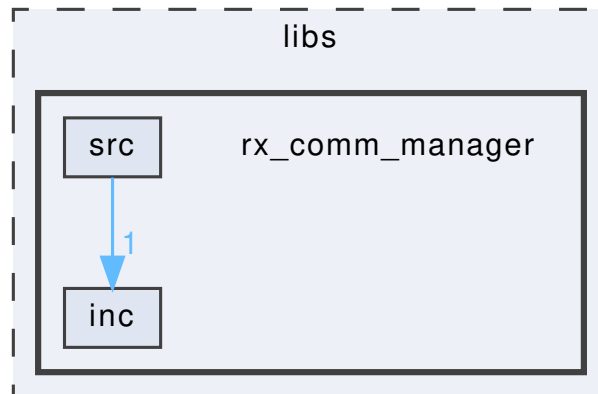


Directories

- directory [rx_comm_manager](#)

3.3 libs/rx_comm_manager Directory Reference

Directory dependency graph for rx_comm_manager:

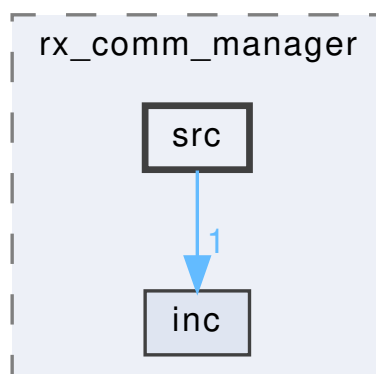


Directories

- directory [inc](#)
- directory [src](#)

3.4 libs/rx_comm_manager/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_comm_manager.c](#)
Unified Multi-Channel Communication Manager Implementation.

Chapter 4

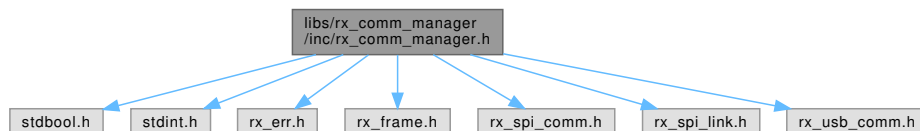
File Documentation

4.1 libs/rx_comm_manager/inc/rx_comm_manager.h File Reference

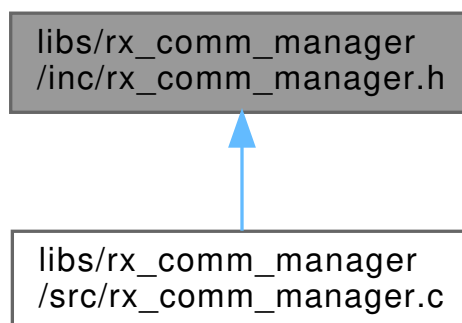
Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

```
#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_frame.h"
#include "rx_spi_comm.h"
#include "rx_spi_link.h"
#include "rx_usb_comm.h"
```

Include dependency graph for rx_comm_manager.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_comm_event_entry_t](#)
Single entry in the static event FIFO queue.
- struct [rx_comm_heartbeat_state_t](#)
Per-transport heartbeat tracking state.
- struct [rx_comm_manager_config_t](#)
Communication manager configuration.
- struct [rx_comm_manager_t](#)
Communication manager handle.
- struct [rx_comm_send_params_t](#)
Parameters for rx_comm_manager_send.

Typedefs

- typedef void(* [rx_comm_frame_callback_t](#)) ([rx_comm_channel_t](#) channel, const [rx_frame_t](#) *frame, void *ctx)
Frame received callback function type.
- typedef void(* [rx_comm_link_status_callback_t](#)) ([rx_comm_channel_t](#) channel, [rx_comm_link_status_t](#) new_status, void *ctx)
Callback invoked when a link status changes.

Enumerations

- enum [rx_comm_manager_constants_t](#): [uint16_t](#) { [k_comm_manager_ascii_buf_size](#) = 2048 , [k_comm_event_queue_depth](#) = 8 }
Communication manager compile-time constants.
- enum [rx_comm_heartbeat_constants_t](#): [uint32_t](#) { [k_heartbeat_implicit_timeout_ms](#) = 200 , [k_heartbeat_ping_interval_ms](#) = 1000 , [k_heartbeat_check_interval_ms](#) = 50 }
Heartbeat timing constants.
- enum [rx_comm_event_retry_constants_t](#): [uint16_t](#) { [k_event_max_retries](#) = 3 , [k_event_initial_backoff_ms](#) = 10 , [k_event_max_backoff_ms](#) = 100 }
Event queue retry constants for SPI HARQ.
- enum [rx_comm_channel_t](#) : [uint8_t](#) { [k_comm_channel_usb](#) = 0 , [k_comm_channel_spi](#) = 1 , [k_comm_channel_count](#) = 2 }
Communication channel identifiers.
- enum [rx_comm_link_status_t](#) : [uint8_t](#) { [k_link_status_unknown](#) = 0 , [k_link_status_healthy](#) = 1 , [k_link_status_dead](#) = 2 }
Link health status from heartbeat monitoring.

Functions

- [rx_err_t rx_comm_manager_init](#) ([rx_comm_manager_t](#) *mgr, const [rx_comm_manager_config_t](#) *cfg)
Initialize communication manager.
- [rx_err_t rx_comm_manager_deinit](#) ([rx_comm_manager_t](#) *mgr)
Deinitialize communication manager.
- [rx_err_t rx_comm_manager_poll](#) ([rx_comm_manager_t](#) *mgr)
Poll all enabled channels for incoming frames.
- [rx_err_t rx_comm_manager_send](#) ([rx_comm_manager_t](#) *mgr, const [rx_comm_send_params_t](#) *params)
Send frame on specific channel.

- rx_err_t [rx_comm_manager_respond](#) (rx_comm_manager_t *mgr, rx_comm_channel_t channel, const uint8_t *payload, uint32_t payload_len)
Send response on same channel as received frame.
- rx_err_t [rx_comm_manager_stream_send](#) (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)
Send data on the stream path (fire-and-forget).
- rx_err_t [rx_comm_manager_event_send](#) (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)
Queue data on the event path (reliable, SPI retry).
- rx_err_t [rx_comm_manager_link_status](#) (const rx_comm_manager_t *mgr, rx_comm_channel_t channel, rx_comm_link_status_t *status)
Query link health status for a transport channel.
- rx_err_t [rx_comm_manager_channel_ready](#) (rx_comm_manager_t *mgr, rx_comm_channel_t channel, bool *ready)
Check if channel is ready for communication.
- const char * [rx_comm_manager_channel_name](#) (rx_comm_channel_t channel)
Get channel name string.
- rx_err_t [rx_comm_manager_process_retransmits](#) (rx_comm_manager_t *mgr, uint32_t current_time_ms)
Process pending retransmissions on SPI channel.
- rx_err_t [rx_comm_manager_set_auto_retransmit](#) (rx_comm_manager_t *mgr, bool enabled, const rx_spi_comm_retransmit_config_t *config)
Enable or disable automatic retransmission on SPI channel.

4.1.1 Detailed Description

Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols.

4.1.1.1 Overview

Production-quality communication manager providing unified abstraction over multiple simultaneous communication channels (USB CDC and SPI) with automatic frame protocol handling, non-blocking polling architecture, optional decoded ASCII output for debugging, and callback-based event notification.

Purpose: Coordinate independent, simultaneous communication channels from RX72N MCU to Raspberry Pi 5 host, enabling command/response messaging over both USB (via CDC class) and SPI (direct hardware connection).

Protocol: Both channels use identical frame protocol (rx_frame) allowing host to send commands on either channel. Manager automatically handles:

- Frame reception and validation
- ASCII decoding output (debugging via USB Port 1)
- Channel-aware response routing
- User callback invocation

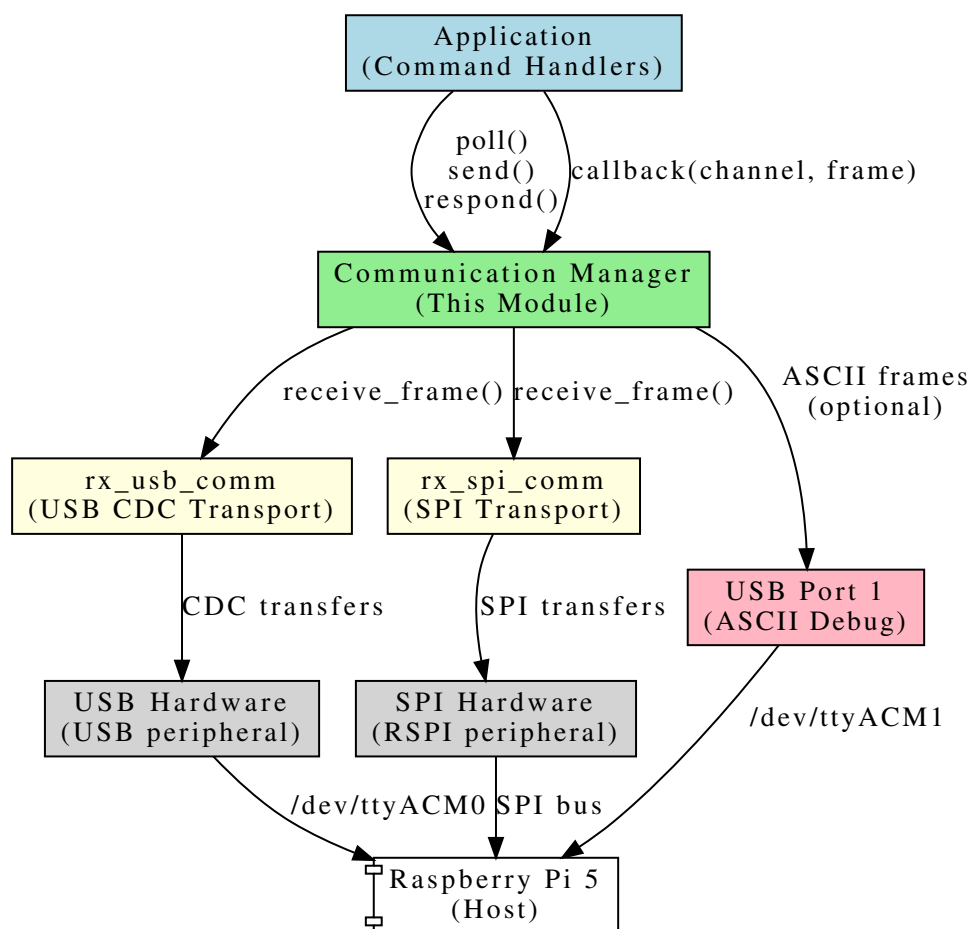
Key Features:

- **Multi-channel support:** Simultaneous USB and SPI communication
- **Non-blocking architecture:** Poll-based, no busy-waiting or blocking calls

- **Protocol-agnostic:** Both channels use same frame format (type, flags, payload, CRC)
- **Automatic ASCII output:** Optional decoded frames to USB Port 1 (human-readable debugging)
- **Callback-driven:** User-defined handler invoked on frame reception
- **Channel awareness:** Responses automatically routed to originating channel
- **Zero-copy where possible:** Direct access to frame buffers
- **Thread-safe design:** Can be called from RTOS task or main loop

4.1.1.2 System Architecture

Communication manager sits between application logic and low-level transport drivers:



4.1.1.3 Channel Configuration

The manager supports two independent communication channels:

Channel	Transport	Device	Usage	RPi5 Interface
USB	CDC class	USB Port 0	Primary protocol channel	<code>/dev/ttyACM0</code>
SPI	RSPI peripheral	RSPI channel	High-speed backup channel	<code>/dev/spidev0.0</code>

Channel	Transport	Device	Usage	RPi5 Interface
Debug	CDC class	USB Port 1	ASCII decoded output (optional)	/dev/ttyACM1

USB Channel (k_comm_channel_usb):

- Plug-and-play connectivity (no additional wiring)
- Full-speed USB 2.0 (up to 12 Mbps theoretical, ~1 MB/s practical)
- Automatic enumeration and driver loading on RPi5
- Suitable for command/response and moderate-bandwidth telemetry

SPI Channel (k_comm_channel_spi):

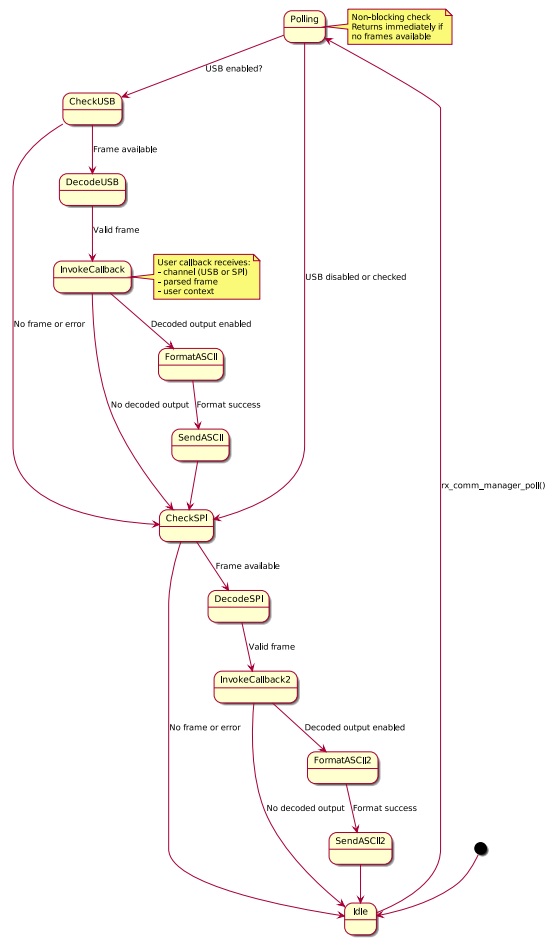
- Dedicated hardware connection (4-6 wires: COPI, CIPO, SCK, CS, optional handshake)
- Configurable speed (typically 10-20 Mbps)
- Lower latency than USB (no protocol overhead)
- Suitable for high-frequency control loops and real-time data

Debug Output (USB Port 1):

- Human-readable ASCII representation of all frames
- Format: [USB/SPI] TYPE=cmd FLAGS=0x00 LEN=42 DATA=...
- Enabled via enable_decoded_output config flag
- Zero impact on protocol channels (independent USB endpoint)

4.1.1.4 Communication Flow State Machine

The manager operates as a stateless polling architecture with per-frame state:



State Descriptions:

- **Idle:** Manager initialized, waiting for poll() call
- **Polling:** Actively checking enabled channels for frames
- **CheckUSB:** Query USB channel for available frame
- **CheckSPI:** Query SPI channel for available frame
- **DecodeUSB/DecodeSPI:** Validate and parse received frame
- **InvokeCallback:** Call user-provided frame handler
- **FormatASCII:** Convert frame to human-readable string
- **SendASCII:** Transmit ASCII representation to debug port

4.1.1.5 Hardware Requirements

Resource	Requirement	Usage
USB Peripheral	1 instance (full-speed)	CDC class with 2 ports (protocol + debug)
RSPI Peripheral	1 channel (optional)	SPI controller mode, 10-20 Mbps
GPIO	4-6 pins (if SPI)	COPI, CIPO, SCK, CS, optional handshake/interrupt
RAM	~2.5 KB per instance	Handle + ASCII buffer (2048 bytes)
Flash	~4 KB	Code + string constants

Resource	Requirement	Usage
RTOS Resources	None	Stateless, no threads/mutexes/semaphores

4.1.1.6 Performance Characteristics

Operation	Execution Time	Notes
rx_comm_manager_init()	~50 us	Handle initialization, no hardware config
rx_comm_manager_poll()	~15-200 us	Depends on channels enabled and frames available
- No frames available	~15 us	Quick check both channels, early return
- USB frame received	~120 us	Frame validation + callback + ASCII (if enabled)
- SPI frame received	~80 us	Faster than USB (less overhead)
- Both frames received	~200 us	Sequential processing of both channels
rx_comm_manager_send()	~100-150 us	Frame encoding + transport layer send
rx_comm_manager_respond()	~100-150 us	Same as send (convenience wrapper)
Callback overhead	~5-50 us	User-defined, depends on handler complexity

All timing measured @ 240 MHz CPU clock, -O2 optimization, typical frame size 64 bytes.

4.1.1.7 Memory Usage

Allocation	Size	Lifetime
rx_comm_manager_t	~12 KB	Application-managed (typically static/global)
- Pointers + state	~80 bytes	Handle metadata, heartbeat, queue indices
- ASCII buffer	2048 bytes	Decoded output formatting (if enabled)
- Event queue	8 x ~1028 bytes	Static FIFO (k_comm_event_queue_depth entries)
- Heartbeat state	~20 bytes	Per-transport heartbeat tracking
Stack (poll)	~120 bytes	Function call duration only
Stack (send)	~80 bytes	Function call duration only
Static data (.rodata)	~400 bytes	Channel names, format strings
Code (.text)	~6 KB	All functions

Total RAM: ~12 KB per manager instance (zero dynamic allocation). The event queue dominates memory: 8 entries x (k_frame_max_payload + metadata).

Memory Layout ([rx_comm_manager_t](#) structure):

Offset	Size	Field	Notes
0x00	8	usb_handle	Pointer to USB transport
0x08	8	spi_handle	Pointer to SPI transport
0x10	8	callback	Function pointer

Offset	Size	Field	Notes
0x18	8	callback_ctx	User context pointer
0x20	2048	ascii_buffer	ASCII formatting buffer
-	1	enable_decoded_output	Boolean flag
-	1	initialized	Boolean flag
-	~20	heartbeat[2]	Per-transport heartbeat state
-	~16	link_status_cb/ctx	Callback + context
-	~8224	event_queue[8]	8 x rx_comm_event_entry_t
-	3	queue head/tail/count	Queue management
Total	**~10.5 KB**		(actual size may vary by compiler)

4.1.1.8 Thread Safety Analysis

Conditional Thread Safety - Safe if following rules are observed:

1. **Single poll context:** Only call `poll()` from ONE thread/task/loop
2. **Callback restrictions:** User callback must not call manager functions (no recursion)
3. **Send synchronization:** If calling `send()/respond()` from multiple threads, use external mutex
4. **Transport thread safety:** Underlying USB/SPI transports must be thread-safe (typically are)

Not safe for:

- Concurrent `poll()` calls from multiple threads
- Calling manager functions from within frame callback
- Simultaneous `send()` calls without external synchronization

Recommendation: Use dedicated communication task:

```
void comm_task_entry(ULONG ctx) {
    rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
    while (1) {
        rx_comm_manager_poll(mgr);
        tx_thread_sleep(1); // 1ms polling period = 1kHz
    }
}
```

4.1.1.9 Integration Example

Complete Setup and Usage:

```

#include "rx_comm_manager.h"
#include "rx_usb_comm.h"
#include "rx_spi_comm.h"

// Frame handler callback
void on_frame_received(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;

    rx_log_info("COMM", "Frame from %s: type=0x%02X len=%u",
        rx_comm_manager_channel_name(channel),
        frame->type, frame->payload_len);

    // Parse frame based on type
    switch (frame->type) {
        case k_frame_type_command:
            // Handle command
            uint8_t response[128];
            uint32_t response_len = process_command(frame->payload,
                frame->payload_len,
                response, sizeof(response));

            // Respond on same channel
            rx_comm_manager_respond(mgr, channel, response, response_len);
            break;

        case k_frame_type_telemetry_req:
            // Send telemetry data
            send_telemetry(mgr, channel);
            break;

        default:
            rx_log_warn("COMM", "Unknown frame type: 0x%02X", frame->type);
            break;
    }
}

// Main application
int main(void) {
    // Initialize transports
    rx_usb_comm_handle_t usb_comm;
    rx_spi_comm_handle_t spi_comm;

    rx_usb_comm_init(&usb_comm, &usb_config);
    rx_spi_comm_init(&spi_comm, &spi_config);

    // Initialize communication manager
    rx_comm_manager_t comm_mgr;
    rx_comm_manager_config_t cfg = {
        .usb_handle = &usb_comm,
        .spi_handle = &spi_comm,
        .callback = on_frame_received,
        .callback_ctx = &comm_mgr, // Pass manager to callback
        .enable_decoded_output = true, // ASCII debug on Port 1
    };

    rx_err_t err = rx_comm_manager_init(&comm_mgr, &cfg);
    if (err != k_rx_ok) {
        rx_log_error("APP", "Comm manager init failed: 0x%X", err);
        return -1;
    }

    // Main communication loop
    while (1) {
        // Poll for incoming frames (non-blocking)
        err = rx_comm_manager_poll(&comm_mgr);

        // err == k_rx_ok: Frame(s) received and processed
        // err == k_rx_err_timeout: No frames available (normal)

        // Other application tasks...

        tx_thread_sleep(1); // 1ms = 1kHz polling rate
    }

    return 0;
}

```

Sending Unsolicited Data (Telemetry):

```

void send_periodic_telemetry(rx_comm_manager_t* mgr) {
    // Gather telemetry data
    telemetry_data_t telemetry;

```

```

gather_telemetry(&telemetry);

// Serialize to buffer
uint8_t payload[256];
uint32_t payload_len = serialize_telemetry(&telemetry, payload, sizeof(payload));

// Send on USB channel (more reliable for telemetry)
rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_telemetry,
    .flags = 0,
    .payload = payload,
    .payload_len = payload_len,
};

rx_err_t err = rx_comm_manager_send(mgr, &params);
if (err != k_rx_ok) {
    rx_log_warn("TELEM", "Failed to send telemetry: 0x%X", err);
}
}

```

Module Dependencies:

- rx_usb_comm: USB CDC transport layer
- rx_spi_comm: SPI transport layer
- rx_frame: Frame protocol definition and encoding/decoding
- rx_err: Standardized error code system
- rx_log: Logging infrastructure

See also

rx_usb_comm.h USB CDC transport implementation

rx_spi_comm.h SPI transport implementation

rx_frame.h Frame protocol specification

docs/sections/01_nanopb_protocol.tex Communication protocol documentation

NASA Power of 10 Compliance:

- Rule 1: [OK] No goto, setjmp/longjmp, or recursion used
- Rule 2: [OK] All loops have compile-time bounded iterations (channel count = 2)
- Rule 3: [OK] Zero dynamic memory allocation (all buffers static)
- Rule 4: [OK] All functions < 60 lines
- Rule 5: [OK] Minimum 2 assertions per function via parameter validation
- Rule 6: [OK] Variables declared at smallest scope
- Rule 7: [OK] All return values checked and propagated
- Rule 8: [OK] Uses C23 typed enums exclusively (no magic numbers)
- Rule 9: [WARN] INTENTIONAL DEVIATION: Uses function pointers for callback (event-driven architecture)
- Rule 10: [OK] Compiles with -Wall -Wextra -Werror (zero warnings)

SOLID Principles:

- **S (Single Responsibility):** Only channel coordination and frame routing. Protocol parsing delegated to rx_frame, transport to rx_usb_comm/rx_spi_comm.
- **O (Open/Closed):** Extensible to new channels by adding enum value and transport handle. Closed for modification (no changes needed for new frame types).
- **L (Liskov Substitution):** USB and SPI transports interchangeable via common frame interface. Can substitute mock transports for testing.
- **I (Interface Segregation):** Clean separation: lifecycle (init/deinit), polling (poll), sending (send/respond), status (channel_ready). Users include only what they need.
- **D (Dependency Inversion):** Depends on abstractions (rx_frame protocol, generic transport interface) not concrete USB/SPI implementations.

Warning

Do NOT call manager functions from within frame callback (recursion risk)

ASCII buffer is 2048 bytes - ensure payload_len + overhead < 2000 bytes to avoid truncation

Polling from multiple threads without synchronization will cause corruption

Attention

Manager does not own USB/SPI handles - caller must initialize transports before manager

Callback is invoked from poll() context - keep processing minimal or defer to task queue

Decoded ASCII output adds ~80 us latency per frame - disable for high-frequency applications

Author

STAR Team

Date

2026-01-27

Version

1.0.0

Copyright

MIT License

Changelog:

- 1.0.0 (2026-01-26): Initial implementation with USB and SPI channels
- 1.1.0 (2026-01-27): Added comprehensive Doxygen documentation, state machine diagrams, performance data

Definition in file [rx_comm_manager.h](#).

4.1.2 Data Structure Documentation

4.1.2.1 struct rx_comm_event_entry_t

Single entry in the static event FIFO queue.

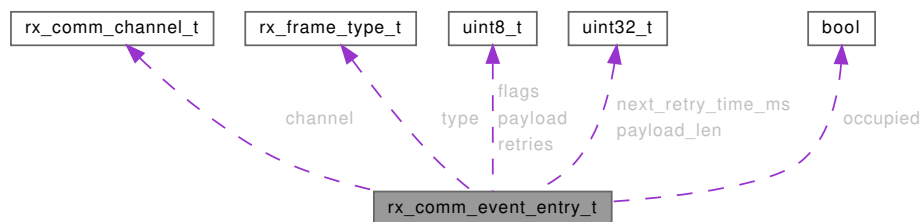
Events (commands) that require reliable delivery are queued here. Each entry contains a copy of the payload and metadata needed for retry logic. SPI entries retry with exponential backoff; USB entries are fire-and-forget.

Since

Version 1.0.0

Definition at line 727 of file [rx_comm_manager.h](#).

Collaboration diagram for rx_comm_event_entry_t:



Data Fields

rx_comm_channel_t	channel	Target transport
uint8_t	flags	Frame flags
uint32_t	next_retry_time_ms	Earliest time (ms) for next retry
bool	occupied	Slot in use
uint8_t	payload[k_frame_max_payload]	Payload copy
uint32_t	payload_len	Payload length (bytes)
uint8_t	retries	Retry attempts so far
rx_frame_type_t	type	Frame type

4.1.2.2 struct rx_comm_heartbeat_state_t

Per-transport heartbeat tracking state.

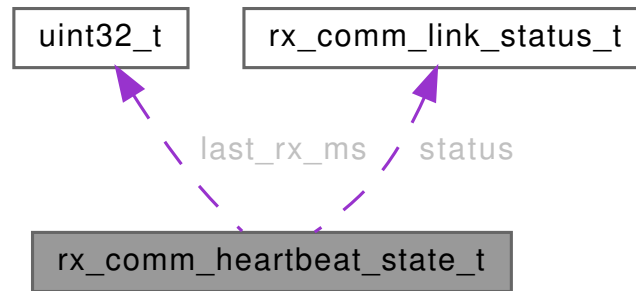
Tracks the last time a valid frame was received on each transport. Used by the poll loop to evaluate link health per the dual-detection heartbeat model.

Since

Version 1.0.0

Definition at line 749 of file [rx_comm_manager.h](#).

Collaboration diagram for rx_comm_heartbeat_state_t:



Data Fields

<code>uint32_t</code>	<code>last_rx_ms</code>	Timestamp (ms) of last valid frame
rx_comm_link_status_t	<code>status</code>	Current link health

4.1.2.3 struct rx_comm_manager_config_t

Communication manager configuration.

Configuration structure passed to [rx_comm_manager_init\(\)](#). Defines which channels are enabled, callback function, and optional features.

Configuration Guidelines:

- At least one channel (`usb_handle` OR `spi_handle`) should be non-NULL
- Both channels can be enabled simultaneously
- Callback is optional (NULL = receive but don't notify)
- Decoded output recommended for development, disable for production

Memory Lifetime:

- Config structure only read during init, can be stack-allocated
- Transport handles must remain valid for manager lifetime
- Callback function must remain valid (typically static function)

Configuration Example:

```
rx_comm_manager_config_t cfg = {
    .usb_handle = &usb_comm,      // Enable USB channel
    .spi_handle = &spi_comm,      // Enable SPI channel
    .callback = on_frame_received, // Frame handler
    .callback_ctx = &app_state,   // User context
    .enable_decoded_output = true, // ASCII debug output
};
```

See also

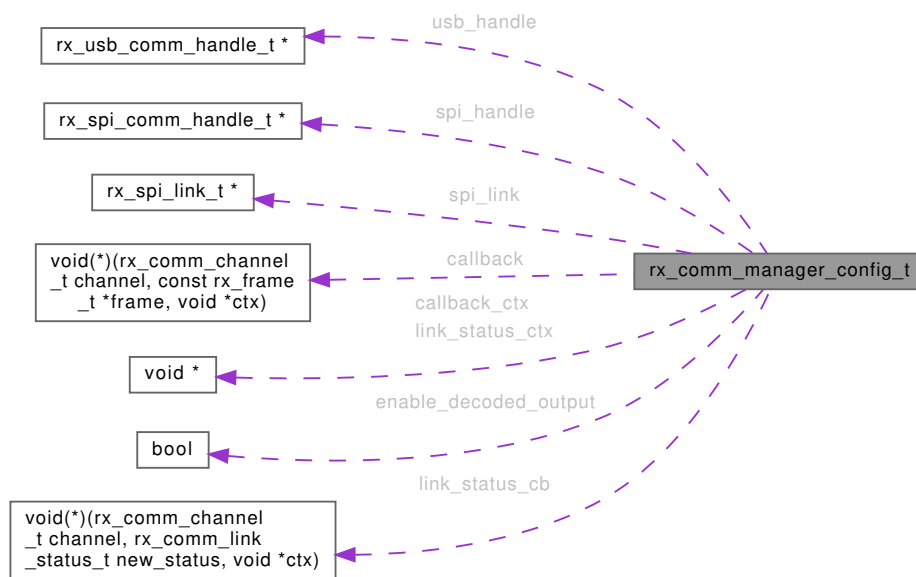
[rx_comm_manager_init\(\)](#) Initialization function

Since

Version 1.0.0

Definition at line 801 of file [rx_comm_manager.h](#).

Collaboration diagram for rx_comm_manager_config_t:



Data Fields

rx_comm_frame_callback_t	callback	Frame received callback function (NULL for no callback). User-defined function invoked when complete frame received on any channel. If NULL, frames are received but no notification occurs (not typical). Null handling: • NULL = Receive frames but don't notify (unusual, for testing only) • Non-NULL = Invoke callback on frame reception (normal operation) Type: rx_comm_frame_callback_t (function pointer) Valid values: Non-NULL function pointer, or nullptr to disable Signature: void callback(rx_comm_channel_t, const rx_frame_t) See also rx_comm_frame_callback_t Callback function type definition
void *	callback_ctx	User context pointer passed to callback. Arbitrary pointer passed to callback as third parameter. Typically used to pass application state, manager pointer, or other context without requiring global variables. Common patterns: • Pointer to manager itself (enables response from callback) • Pointer to application state structure • NULL if no context needed Type: void* (application-defined) Valid values: Any pointer value, or nullptr Lifetime: Must remain valid if callback uses it

bool	enable_decoded_output	Enable decoded ASCII output to USB Port 1. When true, all received and sent frames are converted to human-readable ASCII format and sent to USB Port 1 (/dev/ttyACM1 on RPi5). ASCII Format: <pre>[USB] TYPE=cmd FLAGS=0x00 LEN=42 DATA=48656C6C6F...</pre> <pre>[SPI] TYPE=resp FLAGS=0x01 LEN=128 DATA=...</pre> Performance Impact: • Adds ~80 us latency per frame (formatting + USB send) • Negligible for command/response (< 100 Hz) • May impact high-frequency telemetry (> 1 kHz) Use Cases: • Development: Enable for debugging and protocol analysis • Production: Disable for maximum performance Type: bool Value: true = enable ASCII output, false = disable Default: Recommend true for development, false for production
rx_comm_link_status_callback_t	link_status_cb	Callback invoked when a transport link status changes. Optional. Called when heartbeat monitoring detects a link transition (e.g., healthy -> dead). NULL to disable link status notifications.
void *	link_status_ctx	User context for link_status_cb.

rx_spi_comm_handle_t *	spi_handle	SPI communication handle (NULL to disable SPI channel). Pointer to initialized SPI transport handle. If NULL, SPI channel is disabled and all SPI operations are skipped. Requirements: • Must be initialized via rx_spi_comm_init() before passing to manager • Must remain valid for lifetime of manager • Requires hardware SPI connection between RX72N and RPi5 Null handling: • NULL = SPI channel disabled (valid configuration) • Non-NULL = SPI channel enabled Type: rx_spi_comm_handle_t* (pointer to SPI transport handle) Valid values: Non-NULL initialized handle, or null ptr to disable Lifetime: Must outlive manager instance	
------------------------	------------	--	--

rx_spi_link_t *	spi_link	Optional SPI link layer with HARQ (NULL to use raw SPI). When non-NULL, SPI send/receive operations route through the link layer, which provides FEC encoding/decoding, Chase Combining, and ACK/NACK handshake. When NULL, SPI operates in raw mode (frame + CRC-32 only, no FEC/HARQ). Requirements: • Must be initialized via rx_spi_link_init() before passing to manager • spi_link->spi_handle MUST be the same as config->spi_handle • Must remain valid for lifetime of manager Invariant If spi_link is non-NULL: spi_link->spi_handle == config->spi_handle If spi_link is non-NULL: spi_link->initialized == true If spi_link is non-NULL: spi_link must outlive manager instance Valid values: Initialized rx_spi_link_t*, or nullptr for raw SPI Lifetime: Must outlive manager instance See also rx_spi_link.h SPI link layer API rx_spi_link_init() Must be called before passing spi_link to manager Since Version 1.0.0
-----------------	----------	--

rx_usb_comm_handle_t *	usb_handle	USB communication handle (NULL to disable USB channel). Pointer to initialized USB CDC transport handle. If NULL, USB channel is disabled and all USB operations are skipped. Requirements: • Must be initialized via <code>rx_usb_comm_init()</code> before passing to manager • Must remain valid for lifetime of manager • Handle must be configured with: – Port 0: Protocol channel (frame communication) – Port 1: Debug channel (ASCII output, if decoded output enabled) Null handling: • NULL = USB channel disabled (valid configuration) • Non-NULL = USB channel enabled Type: rx_usb_comm_handle_t* (pointer to USB transport handle) Valid values: Non-NULL initialized handle, or nullptr to disable Lifetime: Must outlive manager instance
------------------------	------------	---

4.1.2.4 struct rx_comm_manager_t

Communication manager handle.

Opaque handle structure containing manager state and resources. Application must allocate this structure (typically static or global) and pass to all manager functions.

Initialization: Zero-initialize structure before passing to `rx_comm_manager_init()`:

```
rx_comm_manager_t mgr = {0}; // Or use memset
```

Memory Layout: See file-level documentation for detailed breakdown.

Thread Safety: Not thread-safe - protect with external mutex if accessed from multiple threads.

Warning

Do not access fields directly - use API functions only

Structure size ~2.5 KB - consider allocation carefully

See also

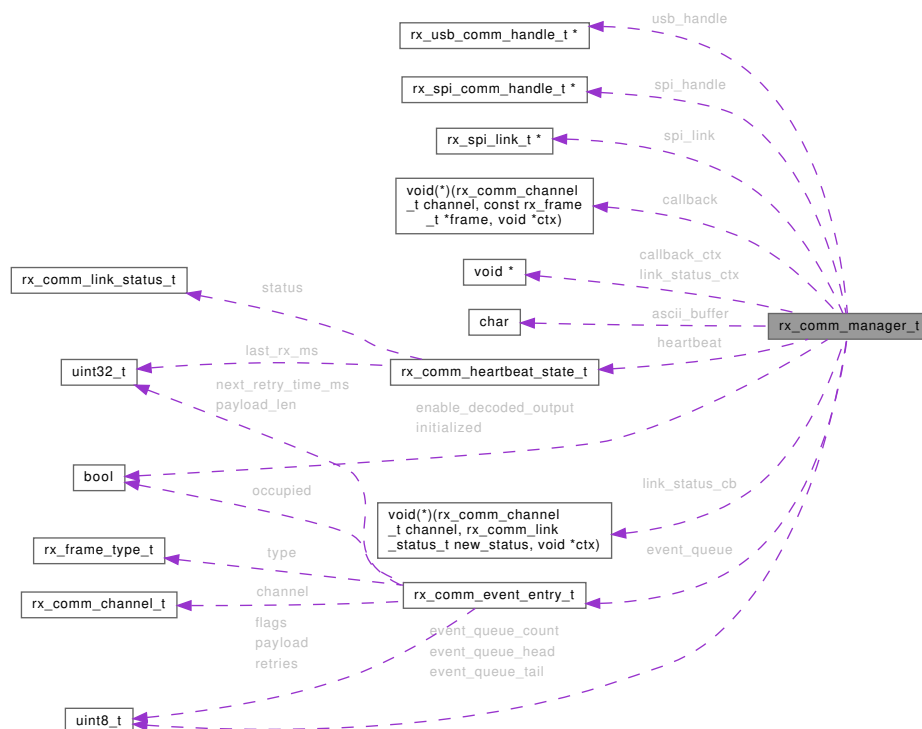
[rx_comm_manager_init\(\)](#) Initialize handle
[rx_comm_manager_deinit\(\)](#) Cleanup handle

Since

Version 1.0.0

Definition at line 976 of file [rx_comm_manager.h](#).

Collaboration diagram for `rx_comm_manager_t`:



Data Fields

char	<code>ascii_buffer[k_comm_manager_ascii_buf_size]</code>	ASCII buffer
rx_comm_frame_callback_t	callback	Frame callback
void *	callback_ctx	Callback context
bool	enable_decoded_output	Enable ASCII output
rx_comm_event_entry_t	<code>event_queue[k_comm_event_queue_depth]</code>	Static event FIFO queue for reliable command delivery.
uint8_t	event_queue_count	Number of occupied event queue slots.
uint8_t	event_queue_head	Event queue write index (next slot to write).
uint8_t	event_queue_tail	Event queue read index (next slot to process).

rx_comm_heartbeat_state_t	heartbeat[k_comm_channel_count]	Per-transport heartbeat tracking (indexed by rx_comm_channel_t).
bool	initialized	Initialization flag
rx_comm_link_status_callback_t	link_status_cb	Link status change callback (optional).
void *	link_status_ctx	Link status callback context.

rx_spi_comm_handle_t *	spi_handle	Optional SPI link layer with HARQ (NULL if disabled). SPI comm handle When non-NULL, points to an rx_spi_link_t handle providing FEC encoding/decoding, Chase Combining, and ACK/NACK handshake for SPI. When NULL, SPI operates in raw mode (frame + CRC-32 only, no FEC/HARQ). Ownership and Lifetime: • Pointer copied from config during rx_comm_manager_init() • Manager does NOT own the link object - caller must allocate/deallocate • Link object must remain valid for entire manager lifetime • Link object must NOT be freed until after rx_comm_manager_deinit() • If manager is deinitialized, spi_link remains valid but decoupled Invariants/Constraints: • If non-NULL: Must point to initialized rx_spi_link_t (spi_link->initialized == true) • If non-NULL: spi_link->spi_handle MUST equal manager's spi_handle • If non-NULL: spi_link must outlive manager instance
		• NULL is allowed to indicate no SPI HARQ (raw SPI mode)

rx_spi_link_t *	spi_link	
rx_usb_comm_handle_t *	usb_handle	USB comm handle

4.1.2.5 struct rx_comm_send_params_t

Parameters for rx_comm_manager_send.

Wrapper structure providing type safety for send operations. Prevents parameter confusion when passing multiple uint8_t/uint32_t values.

Design Rationale: Without this struct, function signature would be error-prone:

```
// BAD: Easy to mix up channel, type, flags (all similar types)
rx_err_t send(mgr, uint8_t ch, uint8_t type, uint8_t flags, ...);
```

With struct, parameters are self-documenting:

```
// GOOD: Clear naming prevents mistakes
rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_command,
    .flags = 0x00,
    // ...
};
```

Usage Example:

```
rx_comm_send_params_t params = {
    .channel = k_comm_channel_spi,
    .type = k_frame_type_telemetry,
    .flags = 0,
    .payload = telemetry_data,
    .payload_len = sizeof(telemetry_data),
};
rx_comm_manager_send(&mgr, &params);
```

See also

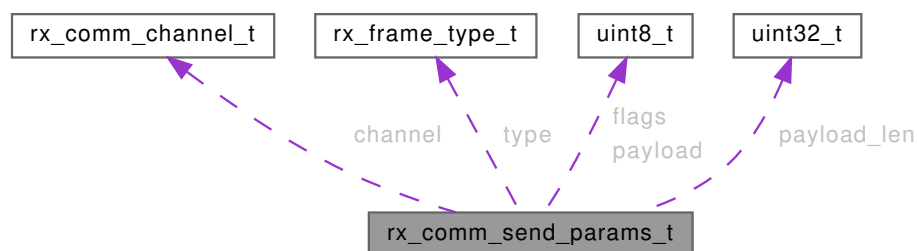
[rx_comm_manager_send\(\)](#) Send function using this structure

Since

Version 1.0.0

Definition at line 1099 of file [rx_comm_manager.h](#).

Collaboration diagram for rx_comm_send_params_t:



Data Fields

rx_comm_channel_t	channel	Channel to send on (USB or SPI)
-----------------------------------	---------	---------------------------------

uint8_t	flags	Frame flags (protocol-specific)
const uint8_t *	payload	Payload data pointer
uint32_t	payload_len	Payload length in bytes
rx_frame_type_t	type	Frame type (command, telemetry, etc.)

4.1.3 Typedef Documentation

4.1.3.1 rx_comm_frame_callback_t

```
typedef void(* rx_comm_frame_callback_t) (rx_comm_channel_t channel, const rx_frame_t *frame, void *ctx)
```

Frame received callback function type.

User-defined function invoked by communication manager when a complete, valid frame is received on any enabled channel. The callback receives:

- Channel identifier (which channel the frame arrived on)
- Parsed frame structure (type, flags, payload)
- User context pointer (passed during initialization)

4.1.3.2 Callback Contract

Responsibilities:

- Parse frame payload based on frame type
- Handle command execution or data processing
- Optionally send response via [rx_comm_manager_respond\(\)](#)
- Return quickly to avoid blocking communication loop

Restrictions:

- Do NOT call [rx_comm_manager_poll\(\)](#) from callback (recursion!)
- Keep processing minimal (~50 us recommended)
- For slow operations, queue to task and return immediately
- Do NOT modify frame structure (it may be const or recycled)

Thread Safety:

- Callback executed in context of `poll()` caller
- If `poll()` called from RTOS task, callback runs in that task context
- Synchronization required if accessing shared resources

Parameters

in	<i>channel</i>	The communication channel the frame was received on • Type: rx_comm_channel_t • Valid values: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> • Use to determine response routing
----	----------------	---

in	<i>frame</i>	Pointer to the received and validated frame • Type: <code>const rx_frame_t*</code> • Valid until callback returns • Contains: type, flags, payload, payload_len (CRC already validated) • Do NOT modify or store pointer (frame may be recycled)
in	<i>ctx</i>	User context pointer • Type: <code>void*</code> (application-defined) • Value passed during <code>rx_comm_manager_init()</code> • Typically pointer to manager, application state, or nullptr • Use to access application resources without globals

Usage Example - Basic Command Handler:

```

void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;

    // Log reception
    rx_log_info("COMM", "RX from %s: type=0x%02X len=%u",
        rx_comm_manager_channel_name(channel),
        frame->type, frame->payload_len);

    // Dispatch based on type
    switch (frame->type) {
        case k_frame_type_command:
            handle_command(mgr, channel, frame);
            break;
        case k_frame_type_telemetry_req:
            send_telemetry(mgr, channel);
            break;
        default:
            rx_log_warn("COMM", "Unknown type: 0x%02X", frame->type);
            break;
    }
}

```

Usage Example - Deferred Processing:

```

// For time-consuming operations, defer to queue
void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    app_state_t* app = (app_state_t*)ctx;

    // Copy frame to queue (frame pointer only valid during callback)
    command_msg_t msg = {
        .channel = channel,
        .type = frame->type,
        .payload_len = frame->payload_len,
    };
    memcpy(msg.payload, frame->payload, frame->payload_len);

    // Queue for processing by command handler task
    UINT status = tx_queue_send(&app->cmd_queue, &msg, TX_NO_WAIT);
    if (status != TX_SUCCESS) {
        rx_log_error("COMM", "Command queue full!");
    }
}

```

Warning

Callback must return quickly (<50 us recommended) to avoid blocking communication
 Do NOT call [rx_comm_manager_poll\(\)](#) from callback (causes recursion)
 Frame pointer only valid during callback - copy data if needed beyond callback scope

See also

[rx_comm_manager_respond\(\)](#) Send response on same channel
[rx_comm_manager_send\(\)](#) Send frame on specific channel
[rx_frame_t](#) Frame structure definition

Since

Version 1.0.0

Definition at line 695 of file [rx_comm_manager.h](#).

4.1.3.3 rx_comm_link_status_callback_t

```
typedef void(* rx_comm_link_status_callback_t) (rx_comm_channel_t channel, rx_comm_link_status_t new_status, void *ctx)
```

Callback invoked when a link status changes.

Parameters

in	<i>channel</i>	Which transport changed status
in	<i>new_status</i>	New link status
in	<i>ctx</i>	User context pointer

Since

Version 1.0.0

Definition at line 764 of file [rx_comm_manager.h](#).

4.1.4 Enumeration Type Documentation**4.1.4.1 rx_comm_channel_t**

```
enum rx_comm_channel_t : uint8_t
```

Communication channel identifiers.

Enumerates the available communication channels. The manager can handle multiple channels simultaneously, with each channel operating independently.

Channel Characteristics:

Channel	Bandwidth	Latency	Setup Complexity	Use Case
USB CDC	~1 MB/s	~1-5 ms	Low (plug-and-play)	Commands, telemetry

Channel	Bandwidth	Latency	Setup Complexity	Use Case
SPI	~2 MB/s	~0.1-1 ms	Medium (requires wiring)	High-speed data, control

Usage Pattern:

```
// Check which channel frame came from
void callback(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
    if (channel == k_comm_channel_usb) {
        // Received via USB
    } else if (channel == k_comm_channel_spi) {
        // Received via SPI
    }
}
```

See also

[rx_comm_manager_channel_name\(\)](#) Get human-readable channel name

[rx_comm_frame_callback_t](#) Callback receives channel identifier

Since

Version 1.0.0

Enumerator

k_comm_channel_usb	USB CDC channel (Port 0 - protocol). Primary USB serial port used for frame protocol communication. Maps to /dev/ttyACM0 on Raspberry Pi 5. Characteristics: • Bandwidth: ~1 MB/s practical (12 Mbps full-speed USB theoretical) • Latency: 1-5 ms (depends on USB polling interval) • Setup: Plug-and-play, automatic driver loading • Reliability: Good (USB protocol has error detection) Hardware: USB full-speed peripheral (RX72N USB module) Value: 0
--------------------	--

k_comm_channel_spi	SPI peripheral channel. Dedicated SPI connection for high-speed, low-latency communication. Requires explicit wiring between RX72N RSPI and RPi5 SPI peripheral. Characteristics: • Bandwidth: ~2 MB/s @ 20 MHz clock • Latency: 0.1-1 ms (depends on polling frequency) • Setup: Requires 4-6 wire connections + configuration • Reliability: Good (frame protocol includes CRC-32) Pin Connections: • COPI: RX72N -> RPi5 (data out from RX72N) • CIPO: RPi5 -> RX72N (data in to RX72N) • SCK: Clock (either device can be controller) • CS: Chip select (active low) • Optional: Handshake/IRQ pins for flow control Hardware: RSPI peripheral (Renesas Serial Peripheral Interface) Value: 1
k_comm_channel_count	Total number of supported channels. Compile-time constant indicating channel count. Used for array sizing and loop bounds checking (NASA Rule 2 compliance). Usage in loops: <pre>for (uint8_t i = 0; i < k_comm_channel_count; i++) { // Process each channel }</pre> Value: 2 (USB + SPI) Invariant: Must equal number of enum values (excluding this one)

Definition at line 530 of file [rx_comm_manager.h](#).

```

00530         : uint8_t {
00531     /**
00532     * @brief USB CDC channel (Port 0 - protocol)
00533     * @details
00534     * Primary USB serial port used for frame protocol communication.
00535     * Maps to `/dev/ttyACM0` on Raspberry Pi 5.
00536     *
00537     * **Characteristics:**
00538     * - Bandwidth: ~1 MB/s practical (12 Mbps full-speed USB theoretical)
00539     * - Latency: 1-5 ms (depends on USB polling interval)
00540     * - Setup: Plug-and-play, automatic driver loading
00541     * - Reliability: Good (USB protocol has error detection)
00542     *
00543     * @par Hardware: USB full-speed peripheral (RX72N USB module)
00544     * @par Value: 0
00545     */
00546     k_comm_channel_usb = 0,
00547
00548     /**

```

```

00549  * @brief SPI peripheral channel
00550  * @details
00551  * Dedicated SPI connection for high-speed, low-latency communication.
00552  * Requires explicit wiring between RX72N RSPI and RPi5 SPI peripheral.
00553  *
00554  * **Characteristics:**
00555  * - Bandwidth: ~2 MB/s @ 20 MHz clock
00556  * - Latency: 0.1-1 ms (depends on polling frequency)
00557  * - Setup: Requires 4-6 wire connections + configuration
00558  * - Reliability: Good (frame protocol includes CRC-32)
00559  *
00560  * **Pin Connections:**
00561  * - COPI: RX72N -> RPi5 (data out from RX72N)
00562  * - CIPO: RPi5 -> RX72N (data in to RX72N)
00563  * - SCK: Clock (either device can be controller)
00564  * - CS: Chip select (active low)
00565  * - Optional: Handshake/IRQ pins for flow control
00566  *
00567  * @par Hardware: RSPI peripheral (Renesas Serial Peripheral Interface)
00568  * @par Value: 1
00569  */
00570 k_comm_channel_spi = 1,
00571
00572 /**
00573  * @brief Total number of supported channels
00574  * @details
00575  * Compile-time constant indicating channel count. Used for array sizing
00576  * and loop bounds checking (NASA Rule 2 compliance).
00577  *
00578  * **Usage in loops:**
00579  * @code{.c}
00580  * for (uint8_t i = 0; i < k_comm_channel_count; i++) {
00581  *     // Process each channel
00582  * }
00583  * @endcode
00584  *
00585  * @par Value: 2 (USB + SPI)
00586  * @par Invariant: Must equal number of enum values (excluding this one)
00587  */
00588 k_comm_channel_count = 2,
00589 } rx_comm_channel_t;

```

4.1.4.2 rx_comm_event_retry_constants_t

```
enum rx_comm_event_retry_constants_t : uint16_t
```

Event queue retry constants for SPI HARQ.

Events (commands) sent via SPI use retry with exponential backoff. USB sends are fire-and-forget (no retries).

Since

Version 1.0.0

Enumerator

k_event_max_retries	Maximum SPI retry attempts
k_event_initial_backoff_ms	Initial backoff delay (ms)
k_event_max_backoff_ms	Maximum backoff delay (ms)

Definition at line 488 of file [rx_comm_manager.h](#).

```

00488         : uint16_t {
00489     k_event_max_retries      = 3, /**< Maximum SPI retry attempts */
00490     k_event_initial_backoff_ms = 10, /**< Initial backoff delay (ms) */
00491     k_event_max_backoff_ms   = 100, /**< Maximum backoff delay (ms) */
00492 } rx_comm_event_retry_constants_t;

```

4.1.4.3 rx_comm_heartbeat_constants_t

enum [rx_comm_heartbeat_constants_t](#) : uint32_t

Heartbeat timing constants.

Dual-detection heartbeat model:

- **Implicit timeout (200ms):** Any valid frame resets the timer. If no frames arrive within 200ms, the link is declared dead.
- **Explicit PING (1000ms):** Sent when link is idle for >1 s as a probe. Firmware primarily receives PINGs from the gateway.
- **Check interval (50ms):** How often to evaluate heartbeat status.

Since

Version 1.0.0

Enumerator

k_heartbeat_implicit_timeout_ms	Declare link dead if no frame for 200ms
k_heartbeat_ping_interval_ms	Send PING probe when idle for 1s
k_heartbeat_check_interval_ms	Heartbeat evaluation interval (200ms / 4)

Definition at line 472 of file [rx_comm_manager.h](#).

```
00472      : uint32_t {
00473  k_heartbeat_implicit_timeout_ms = 200, /**< Declare link dead if no frame for 200ms */
00474  k_heartbeat_ping_interval_ms   = 1000, /**< Send PING probe when idle for 1s */
00475  k_heartbeat_check_interval_ms  = 50,  /**< Heartbeat evaluation interval (200ms / 4) */
00476 } rx_comm_heartbeat_constants_t;
```

4.1.4.4 rx_comm_link_status_t

enum [rx_comm_link_status_t](#) : uint8_t

Link health status from heartbeat monitoring.

Reported per-transport link health. Determined by the dual-detection heartbeat model (implicit 200ms timeout + explicit 1s PING).

Since

Version 1.0.0

Enumerator

k_link_status_unknown	No frames received yet
k_link_status_healthy	Frames received within implicit timeout
k_link_status_dead	No frames within implicit timeout (200ms)

Definition at line 709 of file [rx_comm_manager.h](#).

```
00709      : uint8_t {
00710  k_link_status_unknown = 0, /**< No frames received yet */
00711  k_link_status_healthy = 1, /**< Frames received within implicit timeout */
00712  k_link_status_dead    = 2, /**< No frames within implicit timeout (200ms) */
00713 } rx_comm_link_status_t;
```

4.1.4.5 rx_comm_manager_constants_t

enum [rx_comm_manager_constants_t](#) : uint16_t

Communication manager compile-time constants.

Defines buffer sizes and limits for the communication manager. These constants are chosen based on typical frame sizes and ASCII formatting requirements.

Design Rationale:

- **2048-byte ASCII buffer:** Accommodates maximum frame size (1024 bytes payload) plus formatting overhead (~1000 bytes for hex dump + headers)
- Buffer is stack-allocated in handle structure (not dynamically allocated)
- Sufficient for debugging without excessive memory usage

See also

[rx_comm_manager_t.ascii_buffer](#)

Since

Version 1.0.0

Enumerator

k_comm_manager_ascii_buf_size	ASCII formatting buffer size in bytes. Size of internal buffer used to format decoded ASCII output for debug port. Must accommodate: • Header line: ~80 bytes ([USB] TYPE=cmd FLAGS=0x00 LEN=1024) • Payload hex dump: payload_len * 3 bytes (XX per byte) • Footer + null terminator: ~20 bytes Maximum frame payload: 1024 bytes Required buffer: 80 + (1024 * 3) + 20 = 3172 bytes Allocated: 2048 bytes (limits payload display to ~650 bytes, which is acceptable) Value: 2048 bytes Rationale: Balance between functionality and memory usage
-------------------------------	---

k_comm_event_queue_depth	Maximum number of entries in the event FIFO queue. Static FIFO queue depth for event (command) frames that require reliable delivery. Events are queued and sent with SPI-only retry + exponential backoff. Queue is statically allocated (zero dynamic memory). Value: 8 entries Rationale: Small queue prevents unbounded memory, sufficient for typical command burst scenarios
--------------------------	---

Definition at line 425 of file [rx_comm_manager.h](#).

```

00425         : uint16_t {
00426     /**
00427     * @brief ASCII formatting buffer size in bytes
00428     * @details
00429     * Size of internal buffer used to format decoded ASCII output for debug port.
00430     * Must accommodate:
00431     * - Header line: ~80 bytes ('[USB] TYPE=cmd FLAGS=0x00 LEN=1024')
00432     * - Payload hex dump: payload_len * 3 bytes ('XX ` per byte)
00433     * - Footer + null terminator: ~20 bytes
00434     *
00435     * Maximum frame payload: 1024 bytes
00436     * Required buffer: 80 + (1024 * 3) + 20 = 3172 bytes
00437     * Allocated: 2048 bytes (limits payload display to ~650 bytes, which is acceptable)
00438     *
00439     * @par Value: 2048 bytes
00440     * @par Rationale: Balance between functionality and memory usage
00441     */
00442     k_comm_manager_ascii_buf_size = 2048,
00443
00444     /**
00445     * @brief Maximum number of entries in the event FIFO queue
00446     * @details
00447     * Static FIFO queue depth for event (command) frames that require reliable
00448     * delivery. Events are queued and sent with SPI-only retry + exponential
00449     * backoff. Queue is statically allocated (zero dynamic memory).
00450     *
00451     * @par Value: 8 entries
00452     * @par Rationale: Small queue prevents unbounded memory, sufficient for
00453     * typical command burst scenarios
00454     */
00455     k_comm_event_queue_depth = 8,
00456 } rx_comm_manager_constants_t;

```

4.1.5 Function Documentation

4.1.5.1 rx_comm_manager_channel_name()

```

const char * rx_comm_manager_channel_name (
    rx\_comm\_channel\_t channel)

```

Get channel name string.

Parameters

in	<i>channel</i>	Channel identifier
----	----------------	--------------------

Returns

Human-readable channel name

Get channel name string.

Returns static string name for given channel enum value. Used for logging, debugging, and user interface display. Always returns a valid string (never NULL).

Returned Strings:

- `k_comm_channel_usb` -> "USB"
- `k_comm_channel_spi` -> "SPI"
- Invalid/unknown -> "UNKNOWN"

Use Cases:

- Logging channel activity
- Debugging frame traffic
- User interface display
- Error messages

Parameters

in	<i>channel</i>	Communication channel enum value • Valid range: Any <code>rx_comm_channel_t</code> value • Constraints: None (invalid values return "UNKNOWN")
----	----------------	--

Returns

`const char*` Human-readable channel name (never NULL)

Return values

<i>USB</i>	If <code>channel == k_comm_channel_usb</code>
<i>SPI</i>	If <code>channel == k_comm_channel_spi</code>
<i>UNKNOWN</i>	If channel is invalid or out of range

Precondition

None (accepts any input)

Postcondition

Returns pointer to static string (valid for program lifetime)

Note

Returned string is static - do not free
 Thread-safe (returns read-only static strings)
 Never returns NULL - always safe to use in printf

Example - Logging:

```
static void on_frame_received(rx_comm_channel_t channel,
                             const rx_frame_t* frame,
                             void* ctx)
{
    printf("Frame received on %s: type=%d len=%d\n",
          rx_comm_manager_channel_name(channel),
          frame->header.type,
          frame->header.length);
}
```

Example - Error Messages:

```
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err != k_rx_ok) {
    fprintf(stderr, "Failed to send on %s: error %d\n",
          rx_comm_manager_channel_name(params.channel), err);
}
```

See also

[rx_comm_channel_t](#) Channel enumeration

Since

Version 1.0.0

Definition at line 2015 of file [rx_comm_manager.c](#).

```
02016 {
02017     switch (channel) {
02018     case k_comm_channel_usb:
02019         return "USB";
02020     case k_comm_channel_spi:
02021         return "SPI";
02022     default:
02023         return "UNKNOWN";
02024     }
02025 }
```

References [k_comm_channel_spi](#), and [k_comm_channel_usb](#).

4.1.5.2 rx_comm_manager_channel_ready()

```
rx_err_t rx_comm_manager_channel_ready (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    bool * ready) [nodiscard]
```

Check if channel is ready for communication.

Parameters

in	<i>mgr</i>	Pointer to initialized manager
----	------------	--------------------------------

in	<i>channel</i>	Channel to check
out	<i>ready</i>	Set to true if channel is ready

Returns

- `k_rx_ok` on success
- `k_rx_err_invalid_arg` if mgr or ready is nullptr

Check if channel is ready for communication.

Checks if specified channel is configured and ready for communication. USB channel readiness depends on USB enumeration and configuration by host. SPI channel is ready immediately if handle is configured (no enumeration required).

Readiness Criteria:

- **USB:** `usb_handle` non-NULL AND USB device enumerated and configured by host
- **SPI:** `spi_handle` non-NULL (always ready if configured)

Use Cases:

- Check readiness before sending to avoid errors
- Implement fallback logic (try USB, fall back to SPI)
- Log channel availability status

Parameters

in	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init()
in	<i>channel</i>	Channel to query • Valid range: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> • Constraints: Must be valid channel ID
out	<i>ready</i>	Pointer to receive ready status • Valid range: Non-nullptr to bool • Output values: true (ready) or false (not ready) • On error: Set to false

Returns

- `rx_err_t` Error code

Return values

<i>k_rx_ok</i>	Query successful, ready flag updated
----------------	--------------------------------------

<code>k_rx_err_invalid_arg</code>	mgr or ready is nullptr, or channel invalid
<code>k_rx_err_invalid_state</code>	Manager not initialized

Precondition

mgr must be non-NULL and initialized
 ready must be non-NULL
 channel must be valid (USB or SPI)

Postcondition

*ready set to true or false based on channel state
 On error, *ready set to false (safe default)

Note

USB readiness can change at runtime (host connects/disconnects)
 SPI readiness is static (always ready if handle configured)
 This function does NOT initiate communication - only queries state

Example - Send with Fallback:

```
uint8_t payload[16] = { ... };

// Try USB first
bool usb_ready = false;
rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                             k_comm_channel_usb,
                                             &usb_ready);
if (err == k_rx_ok && usb_ready) {
    err = rx_comm_manager_respond(&g_comm_mgr,
                                 k_comm_channel_usb,
                                 payload,
                                 sizeof(payload));
} else {
    // Fallback to SPI
    bool spi_ready = false;
    err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                       k_comm_channel_spi,
                                       &spi_ready);
    if (err == k_rx_ok && spi_ready) {
        err = rx_comm_manager_respond(&g_comm_mgr,
                                     k_comm_channel_spi,
                                     payload,
                                     sizeof(payload));
    }
}
```

Example - Log Channel Status:

```
void log_channel_status(void)
{
    bool usb_ready = false, spi_ready = false;

    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_usb, &usb_ready);
    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_spi, &spi_ready);

    printf("USB: %s, SPI: %s\n",
          usb_ready ? "READY" : "NOT READY",
          spi_ready ? "READY" : "NOT READY");
}
```

See also

[rx_usb_is_configured\(\)](#) USB enumeration status

[rx_comm_manager_send\(\)](#) Send frame on channel

Since

Version 1.0.0

Definition at line 1922 of file [rx_comm_manager.c](#).

```

01923 {
01924     /* Rule 5: Pre-condition validation */
01925     if (mgr == nullptr || ready == nullptr) {
01926         if (ready != nullptr) {
01927             *ready = false;
01928         }
01929         return k_rx_err_invalid_arg;
01930     }
01931     if (!mgr->initialized) {
01932         *ready = false;
01933         return k_rx_err_invalid_state;
01934     }
01935     switch (channel) {
01936     case k_comm_channel_usb:
01937         *ready = (mgr->usb_handle != nullptr) && rx_usb_is_configured(k_usb_port_proto);
01938         break;
01939     case k_comm_channel_spi:
01940         /* SPI is always "ready" if handle is set */
01941         *ready = (mgr->spi_handle != nullptr);
01942         break;
01943     default:
01944         *ready = false;
01945         rx_log_error(s_tag, "Channel ready: invalid channel");
01946         return k_rx_err_invalid_arg;
01947     }
01948     return k_rx_ok;
01949 }

```

References [rx_comm_manager_t::initialized](#), [k_comm_channel_spi](#), [k_comm_channel_usb](#), [s_tag](#), [rx_comm_manager_t::spi_handle](#), and [rx_comm_manager_t::usb_handle](#).

4.1.5.3 rx_comm_manager_deinit()

```

rx_err_t rx_comm_manager_deinit (
    rx_comm_manager_t * mgr) [nodiscard]

```

Deinitialize communication manager.

Does not deinitialize the underlying USB/SPI handles.

Parameters

in,out	<i>mgr</i>	Pointer to manager handle
--------	------------	---------------------------

Returns

k_rx_ok on success

Deinitialize communication manager.

Marks communication manager as uninitialized, preventing further operations. This function does NOT deinitialize USB or SPI transport handles - caller must separately call rx_usb_comm_deinit() and rx_spi_comm_deinit() if needed.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated). Function primarily serves as safety guard to prevent use-after-deinit.

Parameters

in,out	<i>mgr</i>	Communication manager handle
		• Valid range: Non-nullptr to initialized handle • Constraints: Must have been initialized via rx_comm_manager_init() • Side effects: Clears mgr->initialized flag

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Success - manager deinitialized
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_invalid_state</i>	mgr was not initialized

Precondition

mgr must be non-NULL
mgr must be initialized (mgr->initialized == true)

Postcondition

mgr->initialized = false
Subsequent calls to rx_comm_manager_poll/send will return k_rx_err_invalid_state

Note

- Does NOT free memory (handle is statically allocated)
- Does NOT deinitialize transport handles (USB/SPI)
- Thread-safe if no concurrent operations on same mgr handle

Example:

```
// Deinitialize manager
rx_err_t err = rx_comm_manager_deinit(&g_comm_mgr);
if (err == k_rx_ok) {
    // Manager no longer usable

    // Separately deinitialize transports if needed
    rx_usb_comm_deinit(&g_usb_handle);
    rx_spi_comm_deinit(&g_spi_handle);
}
```

See also

- [rx_comm_manager_init\(\)](#) Initialization
- [rx_usb_comm_deinit\(\)](#) USB transport cleanup
- [rx_spi_comm_deinit\(\)](#) SPI transport cleanup

Since

Version 1.0.0

Definition at line 1249 of file [rx_comm_manager.c](#).

```
01250 {
01251     /* Rule 5: Pre-condition validation */
01252     if (mgr == nullptr) {
01253         rx_log_error(s_tag, "Deinit failed: NULL mgr");
01254         return k_rx_err_invalid_arg;
01255     }
01256     if (!mgr->initialized) {
01257         rx_log_warn(s_tag, "Deinit called on uninitialized mgr");
01258         return k_rx_err_invalid_state;
01259     }
01260
01261     mgr->initialized = false;
01262
01263     rx_log_info(s_tag, "Comm manager deinitialized");
01264     return k_rx_ok;
01265 }
```

References [rx_comm_manager_t::initialized](#), and [s_tag](#).

4.1.5.4 rx_comm_manager_event_send()

```
rx_err_t rx_comm_manager_event_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Queue data on the event path (reliable, SPI retry).

Event path is for commands and other data requiring reliable delivery. The payload is copied into a static FIFO queue and processed by the poll loop. Retry behavior depends on transport:

- **SPI:** Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
- **USB:** Fire-and-forget (USB HW reliability is sufficient)

Queue is processed in FIFO order by [rx_comm_manager_poll\(\)](#).

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
--------	------------	--------------------------------

in	<i>params</i>	Send parameters (channel, type, flags, payload, payload_len)
----	---------------	--

Returns

k_rx_ok on success (queued)
 k_rx_err_invalid_arg if mgr or params is nullptr
 k_rx_err_invalid_state if not initialized
 k_rx_err_no_mem if event queue is full

Precondition

mgr must be initialized
 params must be valid
 params->payload_len <= k_frame_max_payload

Postcondition

Payload copied to queue, will be sent by next poll()

Note

Payload is COPIED into queue - caller can reuse buffer immediately

Warning

Queue depth is k_comm_event_queue_depth (8). If full, returns error.

See also

[rx_comm_manager_stream_send\(\)](#) For fire-and-forget telemetry

Since

Version 1.0.0

Definition at line 1751 of file [rx_comm_manager.c](#).

```

01752 {
01753     if (mgr == nullptr || params == nullptr) {
01754         rx_log_error(s_tag, "Event enqueue failed: NULL arg");
01755         return k_rx_err_invalid_arg;
01756     }
01757
01758     if (!mgr->initialized) {
01759         rx_log_error(s_tag, "Event enqueue failed: not initialized");
01760         return k_rx_err_invalid_state;
01761     }
01762
01763     if (params->payload_len > k_frame_max_payload) {
01764         rx_log_error_val(s_tag, "Event enqueue failed: payload too large", params->payload_len);
01765         return k_rx_err_invalid_arg;
01766     }
01767
01768     if (params->payload_len > 0 && params->payload == nullptr) {
01769         rx_log_error(s_tag, "Event enqueue failed: NULL payload with nonzero len");
01770         return k_rx_err_invalid_arg;

```

```

01771 }
01772
01773 /* Check queue capacity */
01774 if (mgr->event_queue_count >= k_comm_event_queue_depth) {
01775     rx_log_error_val(s_tag, "Event queue full, depth", (uint8_t)mgr->event_queue_count);
01776     return k_rx_err_no_mem;
01777 }
01778
01779 /* Enqueue: copy payload into static slot */
01780 rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_head];
01781 entry->channel = params->channel;
01782 entry->type = params->type;
01783 entry->flags = params->flags;
01784 entry->payload_len = params->payload_len;
01785 entry->retries = 0;
01786 entry->occupied = true;
01787
01788 if (params->payload != nullptr && params->payload_len > 0) {
01789     (void)memcpy(entry->payload, params->payload, params->payload_len);
01790 }
01791
01792 mgr->event_queue_head = (mgr->event_queue_head + 1) % k_comm_event_queue_depth;
01793 mgr->event_queue_count++;
01794
01795 rx_log_debug_val(s_tag, "Event enqueued, queue depth", mgr->event_queue_count);
01796 return k_rx_ok;
01797 }

```

References [rx_comm_event_entry_t::channel](#), [rx_comm_send_params_t::channel](#), [rx_comm_manager_t::event_queue](#), [rx_comm_manager_t::event_queue_count](#), [rx_comm_manager_t::event_queue_head](#), [rx_comm_event_entry_t::flags](#), [rx_comm_send_params_t::flags](#), [rx_comm_manager_t::initialized](#), [k_comm_event_queue_depth](#), [rx_comm_event_entry_t::occupied](#), [rx_comm_event_entry_t::payload](#), [rx_comm_send_params_t::payload](#), [rx_comm_event_entry_t::payload_len](#), [rx_comm_send_params_t::payload_len](#), [rx_comm_event_entry_t::retries](#), [s_tag](#), [rx_comm_event_entry_t::type](#), and [rx_comm_send_params_t::type](#).

4.1.5.5 rx_comm_manager_init()

```

rx_err_t rx_comm_manager_init (
    rx_comm_manager_t * mgr,
    const rx_comm_manager_config_t * cfg) [nodiscard]

```

Initialize communication manager.

Initializes the communication manager with the provided configuration. Validates configuration parameters and sets up internal state for USB and/or SPI channel operation.

Parameters

out	<i>mgr</i>	Pointer to manager handle
in	<i>cfg</i>	Configuration (NULL for defaults, both channels disabled)

Returns

[k_rx_ok](#) on success
[k_rx_err_invalid_arg](#) if *mgr* is nullptr
[k_rx_err_invalid_arg](#) if *spi_link* is non-NULL but not initialized
[k_rx_err_invalid_arg](#) if *spi_link->spi_handle* != *cfg->spi_handle*

Precondition

mgr must not be nullptr
 If cfg->spi_link is non-NULL, spi_link must be initialized via rx_spi_link_init()
 If cfg->spi_link is non-NULL, spi_link->spi_handle must equal cfg->spi_handle
 cfg may be nullptr (results in default configuration with both channels disabled)

Postcondition

mgr->usb_handle == cfg->usb_handle (or nullptr if cfg is nullptr)
 mgr->spi_handle == cfg->spi_handle (or nullptr if cfg is nullptr)
 mgr->spi_link == cfg->spi_link (or nullptr if cfg is nullptr or cfg->spi_link is nullptr)
 All internal state initialized (event queue empty, heartbeat timers reset)

Note

Thread-safe: May be called from any thread, but concurrent calls with same mgr are prohibited

Warning

Must be called before any other manager operations on this handle

See also

rx_spi_link_init() Must call this before passing spi_link to manager
[rx_comm_manager_deinit\(\)](#) Cleanup function

Since

Version 1.0.0

Initialize communication manager.

Performs complete initialization of communication manager handle, configuring enabled channels, callback registration, and ASCII debugging output. This function must be called before any other communication manager operations.

Algorithm:

1. **Validate parameters:** Check mgr pointer for nullptr (NASA Rule 5)
2. **Clear handle:** Zero-fill entire structure via memset (328 bytes)
3. **Apply configuration:** Copy config fields if cfg is non-NULL
 - usb_handle: Pointer to initialized USB comm handle (may be NULL)
 - spi_handle: Pointer to initialized SPI comm handle (may be NULL)
 - callback: User frame handler function (may be NULL)
 - callback_ctx: User context pointer (may be NULL)
 - enable_decoded_output: ASCII debugging flag
4. **Mark initialized:** Set mgr->initialized = true

5. **Post-condition validation:** Verify `ascii_buffer` properly cleared

Configuration Options:

- **cfg = NULL:** All channels disabled, no callback, ASCII output off
- **usb_handle = NULL:** USB channel disabled (poll skips USB)
- **spi_handle = NULL:** SPI channel disabled (poll skips SPI)
- **callback = NULL:** No callback invoked (silent frame consumption)
- **enable_decoded_output = true:** ASCII debugging enabled (Port 1)

Parameters

out	<i>mgr</i>	Pointer to communication manager handle to initialize • Valid range: Non-nullptr to allocated rx_comm_manager_t (328 bytes) • Constraints: Must be allocated by caller (typically static or global) • Side effects: Entire structure zero-filled, then populated from <code>cfg</code> • Lifetime: Must remain valid until rx_comm_manager_deinit() called
-----	------------	---

in	cfg	Optional configuration structure • Valid range: NULL or pointer to valid rx_comm_manager_config_t • NULL behavior: All channels disabled, no callback, ASCII off • Lifetime: Not stored - values copied into mgr • Constraints: If non-NULL, all fields must be valid or nullptr as appropriate
----	-----	--

Returns

rx_err_t Error code indicating initialization success or failure

Return values

<i>k_rx_ok</i>	Success - manager fully initialized and ready for operation
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_invalid_state</i>	Post-condition validation failed (buffer not cleared)

Precondition

mgr must point to allocated memory (uninitialized content OK)
USB/SPI handles (if provided) must already be initialized
Callback function (if provided) must be valid and reentrant

Postcondition

mgr->initialized = true on success
All mgr fields set to config values or zero
mgr->ascii_buffer is zero-filled (256 bytes)

Note

This function does NOT initialize USB or SPI transports - caller must initialize rx_usb_comm and rx_spi_comm separately before passing handles to this function
On success, at least one channel should be enabled (usb_handle or spi_handle non-NULL)
Thread-safe ONLY if each thread uses a different mgr handle

Performance:

- Execution time: ~5 us @ 240 MHz
- Dominated by memset (328 bytes)

Configuration Example - Both Channels Enabled:

```
static rx_comm_manager_t g_comm_mgr;
static rx_usb_comm_handle_t g_usb_handle;
static rx_spi_comm_handle_t g_spi_handle;

// Initialize transport layers first
rx_usb_comm_config_t usb_cfg = { .port = k_usb_port_proto };
rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
if (err != k_rx_ok) return err;

rx_spi_comm_config_t spi_cfg = { .spi_channel = 0 };
err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
if (err != k_rx_ok) return err;

// Initialize communication manager with both channels
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = &g_spi_handle,
    .callback = on_frame_received,
    .callback_ctx = nullptr,
    .enable_decoded_output = true,
};
err = rx_comm_manager_init(&g_comm_mgr, &cfg);
```

Configuration Example - USB Only, No Callback:

```
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = nullptr,           // SPI disabled
    .callback = nullptr,           // No callback
    .callback_ctx = nullptr,
    .enable_decoded_output = false, // No ASCII debugging
};
rx_err_t err = rx_comm_manager_init(&g_comm_mgr, &cfg);
```

See also

- [rx_comm_manager_deinit\(\)](#) Cleanup and resource release
- [rx_comm_manager_poll\(\)](#) Poll for incoming frames
- [rx_usb_comm_init\(\)](#) Initialize USB CDC transport
- [rx_spi_comm_init\(\)](#) Initialize SPI transport

Since

Version 1.0.0

Definition at line 1129 of file [rx_comm_manager.c](#).

```
01130 {
01131     /* Rule 5: Pre-condition validation - nullptr check */
01132     if (mgr == nullptr) {
01133         rx_log_error(s_tag, "Init failed: NULL mgr");
01134         return k_rx_err_invalid_arg;
01135     }
01136
01137     /* Rule 5: Pre-condition validation - SPI link consistency checks */
01138     if (cfg != nullptr && cfg->spi_link != nullptr) {
01139         /* If spi_link is provided, spi_handle must also be provided */
01140         if (cfg->spi_handle == nullptr) {
01141             rx_log_error(s_tag, "Init failed: spi_link requires non-NULL spi_handle");
01142             return k_rx_err_invalid_arg;
01143         }
01144
01145         /* If spi_link is provided, it must be initialized */
01146         if (!cfg->spi_link->initialized) {
01147             rx_log_error(s_tag, "Init failed: spi_link not initialized (call rx_spi_link_init first)");
01148             return k_rx_err_invalid_arg;
01149         }
01150
01151         /* If spi_link is provided, its spi_handle must match config spi_handle */
01152         if (cfg->spi_link->spi_handle != cfg->spi_handle) {
01153             rx_log_error(s_tag, "Init failed: spi_link->spi_handle != cfg->spi_handle");
```

```

01154     return k_rx_err_invalid_arg;
01155 }
01156 }
01157
01158 *mgr = s_zero_mgr;
01159
01160 /* Apply configuration */
01161 if (cfg != nullptr) {
01162     mgr->usb_handle      = cfg->usb_handle;
01163     mgr->spi_handle      = cfg->spi_handle;
01164     mgr->spi_link        = cfg->spi_link;
01165     mgr->callback        = cfg->callback;
01166     mgr->callback_ctx    = cfg->callback_ctx;
01167     mgr->enable_decoded_output = cfg->enable_decoded_output;
01168     mgr->link_status_cb  = cfg->link_status_cb;
01169     mgr->link_status_ctx = cfg->link_status_ctx;
01170 }
01171
01172 mgr->initialized = true;
01173
01174 /* Rule 5: Post-condition validation - verify buffer properly cleared */
01175 if (mgr->ascii_buffer[k_ascii_buffer_first_idx] != '\0') {
01176     /* Buffer not properly cleared by memset - potential memory corruption */
01177     mgr->initialized = false;
01178     rx_log_error(s_tag, "Init failed: buffer corruption");
01179     return k_rx_err_invalid_state;
01180 }
01181
01182 /* Rule 5: Post-condition validation - verify handle/link consistency */
01183 if (mgr->spi_link != nullptr && mgr->spi_link != cfg->spi_link) {
01184     /* Postcondition violation: spi_link not properly assigned */
01185     mgr->initialized = false;
01186     rx_log_error(s_tag, "Init failed: spi_link assignment mismatch");
01187     return k_rx_err_invalid_state;
01188 }
01189
01190 rx_log_info(s_tag, "Comm manager initialized");
01191 return k_rx_ok;
01192 }

```

References [rx_comm_manager_t::ascii_buffer](#), [rx_comm_manager_config_t::callback](#), [rx_comm_manager_t::callback](#), [rx_comm_manager_config_t::callback_ctx](#), [rx_comm_manager_t::callback_ctx](#), [rx_comm_manager_config_t::enable_decoded_output](#), [rx_comm_manager_t::enable_decoded_output](#), [rx_comm_manager_t::initialized](#), [k_ascii_buffer_first_idx](#), [rx_comm_manager_config_t::link_status_cb](#), [rx_comm_manager_config_t::link_status_ctx](#), [rx_comm_manager_t::link_status_ctx](#), [s_tag](#), [s_zero_mgr](#), [rx_comm_manager_config_t::spi_handle](#), [rx_comm_manager_t::spi_handle](#), [rx_comm_manager_config_t::spi_link](#), [rx_comm_manager_t::spi_link](#), [rx_comm_manager_config_t::usb_handle](#), and [rx_comm_manager_t::usb_handle](#).

4.1.5.6 rx_comm_manager_link_status()

```

rx_err_t rx_comm_manager_link_status (
    const rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    rx_comm_link_status_t * status) [nodiscard]

```

Query link health status for a transport channel.

Returns the current heartbeat-derived link status for the given channel. Status is updated by [rx_comm_manager_poll\(\)](#) based on the dual-detection heartbeat model (200ms implicit timeout).

Parameters

in	<i>mgr</i>	Pointer to initialized manager
in	<i>channel</i>	Channel to query
out	<i>status</i>	Receives current link status

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if mgr or status is nullptr

Since

Version 1.0.0

Definition at line 1804 of file [rx_comm_manager.c](#).

```

01807 {
01808     if (mgr == nullptr || status == nullptr) {
01809         rx_log_error(s_tag, "Link status query: NULL arg");
01810         return k_rx_err_invalid_arg;
01811     }
01812
01813     if (!mgr->initialized) {
01814         rx_log_error(s_tag, "Link status query: uninitialized manager");
01815         return k_rx_err_invalid_state;
01816     }
01817
01818     if (channel >= k_comm_channel_count) {
01819         rx_log_error(s_tag, "Link status query: invalid channel");
01820         return k_rx_err_invalid_arg;
01821     }
01822
01823     *status = mgr->heartbeat[channel].status;
01824     return k_rx_ok;
01825 }
```

References [rx_comm_manager_t::heartbeat](#), [rx_comm_manager_t::initialized](#), [k_comm_channel_count](#), [s_tag](#), and [rx_comm_heartbeat_state_t::status](#).

4.1.5.7 rx_comm_manager_poll()

```

rx_err_t rx_comm_manager_poll (
    rx\_comm\_manager\_t * mgr) [nodiscard]
```

Poll all enabled channels for incoming frames.

Non-blocking check of USB and SPI channels. For each complete frame received:

1. If decoded output enabled, format and send ASCII to Port 1
2. Invoke user callback with frame data and channel

Call this from main loop or dedicated communication task.

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
--------	------------	--------------------------------

Returns

- k_rx_ok if at least one frame was received
- k_rx_err_timeout if no frames available on any channel
- k_rx_err_invalid_arg if mgr is nullptr
- k_rx_err_invalid_state if not initialized

Poll all enabled channels for incoming frames.

Performs non-blocking poll of all enabled channels (USB and SPI) sequentially, dispatching any received frames to the registered user callback. Returns aggregate result indicating whether any frames were received across all channels.

Algorithm:

1. **Validate parameters:** Check mgr pointer and initialized flag (NASA Rule 5)
2. **Poll USB channel** via [internal_poll_usb\(\)](#):
 - If k_rx_ok: Mark received=true, continue to SPI
 - If k_rx_err_timeout: Continue to SPI (no data is normal)
 - If other error: Return error immediately (critical failure)
3. **Poll SPI channel** via [internal_poll_spi\(\)](#):
 - If k_rx_ok: Mark received=true
 - If k_rx_err_timeout: Continue (no data is normal)
 - If other error: Return error immediately (critical failure)
4. **Return result:**
 - k_rx_ok if ANY channel received frame
 - k_rx_err_timeout if NO channels received (normal condition)
 - Other error codes propagated from transport layers

Performance: 15-200 us typical, depends on:

- Number of enabled channels (USB/SPI/both)
- Whether frames are available (data path vs fast timeout path)
- ASCII debugging enabled (adds ~50-150 us per frame)
- User callback execution time (not included in measurement)

Design Rationale:

- **Non-blocking:** Never blocks waiting for data (0 ms timeout)
- **Fair scheduling:** Both channels polled each iteration
- **Timeout tolerance:** Timeout is expected, not error condition
- **Error prioritization:** Critical errors returned immediately
- **Aggregation:** k_rx_ok if ANY channel received data

Parameters

in,out	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init() • Side effects: Invokes callback, modifies ascii_buffer if frames received
--------	------------	---

Returns

rx_err_t Error code indicating aggregate poll result

Return values

<i>k_rx_ok</i>	At least one frame received on any channel (success)
<i>k_rx_err_timeout</i>	No frames received on any channel (normal, not an error)
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_invalid_state</i>	mgr not initialized
<i>k_rx_err_crc</i>	CRC verification failed on received frame (data corruption)
<i>(other)</i>	Transport layer critical errors propagated

Precondition

mgr must be non-NULL

mgr must be initialized

At least one channel should be enabled (usb_handle or spi_handle non-NULL)

Postcondition

User callback invoked for each successfully received frame

ASCII output sent to Port 1 if enabled

Note

Non-blocking - always returns immediately

k_rx_err_timeout is NORMAL - means no data available (not an error)

User callback execution time is NOT included in performance measurements

Warning

Call this function regularly (e.g., 100 Hz) to avoid buffer overruns

User callback must not block for extended periods

Performance:

- No frames available: ~15 us (both channels timeout, fast path)
- USB frame received: ~100-250 us (USB bulk transfer + callback)
- SPI frame received: ~50-150 us (SPI hardware transfer + callback)
- Both frames: ~150-400 us (both channels + callbacks)
- With ASCII debugging: Add ~50-150 us per frame

Typical Usage - ThreadX Communication Task:

```
static void comm_task_entry(ULONG thread_input)
{
    (void)thread_input;

    // Initialize communication manager
    // ... (see rx_comm_manager_init() documentation) ...

    // Main polling loop @ 100 Hz
    while (1) {
        // Poll all channels (non-blocking)
        rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);

        // Handle critical errors (not timeout)
        if (err != k_rx_ok && err != k_rx_err_timeout) {
            // Log error, implement recovery strategy
            log_error("Comm poll failed: %d", err);
            tx_thread_sleep(100); // 1 second backoff
            continue;
        }

        // Timeout is normal - just means no data available

        // Sleep 10 ticks (10 ms @ 1 kHz tick rate = 100 Hz polling)
        tx_thread_sleep(10);
    }
}
```

Error Handling - Distinguishing Timeout from Errors:

```
rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
if (err == k_rx_ok) {
    // At least one frame received and processed
} else if (err == k_rx_err_timeout) {
    // No frames available - this is NORMAL, not an error
} else {
    // Critical error - handle appropriately
    handle_comm_error(err);
}
```

See also

[internal_poll_usb\(\)](#) USB channel polling implementation
[internal_poll_spi\(\)](#) SPI channel polling implementation
[internal_handle_frame\(\)](#) Frame dispatch
[rx_comm_manager_init\(\)](#) Initialization with channel configuration

Since

Version 1.0.0

Definition at line 1385 of file rx_comm_manager.c.

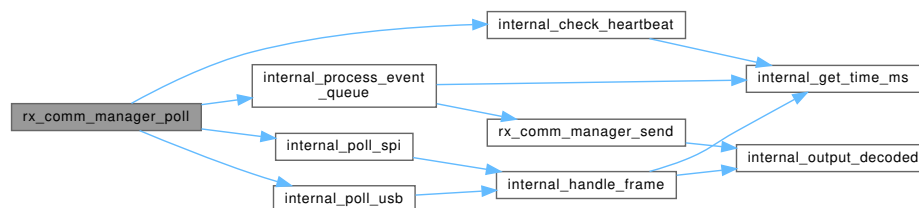
```

01386 {
01387     /* Rule 5: Pre-condition validation */
01388     if (mgr == nullptr) {
01389         return k_rx_err_invalid_arg;
01390     }
01391     if (!mgr->initialized) {
01392         return k_rx_err_invalid_state;
01393     }
01394
01395     bool received = false;
01396
01397     /* Poll USB channel */
01398     rx_err_t usb_err = internal_poll_usb(mgr);
01399     if (usb_err == k_rx_ok) {
01400         received = true;
01401     } else if (usb_err != k_rx_err_timeout) {
01402         /* Propagate non-timeout errors immediately */
01403         rx_log_error_val(s_tag, "USB poll error", usb_err);
01404         return usb_err;
01405     }
01406
01407     /* Poll SPI channel */
01408     rx_err_t spi_err = internal_poll_spi(mgr);
01409     if (spi_err == k_rx_ok) {
01410         received = true;
01411     } else if (spi_err != k_rx_err_timeout) {
01412         /* Propagate non-timeout errors immediately */
01413         rx_log_error_val(s_tag, "SPI poll error", spi_err);
01414         return spi_err;
01415     }
01416
01417     /* Process event queue: send one queued event per poll cycle */
01418     internal_process_event_queue(mgr);
01419
01420     /* Heartbeat: check implicit timeout on each transport */
01421     internal_check_heartbeat(mgr);
01422
01423     return received ? k_rx_ok : k_rx_err_timeout;
01424 }

```

References [rx_comm_manager_t::initialized](#), [internal_check_heartbeat\(\)](#), [internal_poll_spi\(\)](#), [internal_poll_usb\(\)](#), [internal_process_event_queue\(\)](#), and [s_tag](#).

Here is the call graph for this function:



4.1.5.8 rx_comm_manager_process_retransmits()

```

rx_err_t rx_comm_manager_process_retransmits (
    rx_comm_manager_t * mgr,
    uint32_t current_time_ms) [nodiscard]

```

Process pending retransmissions on SPI channel.

Thin pass-through to [rx_spi_comm_process_retransmits\(\)](#) on the SPI handle. Call this from the communication task polling loop.

Parameters

in,out	<i>mgr</i>	Communication manager
--------	------------	-----------------------

in	<i>current_time_ms</i>	Current system time in milliseconds
----	------------------------	-------------------------------------

Returns

rx_err_t Error code from rx_spi_comm_process_retransmits()

Return values

<i>k_rx_ok</i>	No action needed or retransmit sent
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_invalid_state</i>	mgr not initialized
<i>k_rx_err_retry_limit</i>	Max retries exceeded

Since

Version 1.0.0

Definition at line 2032 of file [rx_comm_manager.c](#).

```

02033 {
02034     if (mgr == nullptr) {
02035         return k_rx_err_invalid_arg;
02036     }
02037
02038     if (!mgr->initialized) {
02039         return k_rx_err_invalid_state;
02040     }
02041
02042     if (mgr->spi_handle == nullptr) {
02043         return k_rx_ok; /* No SPI channel configured */
02044     }
02045
02046     return rx_spi_comm_process_retransmits(mgr->spi_handle, current_time_ms);
02047 }

```

References [rx_comm_manager_t::initialized](#), and [rx_comm_manager_t::spi_handle](#).

4.1.5.9 rx_comm_manager_respond()

```

rx_err_t rx_comm_manager_respond (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    const uint8_t * payload,
    uint32_t payload_len) [nodiscard]

```

Send response on same channel as received frame.

Convenience function for responding to commands. Sends a RESPONSE type frame on the specified channel.

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
in	<i>channel</i>	Channel to respond on (from callback)
in	<i>payload</i>	Response payload data

in	<i>payload_len</i>	Response payload length
----	--------------------	-------------------------

Returns

k_rx_ok on success

Send response on same channel as received frame.

Convenience function for sending response frames. Automatically sets frame type to k_frame_type_response and flags to k_frame_flag_none. Simplifies common use case of responding to commands/requests.

Algorithm:

1. Build [rx_comm_send_params_t](#) with:
 - channel: User-specified
 - type: k_frame_type_response (fixed)
 - flags: k_frame_flag_none (fixed)
 - payload: User-specified
 - payload_len: User-specified
2. Call [rx_comm_manager_send\(\)](#) with constructed params
3. Return result from [rx_comm_manager_send\(\)](#)

When to Use:

- Responding to commands with telemetry data
- Sending acknowledgments
- Replying to requests

When NOT to Use:

- Need custom frame type (use [rx_comm_manager_send](#) instead)
- Need custom flags (use [rx_comm_manager_send](#) instead)

Parameters

in	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init()
----	------------	---

in	<i>channel</i>	Channel to send response on • Valid range: k_comm_channel_usb or k_comm_channel_spi • Constraints: Channel handle must be configured
in	<i>payload</i>	Response payload data • Valid range: Non-NULL if payload_len > 0, may be NULL if payload_len == 0 • Constraints: Must contain valid data for payload_len bytes • Lifetime: Copied immediately, does not need to persist
in	<i>payload_len</i>	Response payload length in bytes • Valid range: [0, k_frame_max_payload] (0-255 bytes) • Units: Bytes

Returns

rx_err_t Error code (same as rx_comm_manager_send)

Return values

<i>k_rx_ok</i>	Response sent successfully
<i>k_rx_err_invalid_arg</i>	mgr is nullptr or payload invalid
<i>k_rx_err_invalid_state</i>	Manager not initialized or channel not configured
<i>(other)</i>	Errors propagated from rx_comm_manager_send()

Precondition

Same preconditions as [rx_comm_manager_send\(\)](#)

Postcondition

Response frame transmitted with type=response, flags=none

Note

Equivalent to calling [rx_comm_manager_send\(\)](#) with type=k_frame_type_response

This is a thin wrapper - no additional validation performed

Example - Send Telemetry Response:

```
// Received a telemetry request on USB, send response
static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
{
    uint8_t telemetry[16];

    // Gather telemetry data
    telemetry[0] = get_temperature_celsius();
    telemetry[1] = get_current_ma();
    // ... fill remaining fields ...

    // Send response on same channel request came from
    rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                          channel,
                                          telemetry,
                                          sizeof(telemetry));

    if (err != k_rx_ok) {
        // Handle send error
    }
}
```

Example - Send Empty Acknowledgment:

```
// Send empty ACK response
rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                       k_comm_channel_usb,
                                       NULL, // No payload
                                       0);   // Zero length
```

See also

[rx_comm_manager_send\(\)](#) Full send function with all parameters

[rx_comm_send_params_t](#) Send parameters structure

Since

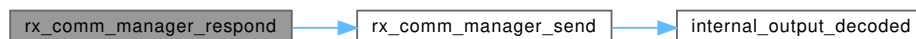
Version 1.0.0

Definition at line 1723 of file [rx_comm_manager.c](#).

```
01727 {
01728     const rx_comm_send_params_t params = {
01729         .channel    = channel,
01730         .type       = k_frame_type_response,
01731         .flags      = k_frame_flag_none,
01732         .payload    = payload,
01733         .payload_len = payload_len,
01734     };
01735     return rx_comm_manager_send(mgr, &params);
01736 }
```

References [rx_comm_manager_send\(\)](#).

Here is the call graph for this function:



4.1.5.10 rx_comm_manager_send()

```
rx_err_t rx_comm_manager_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send frame on specific channel.

Encodes and sends a frame on the specified channel. If decoded output is enabled, also sends ASCII representation to Port 1.

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
--------	------------	--------------------------------

in	<i>params</i>	Send parameters (channel, type, flags, payload, payload_len)
----	---------------	--

Returns

- k_rx_ok on success
- k_rx_err_invalid_arg if mgr or payload is nullptr
- k_rx_err_invalid_state if not initialized or channel not enabled

Send frame on specific channel.

Routes frame transmission to the appropriate transport layer (USB or SPI) based on specified channel. Supports configurable frame type, flags, and variable-length payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.

Algorithm:

1. **Validate parameters** (NASA Rule 5):

- mgr and params pointers non-NULL
- Manager initialized flag
- Payload pointer valid if payload_len > 0
- payload_len <= k_frame_max_payload (255 bytes)

2. **Route to channel:**

- USB: Verify usb_handle non-NULL, call rx_usb_comm_send()
- SPI: Verify spi_handle non-NULL, call rx_spi_comm_send()
- Invalid channel: Return k_rx_err_invalid_arg

3. **On success, if ASCII debugging enabled:**

- Build temporary rx_frame_t from params
- Format as ASCII via [internal_output_decoded\(\)](#)
- Send to USB Port 1 (TX direction)

Performance:

- USB: ~30-150 us (depends on payload size, USB bulk transfer)
- SPI: ~20-100 us (depends on payload size, SPI hardware transfer)
- ASCII debugging: Add ~50-150 us if enabled

Parameters

in	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init() • Side effects: May modify ascii_buffer if decoded output enabled
----	------------	--

in	<i>params</i>	Send parameters structure • Valid range: Non-nullptr to valid rx_comm_send_params_t • Constraints: All fields must be valid: – channel: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> – type: Frame type (command, request, response, etc.) – flags: Frame flags (<code>k_frame_flag_none</code> or specific flags) – payload: Non-NULL if <code>payload_len > 0</code>, may be NULL if <code>payload_len == 0</code> – <code>payload_len</code>: <code>[0, k_frame_max_payload]</code> (0-255 bytes) • Lifetime: Not stored - values used immediately
----	---------------	--

Returns

`rx_err_t` Error code indicating send success or failure

Return values

<i>k_rx_ok</i>	Frame sent successfully
<i>k_rx_err_invalid_arg</i>	mgr or params is nullptr, or payload invalid
<i>k_rx_err_invalid_arg</i>	<code>payload_len > k_frame_max_payload</code> (255 bytes)
<i>k_rx_err_invalid_arg</i>	Invalid channel specified
<i>k_rx_err_invalid_arg</i>	Payload is nullptr but <code>payload_len > 0</code>
<i>k_rx_err_invalid_state</i>	Manager not initialized
<i>k_rx_err_invalid_state</i>	Channel not configured (handle is nullptr)
<i>k_rx_err_hw</i>	USB/SPI hardware error (transport layer)
<i>(other)</i>	Transport layer errors propagated

Precondition

mgr must be non-NULL and initialized

params must be non-NULL with all valid fields

Channel handle (USB/SPI) must be initialized

If `payload_len > 0`, payload must point to valid data

Postcondition

Frame transmitted on specified channel if successful

ASCII output sent to Port 1 if enabled and successful

Note

Payload sequence number and CRC-32 managed by transport layer

ASCII debugging adds overhead - disable in production

Warning

Ensure channel is ready before sending (use `rx_comm_manager_channel_ready`)

Send Parameters Structure:

```
typedef struct {
    rx_comm_channel_t channel;    // USB or SPI
    uint8_t type;                // Frame type
    uint8_t flags;                // Frame flags
    const uint8_t* payload;       // Payload data
    uint32_t payload_len;         // Payload length [0, 255]
} rx_comm_send_params_t;
```

Example - Send Command on USB:

```
uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_command,
    .flags = k_frame_flag_none,
    .payload = cmd_payload,
    .payload_len = sizeof(cmd_payload),
};

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err == k_rx_ok) {
    // Command sent successfully
}
```

Example - Send Empty Frame (No Payload):

```
rx_comm_send_params_t params = {
    .channel = k_comm_channel_spi,
    .type = k_frame_type_response,
    .flags = k_frame_flag_ack,
    .payload = nullptr,          // No payload
    .payload_len = 0,            // Zero length
};

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
```

See also

[rx_comm_manager_respond\(\)](#) Convenience wrapper for response frames

[rx_comm_send_params_t](#) Send parameters structure

[rx_usb_comm_send\(\)](#) USB CDC send implementation

[rx_spi_comm_send\(\)](#) SPI send implementation

Since

Version 1.0.0

Definition at line 1540 of file `rx_comm_manager.c`.

```
01541 {
01542     /* Rule 5: Pre-condition validation */
01543     if (mgr == nullptr || params == nullptr) {
01544         rx_log_error(s_tag, "Send failed: NULL arg");
01545         return k_rx_err_invalid_arg;
01546     }
01547     if (!mgr->initialized) {
01548         rx_log_error(s_tag, "Send failed: not initialized");
01549         return k_rx_err_invalid_state;
01550     }
```

```

01551 if (params->payload == nullptr && params->payload_len > 0) {
01552     rx_log_error(s_tag, "Send failed: NULL payload with nonzero len");
01553     return k_rx_err_invalid_arg;
01554 }
01555 /* Validate payload length is within frame limits */
01556 if (params->payload_len > k_frame_max_payload) {
01557     rx_log_error_val(s_tag, "Send failed: payload too large", params->payload_len);
01558     return k_rx_err_invalid_arg;
01559 }
01560
01561 /* Send on appropriate channel */
01562 rx_err_t err = k_rx_ok;
01563 switch (params->channel) {
01564     case k_comm_channel_usb:
01565         if (mgr->usb_handle == nullptr) {
01566             rx_log_error(s_tag, "Send failed: USB handle NULL");
01567             return k_rx_err_invalid_state;
01568         }
01569         err = rx_usb_comm_send(mgr->usb_handle,
01570                               params->type,
01571                               params->flags,
01572                               params->payload,
01573                               params->payload_len);
01574         break;
01575     case k_comm_channel_spi:
01576         if (mgr->spi_handle == nullptr) {
01577             rx_log_error(s_tag, "Send failed: SPI handle NULL");
01578             return k_rx_err_invalid_state;
01579         }
01580         /* Use HARQ link layer if available, otherwise raw SPI */
01581         if (mgr->spi_link != nullptr) {
01582             /* NOTE: params->flags intentionally NOT forwarded to rx_spi_link_send().
01583              * The link layer manages its own flags internally (k_frame_flag_requires_ack,
01584              * k_frame_flag_fec_enabled, k_frame_flag_retransmit) based on HARQ state.
01585              * Application-level flags in params->flags would be ignored if passed.
01586              * CLANG WARNING: bugprone-branch-clone is a false positive here.
01587              * The two branches call different functions with different signatures:
01588              * - rx_spi_link_send(link, type, payload, len) - no flags parameter
01589              * - rx_spi_comm_send(handle, type, flags, payload, len) - includes flags */
01590             err = rx_spi_link_send(mgr->spi_link, params->type, params->payload, params->payload_len);
01591         } else {
01592             err = rx_spi_comm_send(mgr->spi_handle,
01593                                   params->type,
01594                                   params->flags,
01595                                   params->payload,
01596                                   params->payload_len);
01597         }
01598         break;
01599     default:
01600         rx_log_error(s_tag, "Send failed: invalid channel");
01601         return k_rx_err_invalid_arg;
01602 }
01603 if (err != k_rx_ok) {
01604     rx_log_error_val(s_tag, "Transport send failed", err);
01605 }
01606
01607 /* Output decoded ASCII on success */
01608 if (err == k_rx_ok && mgr->enable_decoded_output) {
01609     /* Build a temporary frame for ASCII formatting */
01610     rx_frame_t frame = {0}; /* Zero-initialize to clear padding/unset fields */
01611     frame.header.sequence = k_frame_seq_placeholder;
01612     frame.header.length = (uint16_t)params->payload_len;
01613     frame.header.type = (uint8_t)params->type;
01614     frame.header.flags = params->flags;
01615     if (params->payload_len > 0 && params->payload_len <= k_frame_max_payload) {
01616         (void)memcpy(frame.payload, params->payload, params->payload_len);
01617     }
01618     frame.crc = k_frame_crc_placeholder;
01619     internal_output_decoded(mgr, &frame, true);
01620 }
01621 return err;
01622 }

```

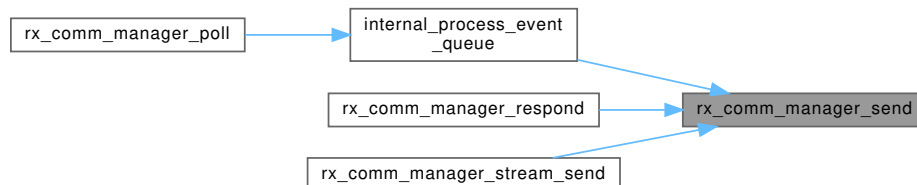
References [rx_comm_send_params_t::channel](#), [rx_comm_manager_t::enable_decoded_output](#), [rx_comm_send_params_t::flags](#), [rx_comm_manager_t::initialized](#), [internal_output_decoded\(\)](#), [k_comm_channel_spi](#), [k_comm_channel_usb](#), [k_frame_crc_placeholder](#), [k_frame_seq_placeholder](#), [rx_comm_send_params_t::payload](#), [rx_comm_send_params_t::payload_len](#), [s_tag](#), [rx_comm_manager_t::spi_handle](#), [rx_comm_manager_t::spi_link](#), [rx_comm_send_params_t::type](#), and [rx_comm_manager_t::usb_handle](#).

Referenced by [internal_process_event_queue\(\)](#), [rx_comm_manager_respond\(\)](#), and [rx_comm_manager_stream_send\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.5.11 rx_comm_manager_set_auto_retransmit()

```

rx_err_t rx_comm_manager_set_auto_retransmit (
    rx_comm_manager_t * mgr,
    bool enabled,
    const rx_spi_comm_retransmit_config_t * config) [nodiscard]
  
```

Enable or disable automatic retransmission on SPI channel.

Thin pass-through to [rx_spi_comm_set_auto_retransmit\(\)](#) on the SPI handle.

Parameters

in,out	<i>mgr</i>	Communication manager
in	<i>enabled</i>	true to enable, false to disable
in	<i>config</i>	Retransmit config (NULL to keep current/defaults)

Returns

`rx_err_t` Error code from [rx_spi_comm_set_auto_retransmit\(\)](#)

Since

Version 1.0.0

Definition at line 2049 of file [rx_comm_manager.c](#).

```

02052 {
02053     if (mgr == nullptr) {
02054         return k_rx_err_invalid_arg;
02055     }
02056
02057     if (!mgr->initialized) {
02058         return k_rx_err_invalid_state;
02059     }
02060
02061     if (mgr->spi_handle == nullptr) {
02062         return k_rx_err_not_supported;
02063     }
02064
02065     return rx_spi_comm_set_auto_retransmit(mgr->spi_handle, enabled, config);
02066 }
  
```

References [rx_comm_manager_t::initialized](#), and [rx_comm_manager_t::spi_handle](#).

4.1.5.12 rx_comm_manager_stream_send()

```
rx_err_t rx_comm_manager_stream_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send data on the stream path (fire-and-forget).

Stream path is for time-sensitive data like telemetry where stale data has no value. Sends immediately with NO retries on any transport. If the send fails, the data is simply dropped.

- USB CDC: Direct send, no retries (USB HW provides reliability)
- SPI: Direct send, no retries (stale telemetry is worthless)

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
in	<i>params</i>	Send parameters (channel, type, flags, payload, payload_len)

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if mgr or params is nullptr
 k_rx_err_invalid_state if not initialized or channel not enabled

Precondition

mgr must be initialized
 params must be valid

Postcondition

Data sent or silently dropped on failure

Note

This is a thin wrapper over [rx_comm_manager_send\(\)](#) for semantic clarity

See also

[rx_comm_manager_event_send\(\)](#) For reliable command delivery

Since

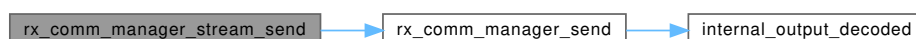
Version 1.0.0

Definition at line 1743 of file [rx_comm_manager.c](#).

```
01744 {
01745     /* Stream path: fire-and-forget, no retries, no queuing.
01746      * Telemetry and other time-sensitive data goes here.
01747      * Stale data has no value, so failure is silently accepted. */
01748     return rx_comm_manager_send(mgr, params);
01749 }
```

References [rx_comm_manager_send\(\)](#).

Here is the call graph for this function:



4.2 rx_comm_manager.h

[Go to the documentation of this file.](#)

```

00001 /* lib/rx_comm_manager/inc/rx_comm_manager.h */
00002
00003 /**
00004  * @file rx_comm_manager.h
00005  * @brief Unified Multi-Channel Communication Manager for USB CDC and SPI Protocols
00006  *
00007  * @details
00008  * ## Overview
00009  * Production-quality communication manager providing unified abstraction over
00010  * multiple simultaneous communication channels (USB CDC and SPI) with automatic
00011  * frame protocol handling, non-blocking polling architecture, optional decoded
00012  * ASCII output for debugging, and callback-based event notification.
00013  *
00014  * **Purpose**: Coordinate independent, simultaneous communication channels from
00015  * RX72N MCU to Raspberry Pi 5 host, enabling command/response messaging over
00016  * both USB (via CDC class) and SPI (direct hardware connection).
00017  *
00018  * **Protocol**: Both channels use identical frame protocol (rx_frame) allowing
00019  * host to send commands on either channel. Manager automatically handles:
00020  * - Frame reception and validation
00021  * - ASCII decoding output (debugging via USB Port 1)
00022  * - Channel-aware response routing
00023  * - User callback invocation
00024  *
00025  * **Key Features**:
00026  * - **Multi-channel support**: Simultaneous USB and SPI communication
00027  * - **Non-blocking architecture**: Poll-based, no busy-waiting or blocking calls
00028  * - **Protocol-agnostic**: Both channels use same frame format (type, flags, payload, CRC)
00029  * - **Automatic ASCII output**: Optional decoded frames to USB Port 1 (human-readable debugging)
00030  * - **Callback-driven**: User-defined handler invoked on frame reception
00031  * - **Channel awareness**: Responses automatically routed to originating channel
00032  * - **Zero-copy where possible**: Direct access to frame buffers
00033  * - **Thread-safe design**: Can be called from RTOS task or main loop
00034  *
00035  * ## System Architecture
00036  *
00037  * Communication manager sits between application logic and low-level transport drivers:
00038  *
00039  * @dot
00040  * digraph comm_manager_arch {
00041  *   rankdir=TB;
00042  *   node [shape=box, style=rounded];
00043  *
00044  *   app [label="Application\n(Command Handlers)", fillcolor=lightblue, style=filled];
00045  *   comm_mgr [label="Communication Manager\n(This Module)", fillcolor=lightgreen, style=filled];
00046  *   usb_comm [label="rx_usb_comm\n(USB CDC Transport)", fillcolor=lightyellow, style=filled];
00047  *   spi_comm [label="rx_spi_comm\n(SPI Transport)", fillcolor=lightyellow, style=filled];
00048  *   usb_hw [label="USB Hardware\n(USB peripheral)", fillcolor=lightgray, style=filled];
00049  *   spi_hw [label="SPI Hardware\n(RSPI peripheral)", fillcolor=lightgray, style=filled];
00050  *   rpi5 [label="Raspberry Pi 5\n(Host)", shape=component];
00051  *
00052  *   app -> comm_mgr [label="poll()\nsend()\nrespond()"];
00053  *   comm_mgr -> app [label="callback(channel, frame)", dir=back];
00054  *   comm_mgr -> usb_comm [label="receive_frame()"];
00055  *   comm_mgr -> spi_comm [label="receive_frame()"];
00056  *   usb_comm -> usb_hw [label="CDC transfers"];
00057  *   spi_comm -> spi_hw [label="SPI transfers"];
00058  *   usb_hw -> rpi5 [label="/dev/ttyACM0"];
00059  *   spi_hw -> rpi5 [label="SPI bus"];
00060  *
00061  *   decoded [label="USB Port 1\n(ASCII Debug)", fillcolor=lightpink, style=filled];
00062  *   comm_mgr -> decoded [label="ASCII frames\n(optional)"];
00063  *   decoded -> rpi5 [label="/dev/ttyACM1"];
00064  * }
00065  * @enddot
00066  *
00067  * ## Channel Configuration
00068  *
00069  * The manager supports two independent communication channels:
00070  *
00071  * | Channel | Transport | Device | Usage | RPi5 Interface |
00072  * |-----|-----|-----|-----|-----|
00073  * | **USB** | CDC class | USB Port 0 | Primary protocol channel | /dev/ttyACM0 |
00074  * | **SPI** | RSPI peripheral | RSPI channel | High-speed backup channel | /dev/spidev0.0 |
00075  * | **Debug** | CDC class | USB Port 1 | ASCII decoded output (optional) | /dev/ttyACM1 |
00076  *
00077  * **USB Channel (k_comm_channel_usb)**:
00078  * - Plug-and-play connectivity (no additional wiring)
00079  * - Full-speed USB 2.0 (up to 12 Mbps theoretical, ~1 MB/s practical)
00080  * - Automatic enumeration and driver loading on RPi5
00081  * - Suitable for command/response and moderate-bandwidth telemetry
00082  *

```

```

00083 **SPI Channel (k_comm_channel_spi):**
00084 * - Dedicated hardware connection (4-6 wires: COPI, CIPO, SCK, CS, optional handshake)
00085 * - Configurable speed (typically 10-20 Mbps)
00086 * - Lower latency than USB (no protocol overhead)
00087 * - Suitable for high-frequency control loops and real-time data
00088 *
00089 **Debug Output (USB Port 1):**
00090 * - Human-readable ASCII representation of all frames
00091 * - Format: `[USB/SPI] TYPE=cmd FLAGS=0x00 LEN=42 DATA=...`
00092 * - Enabled via `enable_decoded_output` config flag
00093 * - Zero impact on protocol channels (independent USB endpoint)
00094 *
00095 * ## Communication Flow State Machine
00096 *
00097 * The manager operates as a stateless polling architecture with per-frame state:
00098 *
00099 * @startuml
00100 * [*] --> Idle
00101 * Idle --> Polling : rx_comm_manager_poll()
00102 * Polling --> CheckUSB : USB enabled?
00103 * Polling --> CheckSPI : USB disabled or checked
00104 * CheckUSB --> DecodeUSB : Frame available
00105 * CheckUSB --> CheckSPI : No frame or error
00106 * DecodeUSB --> InvokeCallback : Valid frame
00107 * InvokeCallback --> FormatASCII : Decoded output enabled
00108 * InvokeCallback --> CheckSPI : No decoded output
00109 * FormatASCII --> SendASCII : Format success
00110 * SendASCII --> CheckSPI
00111 * CheckSPI --> DecodeSPI : Frame available
00112 * CheckSPI --> Idle : No frame or error
00113 * DecodeSPI --> InvokeCallback2 : Valid frame
00114 * InvokeCallback2 --> FormatASCII2 : Decoded output enabled
00115 * InvokeCallback2 --> Idle : No decoded output
00116 * FormatASCII2 --> SendASCII2 : Format success
00117 * SendASCII2 --> Idle
00118 *
00119 * note right of Polling
00120 *   Non-blocking check
00121 *   Returns immediately if
00122 *   no frames available
00123 * end note
00124 *
00125 * note right of InvokeCallback
00126 *   User callback receives:
00127 *   - channel (USB or SPI)
00128 *   - parsed frame
00129 *   - user context
00130 * end note
00131 * @enduml
00132 *
00133 * @par State Descriptions:
00134 * - **Idle**: Manager initialized, waiting for poll() call
00135 * - **Polling**: Actively checking enabled channels for frames
00136 * - **CheckUSB**: Query USB channel for available frame
00137 * - **CheckSPI**: Query SPI channel for available frame
00138 * - **DecodeUSB/DecodeSPI**: Validate and parse received frame
00139 * - **InvokeCallback**: Call user-provided frame handler
00140 * - **FormatASCII**: Convert frame to human-readable string
00141 * - **SendASCII**: Transmit ASCII representation to debug port
00142 *
00143 * ## Hardware Requirements
00144 *
00145 * | Resource | Requirement | Usage |
00146 * |-----|-----|-----|
00147 * | **USB Peripheral** | 1 instance (full-speed) | CDC class with 2 ports (protocol + debug) |
00148 * | **RSPI Peripheral** | 1 channel (optional) | SPI controller mode, 10-20 Mbps |
00149 * | **GPIO** | 4-6 pins (if SPI) | COPI, CIPO, SCK, CS, optional handshake/interrupt |
00150 * | **RAM** | ~2.5 KB per instance | Handle + ASCII buffer (2048 bytes) |
00151 * | **Flash** | ~4 KB | Code + string constants |
00152 * | **RTOS Resources** | None | Stateless, no threads/mutexes/semaphores |
00153 *
00154 * ## Performance Characteristics
00155 *
00156 * | Operation | Execution Time | Notes |
00157 * |-----|-----|-----|
00158 * | **rx_comm_manager_init()** | ~50 us | Handle initialization, no hardware config |
00159 * | **rx_comm_manager_poll()** | ~15-200 us | Depends on channels enabled and frames available |
00160 * | - No frames available | ~15 us | Quick check both channels, early return |
00161 * | - USB frame received | ~120 us | Frame validation + callback + ASCII (if enabled) |
00162 * | - SPI frame received | ~80 us | Faster than USB (less overhead) |
00163 * | - Both frames received | ~200 us | Sequential processing of both channels |
00164 * | **rx_comm_manager_send()** | ~100-150 us | Frame encoding + transport layer send |
00165 * | **rx_comm_manager_respond()** | ~100-150 us | Same as send (convenience wrapper) |
00166 * | **Callback overhead** | ~5-50 us | User-defined, depends on handler complexity |
00167 *
00168 * All timing measured @ 240 MHz CPU clock, -O2 optimization, typical frame size 64 bytes.
00169 *

```

```

00170 * ## Memory Usage
00171 *
00172 * | Allocation | Size | Lifetime |
00173 * |-----|-----|-----|
00174 * | **rx_comm_manager_t** | ~12 KB | Application-managed (typically static/global) |
00175 * | - Pointers + state | ~80 bytes | Handle metadata, heartbeat, queue indices |
00176 * | - ASCII buffer | 2048 bytes | Decoded output formatting (if enabled) |
00177 * | - Event queue | 8 x ~1028 bytes | Static FIFO (k_comm_event_queue_depth entries) |
00178 * | - Heartbeat state | ~20 bytes | Per-transport heartbeat tracking |
00179 * | **Stack (poll)** | ~120 bytes | Function call duration only |
00180 * | **Stack (send)** | ~80 bytes | Function call duration only |
00181 * | **Static data (.rodata)** | ~400 bytes | Channel names, format strings |
00182 * | **Code (.text)** | ~6 KB | All functions |
00183 *
00184 * Total RAM: ~12 KB per manager instance (zero dynamic allocation).
00185 * The event queue dominates memory: 8 entries x (k_frame_max_payload + metadata).
00186 *
00187 * **Memory Layout** (rx_comm_manager_t structure):
00188 *
00189 * | Offset | Size | Field | Notes |
00190 * |-----|-----|-----|-----|
00191 * | 0x00 | 8 | usb_handle | Pointer to USB transport |
00192 * | 0x08 | 8 | spi_handle | Pointer to SPI transport |
00193 * | 0x10 | 8 | callback | Function pointer |
00194 * | 0x18 | 8 | callback_ctx | User context pointer |
00195 * | 0x20 | 2048 | ascii_buffer | ASCII formatting buffer |
00196 * | - | 1 | enable_decoded_output | Boolean flag |
00197 * | - | 1 | initialized | Boolean flag |
00198 * | - | ~20 | heartbeat[2] | Per-transport heartbeat state |
00199 * | - | ~16 | link_status_cb/ctx | Callback + context |
00200 * | - | ~8224 | event_queue[8] | 8 x rx_comm_event_entry_t |
00201 * | - | 3 | queue head/tail/count | Queue management |
00202 * | **Total** | **~10.5 KB** | | (actual size may vary by compiler) |
00203 *
00204 * ## Thread Safety Analysis
00205 *
00206 * **Conditional Thread Safety** - Safe if following rules are observed:
00207 *
00208 * 1. **Single poll context**: Only call `poll()` from ONE thread/task/loop
00209 * 2. **Callback restrictions**: User callback must not call manager functions (no recursion)
00210 * 3. **Send synchronization**: If calling `send()`/`respond()` from multiple threads, use external mutex
00211 * 4. **Transport thread safety**: Underlying USB/SPI transports must be thread-safe (typically are)
00212 *
00213 * **Not safe for**:
00214 * - Concurrent `poll()` calls from multiple threads
00215 * - Calling manager functions from within frame callback
00216 * - Simultaneous `send()` calls without external synchronization
00217 *
00218 * **Recommendation**: Use dedicated communication task:
00219 * ```c
00220 * void comm_task_entry(ULONG ctx) {
00221 *     rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
00222 *     while (1) {
00223 *         rx_comm_manager_poll(mgr);
00224 *         tx_thread_sleep(1); // 1ms polling period = 1kHz
00225 *     }
00226 * }
00227 * ```
00228 *
00229 * ## Integration Example
00230 *
00231 * @par Complete Setup and Usage:
00232 * @code{.c}
00233 * #include "rx_comm_manager.h"
00234 * #include "rx_usb_comm.h"
00235 * #include "rx_spi_comm.h"
00236 *
00237 * // Frame handler callback
00238 * void on_frame_received(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00239 *     rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
00240 *
00241 *     rx_log_info("COMM", "Frame from %s: type=0x%02X len=%u",
00242 *         rx_comm_manager_channel_name(channel),
00243 *         frame->type, frame->payload_len);
00244 *
00245 *     // Parse frame based on type
00246 *     switch (frame->type) {
00247 *     case k_frame_type_command:
00248 *         // Handle command
00249 *         uint8_t response[128];
00250 *         uint32_t response_len = process_command(frame->payload,
00251 *             frame->payload_len,
00252 *             response, sizeof(response));
00253 *
00254 *         // Respond on same channel
00255 *         rx_comm_manager_respond(mgr, channel, response, response_len);
00256 *         break;

```

```

00257 *
00258 *     case k_frame_type_telemetry_req:
00259 *         // Send telemetry data
00260 *         send_telemetry(mgr, channel);
00261 *         break;
00262 *
00263 *     default:
00264 *         rx_log_warn("COMM", "Unknown frame type: 0x%02X", frame->type);
00265 *         break;
00266 * }
00267 * }
00268 *
00269 * // Main application
00270 * int main(void) {
00271 *     // Initialize transports
00272 *     rx_usb_comm_handle_t usb_comm;
00273 *     rx_spi_comm_handle_t spi_comm;
00274 *
00275 *     rx_usb_comm_init(&usb_comm, &usb_config);
00276 *     rx_spi_comm_init(&spi_comm, &spi_config);
00277 *
00278 *     // Initialize communication manager
00279 *     rx_comm_manager_t comm_mgr;
00280 *     rx_comm_manager_config_t cfg = {
00281 *         .usb_handle = &usb_comm,
00282 *         .spi_handle = &spi_comm,
00283 *         .callback = on_frame_received,
00284 *         .callback_ctx = &comm_mgr, // Pass manager to callback
00285 *         .enable_decoded_output = true, // ASCII debug on Port 1
00286 *     };
00287 *
00288 *     rx_err_t err = rx_comm_manager_init(&comm_mgr, &cfg);
00289 *     if (err != k_rx_ok) {
00290 *         rx_log_error("APP", "Comm manager init failed: 0x%X", err);
00291 *         return -1;
00292 *     }
00293 *
00294 *     // Main communication loop
00295 *     while (1) {
00296 *         // Poll for incoming frames (non-blocking)
00297 *         err = rx_comm_manager_poll(&comm_mgr);
00298 *
00299 *         // err == k_rx_ok: Frame(s) received and processed
00300 *         // err == k_rx_err_timeout: No frames available (normal)
00301 *
00302 *         // Other application tasks...
00303 *
00304 *         tx_thread_sleep(1); // 1ms = 1kHz polling rate
00305 *     }
00306 *
00307 *     return 0;
00308 * }
00309 * @endcode
00310 *
00311 * @par Sending Unsolicited Data (Telemetry):
00312 * @code{.c}
00313 * void send_periodic_telemetry(rx_comm_manager_t* mgr) {
00314 *     // Gather telemetry data
00315 *     telemetry_data_t telemetry;
00316 *     gather_telemetry(&telemetry);
00317 *
00318 *     // Serialize to buffer
00319 *     uint8_t payload[256];
00320 *     uint32_t payload_len = serialize_telemetry(&telemetry, payload, sizeof(payload));
00321 *
00322 *     // Send on USB channel (more reliable for telemetry)
00323 *     rx_comm_send_params_t params = {
00324 *         .channel = k_comm_channel_usb,
00325 *         .type = k_frame_type_telemetry,
00326 *         .flags = 0,
00327 *         .payload = payload,
00328 *         .payload_len = payload_len,
00329 *     };
00330 *
00331 *     rx_err_t err = rx_comm_manager_send(mgr, &params);
00332 *     if (err != k_rx_ok) {
00333 *         rx_log_warn("TELEM", "Failed to send telemetry: 0x%X", err);
00334 *     }
00335 * }
00336 * @endcode
00337 *
00338 * @par Module Dependencies:
00339 * - rx_usb_comm: USB CDC transport layer
00340 * - rx_spi_comm: SPI transport layer
00341 * - rx_frame: Frame protocol definition and encoding/decoding
00342 * - rx_err: Standardized error code system
00343 * - rx_log: Logging infrastructure

```

```

00344 *
00345 * @see rx_usb_comm.h USB CDC transport implementation
00346 * @see rx_spi_comm.h SPI transport implementation
00347 * @see rx_frame.h Frame protocol specification
00348 * @see docs/sections/01_nanopb_protocol.tex Communication protocol documentation
00349 *
00350 * @par NASA Power of 10 Compliance:
00351 * - Rule 1: [OK] No goto, setjmp/longjmp, or recursion used
00352 * - Rule 2: [OK] All loops have compile-time bounded iterations (channel count = 2)
00353 * - Rule 3: [OK] Zero dynamic memory allocation (all buffers static)
00354 * - Rule 4: [OK] All functions < 60 lines
00355 * - Rule 5: [OK] Minimum 2 assertions per function via parameter validation
00356 * - Rule 6: [OK] Variables declared at smallest scope
00357 * - Rule 7: [OK] All return values checked and propagated
00358 * - Rule 8: [OK] Uses C23 typed enums exclusively (no magic numbers)
00359 * - Rule 9: [WARN] INTENTIONAL DEVIATION: Uses function pointers for callback (event-driven architecture)
00360 * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror (zero warnings)
00361 *
00362 * @par SOLID Principles:
00363 * - **S (Single Responsibility):** Only channel coordination and frame routing. Protocol parsing delegated to rx_frame,
transport to rx_usb_comm/rx_spi_comm.
00364 * - **O (Open/Closed):** Extensible to new channels by adding enum value and transport handle. Closed for modification
(no changes needed for new frame types).
00365 * - **L (Liskov Substitution):** USB and SPI transports interchangeable via common frame interface. Can substitute
mock transports for testing.
00366 * - **I (Interface Segregation):** Clean separation: lifecycle (init/deinit), polling (poll), sending (send/respond), status
(channel_ready). Users include only what they need.
00367 * - **D (Dependency Inversion):** Depends on abstractions (rx_frame protocol, generic transport interface) not concrete
USB/SPI implementations.
00368 *
00369 * @warning Do NOT call manager functions from within frame callback (recursion risk)
00370 * @warning ASCII buffer is 2048 bytes - ensure payload_len + overhead < 2000 bytes to avoid truncation
00371 * @warning Polling from multiple threads without synchronization will cause corruption
00372 *
00373 * @attention Manager does not own USB/SPI handles - caller must initialize transports before manager
00374 * @attention Callback is invoked from poll() context - keep processing minimal or defer to task queue
00375 * @attention Decoded ASCII output adds ~80 us latency per frame - disable for high-frequency applications
00376 *
00377 * @author STAR Team
00378 * @date 2026-01-27
00379 * @version 1.0.0
00380 * @copyright MIT License
00381 *
00382 * @par Changelog:
00383 * - 1.0.0 (2026-01-26): Initial implementation with USB and SPI channels
00384 * - 1.1.0 (2026-01-27): Added comprehensive Doxygen documentation, state machine diagrams, performance data
00385 */
00386
00387 #pragma once
00388
00389 #include <stdbool.h>
00390 #include <stdint.h>
00391
00392 #include "rx_err.h"
00393 #include "rx_frame.h"
00394 #include "rx_spi_comm.h"
00395 #include "rx_spi_link.h"
00396 #include "rx_usb_comm.h"
00397
00398 #ifdef __cplusplus
00399 extern "C" {
00400 #endif
00401
00402 /*
=====
00403 * Constants
00404 *
=====
00405 */
00406
00407 /**
00408 * @enum rx_comm_manager_constants_t
00409 * @brief Communication manager compile-time constants
00410 *
00411 * @details
00412 * Defines buffer sizes and limits for the communication manager. These
00413 * constants are chosen based on typical frame sizes and ASCII formatting
00414 * requirements.
00415 *
00416 * **Design Rationale:**
00417 * - **2048-byte ASCII buffer:** Accommodates maximum frame size (1024 bytes payload)
plus formatting overhead (~1000 bytes for hex dump + headers)
00418 * - Buffer is stack-allocated in handle structure (not dynamically allocated)
00419 * - Sufficient for debugging without excessive memory usage
00420 *
00421 * @see rx_comm_manager_t.ascii_buffer
00422 * @since Version 1.0.0
00423 */

```

```

00424 */
00425 typedef enum : uint16_t {
00426 /**
00427  * @brief ASCII formatting buffer size in bytes
00428  * @details
00429  * Size of internal buffer used to format decoded ASCII output for debug port.
00430  * Must accommodate:
00431  * - Header line: ~80 bytes ('[USB] TYPE=cmd FLAGS=0x00 LEN=1024')
00432  * - Payload hex dump: payload_len * 3 bytes ('XX ` per byte)
00433  * - Footer + null terminator: ~20 bytes
00434  *
00435  * Maximum frame payload: 1024 bytes
00436  * Required buffer: 80 + (1024 * 3) + 20 = 3172 bytes
00437  * Allocated: 2048 bytes (limits payload display to ~650 bytes, which is acceptable)
00438  *
00439  * @par Value: 2048 bytes
00440  * @par Rationale: Balance between functionality and memory usage
00441  */
00442 k_comm_manager_ascii_buf_size = 2048,
00443
00444 /**
00445  * @brief Maximum number of entries in the event FIFO queue
00446  * @details
00447  * Static FIFO queue depth for event (command) frames that require reliable
00448  * delivery. Events are queued and sent with SPI-only retry + exponential
00449  * backoff. Queue is statically allocated (zero dynamic memory).
00450  *
00451  * @par Value: 8 entries
00452  * @par Rationale: Small queue prevents unbounded memory, sufficient for
00453  * typical command burst scenarios
00454  */
00455 k_comm_event_queue_depth = 8,
00456 } rx_comm_manager_constants_t;
00457
00458 /**
00459  * @enum rx_comm_heartbeat_constants_t
00460  * @brief Heartbeat timing constants
00461  *
00462  * @details
00463  * Dual-detection heartbeat model:
00464  * - **Implicit timeout (200ms)**: Any valid frame resets the timer. If no
00465  * frames arrive within 200ms, the link is declared dead.
00466  * - **Explicit PING (1000ms)**: Sent when link is idle for >1s as a probe.
00467  * Firmware primarily receives PINGs from the gateway.
00468  * - **Check interval (50ms)**: How often to evaluate heartbeat status.
00469  *
00470  * @since Version 1.0.0
00471  */
00472 typedef enum : uint32_t {
00473 k_heartbeat_implicit_timeout_ms = 200, /**< Declare link dead if no frame for 200ms */
00474 k_heartbeat_ping_interval_ms = 1000, /**< Send PING probe when idle for 1s */
00475 k_heartbeat_check_interval_ms = 50, /**< Heartbeat evaluation interval (200ms / 4) */
00476 } rx_comm_heartbeat_constants_t;
00477
00478 /**
00479  * @enum rx_comm_event_retry_constants_t
00480  * @brief Event queue retry constants for SPI HARQ
00481  *
00482  * @details
00483  * Events (commands) sent via SPI use retry with exponential backoff.
00484  * USB sends are fire-and-forget (no retries).
00485  *
00486  * @since Version 1.0.0
00487  */
00488 typedef enum : uint16_t {
00489 k_event_max_retries = 3, /**< Maximum SPI retry attempts */
00490 k_event_initial_backoff_ms = 10, /**< Initial backoff delay (ms) */
00491 k_event_max_backoff_ms = 100, /**< Maximum backoff delay (ms) */
00492 } rx_comm_event_retry_constants_t;
00493
00494 /**
=====
00495  * Types
00496  *
=====
00497  */
00498
00499 /**
00500  * @enum rx_comm_channel_t
00501  * @brief Communication channel identifiers
00502  *
00503  * @details
00504  * Enumerates the available communication channels. The manager can handle
00505  * multiple channels simultaneously, with each channel operating independently.
00506  *
00507  * **Channel Characteristics:**
00508  *

```

```

00509 * | Channel | Bandwidth | Latency | Setup Complexity | Use Case |
00510 * |-----|-----|-----|-----|-----|
00511 * | USB CDC | ~1 MB/s | ~1-5 ms | Low (plug-and-play) | Commands, telemetry |
00512 * | SPI | ~2 MB/s | ~0.1-1 ms | Medium (requires wiring) | High-speed data, control |
00513 *
00514 * @par Usage Pattern:
00515 * @code{.c}
00516 * // Check which channel frame came from
00517 * void callback(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00518 *     if (channel == k_comm_channel_usb) {
00519 *         // Received via USB
00520 *     } else if (channel == k_comm_channel_spi) {
00521 *         // Received via SPI
00522 *     }
00523 * }
00524 * @endcode
00525 *
00526 * @see rx_comm_manager_channel_name() Get human-readable channel name
00527 * @see rx_comm_frame_callback_t Callback receives channel identifier
00528 * @since Version 1.0.0
00529 */
00530 typedef enum : uint8_t {
00531 /**
00532  * @brief USB CDC channel (Port 0 - protocol)
00533  * @details
00534  * Primary USB serial port used for frame protocol communication.
00535  * Maps to `/dev/ttyACM0` on Raspberry Pi 5.
00536  *
00537  * **Characteristics:**
00538  * - Bandwidth: ~1 MB/s practical (12 Mbps full-speed USB theoretical)
00539  * - Latency: 1-5 ms (depends on USB polling interval)
00540  * - Setup: Plug-and-play, automatic driver loading
00541  * - Reliability: Good (USB protocol has error detection)
00542  *
00543  * @par Hardware: USB full-speed peripheral (RX72N USB module)
00544  * @par Value: 0
00545  */
00546 k_comm_channel_usb = 0,
00547
00548 /**
00549  * @brief SPI peripheral channel
00550  * @details
00551  * Dedicated SPI connection for high-speed, low-latency communication.
00552  * Requires explicit wiring between RX72N RSPI and RPi5 SPI peripheral.
00553  *
00554  * **Characteristics:**
00555  * - Bandwidth: ~2 MB/s @ 20 MHz clock
00556  * - Latency: 0.1-1 ms (depends on polling frequency)
00557  * - Setup: Requires 4-6 wire connections + configuration
00558  * - Reliability: Good (frame protocol includes CRC-32)
00559  *
00560  * **Pin Connections:**
00561  * - COPI: RX72N -> RPi5 (data out from RX72N)
00562  * - CIPO: RPi5 -> RX72N (data in to RX72N)
00563  * - SCK: Clock (either device can be controller)
00564  * - CS: Chip select (active low)
00565  * - Optional: Handshake/IRQ pins for flow control
00566  *
00567  * @par Hardware: RSPI peripheral (Renesas Serial Peripheral Interface)
00568  * @par Value: 1
00569  */
00570 k_comm_channel_spi = 1,
00571
00572 /**
00573  * @brief Total number of supported channels
00574  * @details
00575  * Compile-time constant indicating channel count. Used for array sizing
00576  * and loop bounds checking (NASA Rule 2 compliance).
00577  *
00578  * **Usage in loops:**
00579  * @code{.c}
00580  * for (uint8_t i = 0; i < k_comm_channel_count; i++) {
00581  *     // Process each channel
00582  * }
00583  * @endcode
00584  *
00585  * @par Value: 2 (USB + SPI)
00586  * @par Invariant: Must equal number of enum values (excluding this one)
00587  */
00588 k_comm_channel_count = 2,
00589 } rx_comm_channel_t;
00590
00591 /**
00592  * @typedef rx_comm_frame_callback_t
00593  * @brief Frame received callback function type
00594  *
00595  * @details

```

```

00596 * User-defined function invoked by communication manager when a complete,
00597 * valid frame is received on any enabled channel. The callback receives:
00598 * - Channel identifier (which channel the frame arrived on)
00599 * - Parsed frame structure (type, flags, payload)
00600 * - User context pointer (passed during initialization)
00601 *
00602 * ## Callback Contract
00603 *
00604 * **Responsibilities:**
00605 * - Parse frame payload based on frame type
00606 * - Handle command execution or data processing
00607 * - Optionally send response via `rx_comm_manager_respond()`
00608 * - Return quickly to avoid blocking communication loop
00609 *
00610 * **Restrictions:**
00611 * - Do NOT call `rx_comm_manager_poll()` from callback (recursion!)
00612 * - Keep processing minimal (~50 us recommended)
00613 * - For slow operations, queue to task and return immediately
00614 * - Do NOT modify frame structure (it may be const or recycled)
00615 *
00616 * **Thread Safety:**
00617 * - Callback executed in context of `poll()` caller
00618 * - If poll() called from RTOS task, callback runs in that task context
00619 * - Synchronization required if accessing shared resources
00620 *
00621 * @param[in] channel The communication channel the frame was received on
00622 * - Type: rx_comm_channel_t
00623 * - Valid values: k_comm_channel_usb or k_comm_channel_spi
00624 * - Use to determine response routing
00625 *
00626 * @param[in] frame Pointer to the received and validated frame
00627 * - Type: const rx_frame_t*
00628 * - Valid until callback returns
00629 * - Contains: type, flags, payload, payload_len (CRC already validated)
00630 * - Do NOT modify or store pointer (frame may be recycled)
00631 *
00632 * @param[in] ctx User context pointer
00633 * - Type: void* (application-defined)
00634 * - Value passed during `rx_comm_manager_init()`
00635 * - Typically pointer to manager, application state, or nullptr
00636 * - Use to access application resources without globals
00637 *
00638 * @par Usage Example - Basic Command Handler:
00639 * @code{.c}
00640 * void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00641 *     rx_comm_manager_t* mgr = (rx_comm_manager_t*)ctx;
00642 *
00643 *     // Log reception
00644 *     rx_log_info("COMM", "RX from %s: type=0x%02X len=%u",
00645 *         rx_comm_manager_channel_name(channel),
00646 *         frame->type, frame->payload_len);
00647 *
00648 *     // Dispatch based on type
00649 *     switch (frame->type) {
00650 *     case k_frame_type_command:
00651 *         handle_command(mgr, channel, frame);
00652 *         break;
00653 *     case k_frame_type_telemetry_req:
00654 *         send_telemetry(mgr, channel);
00655 *         break;
00656 *     default:
00657 *         rx_log_warn("COMM", "Unknown type: 0x%02X", frame->type);
00658 *         break;
00659 *     }
00660 * }
00661 * @endcode
00662 *
00663 * @par Usage Example - Deferred Processing:
00664 * @code{.c}
00665 * // For time-consuming operations, defer to queue
00666 * void on_frame(rx_comm_channel_t channel, const rx_frame_t* frame, void* ctx) {
00667 *     app_state_t* app = (app_state_t*)ctx;
00668 *
00669 *     // Copy frame to queue (frame pointer only valid during callback)
00670 *     command_msg_t msg = {
00671 *         .channel = channel,
00672 *         .type = frame->type,
00673 *         .payload_len = frame->payload_len,
00674 *     };
00675 *     memcpy(msg.payload, frame->payload, frame->payload_len);
00676 *
00677 *     // Queue for processing by command handler task
00678 *     UINT status = tx_queue_send(&app->cmd_queue, &msg, TX_NO_WAIT);
00679 *     if (status != TX_SUCCESS) {
00680 *         rx_log_error("COMM", "Command queue full!");
00681 *     }
00682 * }

```

```

00683 * @endcode
00684 *
00685 * @warning Callback must return quickly (<50 us recommended) to avoid blocking communication
00686 * @warning Do NOT call rx_comm_manager_poll() from callback (causes recursion)
00687 * @warning Frame pointer only valid during callback - copy data if needed beyond callback scope
00688 *
00689 * @see rx_comm_manager_respond() Send response on same channel
00690 * @see rx_comm_manager_send() Send frame on specific channel
00691 * @see rx_frame_t Frame structure definition
00692 *
00693 * @since Version 1.0.0
00694 */
00695 typedef void (*rx_comm_frame_callback_t)(rx_comm_channel_t channel,
00696                                         const rx_frame_t* frame,
00697                                         void* ctx);
00698
00699 /**
00700 * @enum rx_comm_link_status_t
00701 * @brief Link health status from heartbeat monitoring
00702 *
00703 * @details
00704 * Reported per-transport link health. Determined by the dual-detection
00705 * heartbeat model (implicit 200ms timeout + explicit 1s PING).
00706 *
00707 * @since Version 1.0.0
00708 */
00709 typedef enum : uint8_t {
00710     k_link_status_unknown = 0, /**< No frames received yet */
00711     k_link_status_healthy = 1, /**< Frames received within implicit timeout */
00712     k_link_status_dead = 2, /**< No frames within implicit timeout (200ms) */
00713 } rx_comm_link_status_t;
00714
00715 /**
00716 * @struct rx_comm_event_entry_t
00717 * @brief Single entry in the static event FIFO queue
00718 *
00719 * @details
00720 * Events (commands) that require reliable delivery are queued here.
00721 * Each entry contains a copy of the payload and metadata needed for
00722 * retry logic. SPI entries retry with exponential backoff; USB entries
00723 * are fire-and-forget.
00724 *
00725 * @since Version 1.0.0
00726 */
00727 typedef struct {
00728     rx_comm_channel_t channel;                /**< Target transport */
00729     rx_frame_type_t type;                    /**< Frame type */
00730     uint8_t flags;                          /**< Frame flags */
00731     uint8_t payload[k_frame_max_payload];    /**< Payload copy */
00732     uint32_t payload_len;                    /**< Payload length (bytes) */
00733     uint8_t retries;                        /**< Retry attempts so far */
00734     uint32_t next_retry_time_ms;            /**< Earliest time (ms) for next retry */
00735     bool occupied;                          /**< Slot in use */
00736 } rx_comm_event_entry_t;
00737
00738 /**
00739 * @struct rx_comm_heartbeat_state_t
00740 * @brief Per-transport heartbeat tracking state
00741 *
00742 * @details
00743 * Tracks the last time a valid frame was received on each transport.
00744 * Used by the poll loop to evaluate link health per the dual-detection
00745 * heartbeat model.
00746 *
00747 * @since Version 1.0.0
00748 */
00749 typedef struct {
00750     uint32_t last_rx_ms; /**< Timestamp (ms) of last valid frame */
00751     rx_comm_link_status_t status; /**< Current link health */
00752 } rx_comm_heartbeat_state_t;
00753
00754 /**
00755 * @typedef rx_comm_link_status_callback_t
00756 * @brief Callback invoked when a link status changes
00757 *
00758 * @param[in] channel Which transport changed status
00759 * @param[in] new_status New link status
00760 * @param[in] ctx User context pointer
00761 *
00762 * @since Version 1.0.0
00763 */
00764 typedef void (*rx_comm_link_status_callback_t)(rx_comm_channel_t channel,
00765                                                rx_comm_link_status_t new_status,
00766                                                void* ctx);
00767
00768 /**
00769 * @struct rx_comm_manager_config_t

```

```

00770 * @brief Communication manager configuration
00771 *
00772 * @details
00773 * Configuration structure passed to `rx_comm_manager_init()`. Defines which
00774 * channels are enabled, callback function, and optional features.
00775 *
00776 * **Configuration Guidelines:**
00777 * - At least one channel (usb_handle OR spi_handle) should be non-NULL
00778 * - Both channels can be enabled simultaneously
00779 * - Callback is optional (NULL = receive but don't notify)
00780 * - Decoded output recommended for development, disable for production
00781 *
00782 * **Memory Lifetime:**
00783 * - Config structure only read during init, can be stack-allocated
00784 * - Transport handles must remain valid for manager lifetime
00785 * - Callback function must remain valid (typically static function)
00786 *
00787 * @par Configuration Example:
00788 * @code{.c}
00789 * rx_comm_manager_config_t cfg = {
00790 *     .usb_handle = &usb_comm,           // Enable USB channel
00791 *     .spi_handle = &spi_comm,           // Enable SPI channel
00792 *     .callback = on_frame_received,     // Frame handler
00793 *     .callback_ctx = &app_state,       // User context
00794 *     .enable_decoded_output = true,     // ASCII debug output
00795 * };
00796 * @endcode
00797 *
00798 * @see rx_comm_manager_init() Initialization function
00799 * @since Version 1.0.0
00800 */
00801 typedef struct {
00802     /**
00803      * @brief USB communication handle (NULL to disable USB channel)
00804      * @details
00805      * Pointer to initialized USB CDC transport handle. If NULL, USB channel
00806      * is disabled and all USB operations are skipped.
00807      *
00808      * **Requirements:**
00809      * - Must be initialized via `rx_usb_comm_init()` before passing to manager
00810      * - Must remain valid for lifetime of manager
00811      * - Handle must be configured with:
00812      *   - Port 0: Protocol channel (frame communication)
00813      *   - Port 1: Debug channel (ASCII output, if decoded output enabled)
00814      *
00815      * **Null handling:**
00816      * - NULL = USB channel disabled (valid configuration)
00817      * - Non-NULL = USB channel enabled
00818      *
00819      * @par Type: rx_usb_comm_handle_t* (pointer to USB transport handle)
00820      * @par Valid values: Non-NULL initialized handle, or nullptr to disable
00821      * @par Lifetime: Must outlive manager instance
00822      */
00823     rx_usb_comm_handle_t* usb_handle;
00824
00825     /**
00826      * @brief SPI communication handle (NULL to disable SPI channel)
00827      * @details
00828      * Pointer to initialized SPI transport handle. If NULL, SPI channel
00829      * is disabled and all SPI operations are skipped.
00830      *
00831      * **Requirements:**
00832      * - Must be initialized via `rx_spi_comm_init()` before passing to manager
00833      * - Must remain valid for lifetime of manager
00834      * - Requires hardware SPI connection between RX72N and RPi5
00835      *
00836      * **Null handling:**
00837      * - NULL = SPI channel disabled (valid configuration)
00838      * - Non-NULL = SPI channel enabled
00839      *
00840      * @par Type: rx_spi_comm_handle_t* (pointer to SPI transport handle)
00841      * @par Valid values: Non-NULL initialized handle, or nullptr to disable
00842      * @par Lifetime: Must outlive manager instance
00843      */
00844     rx_spi_comm_handle_t* spi_handle;
00845
00846     /**
00847      * @brief Optional SPI link layer with HARQ (NULL to use raw SPI)
00848      * @details
00849      * When non-NULL, SPI send/receive operations route through the link
00850      * layer, which provides FEC encoding/decoding, Chase Combining, and
00851      * ACK/NACK handshake. When NULL, SPI operates in raw mode (frame +
00852      * CRC-32 only, no FEC/HARQ).
00853      *
00854      * **Requirements:**
00855      * - Must be initialized via rx_spi_link_init() before passing to manager
00856      * - spi_link->spi_handle MUST be the same as config->spi_handle

```

```

00857 * - Must remain valid for lifetime of manager
00858 *
00859 * @invariant If spi_link is non-NULL: spi_link->spi_handle == config->spi_handle
00860 * @invariant If spi_link is non-NULL: spi_link->initialized == true
00861 * @invariant If spi_link is non-NULL: spi_link must outlive manager instance
00862 *
00863 * @par Valid values: Initialized rx_spi_link_t*, or nullptr for raw SPI
00864 * @par Lifetime: Must outlive manager instance
00865 *
00866 * @see rx_spi_link.h SPI link layer API
00867 * @see rx_spi_link_init() Must be called before passing spi_link to manager
00868 * @since Version 1.0.0
00869 */
00870 rx_spi_link_t* spi_link;
00871
00872 /**
00873 * @brief Frame received callback function (NULL for no callback)
00874 * @details
00875 * User-defined function invoked when complete frame received on any channel.
00876 * If NULL, frames are received but no notification occurs (not typical).
00877 *
00878 * **Null handling:**
00879 * - NULL = Receive frames but don't notify (unusual, for testing only)
00880 * - Non-NULL = Invoke callback on frame reception (normal operation)
00881 *
00882 * @par Type: rx_comm_frame_callback_t (function pointer)
00883 * @par Valid values: Non-NULL function pointer, or nullptr to disable
00884 * @par Signature: `void callback(rx_comm_channel_t, const rx_frame_t*, void*)`
00885 *
00886 * @see rx_comm_frame_callback_t Callback function type definition
00887 */
00888 rx_comm_frame_callback_t callback;
00889
00890 /**
00891 * @brief User context pointer passed to callback
00892 * @details
00893 * Arbitrary pointer passed to callback as third parameter. Typically used
00894 * to pass application state, manager pointer, or other context without
00895 * requiring global variables.
00896 *
00897 * **Common patterns:**
00898 * - Pointer to manager itself (enables response from callback)
00899 * - Pointer to application state structure
00900 * - NULL if no context needed
00901 *
00902 * @par Type: void* (application-defined)
00903 * @par Valid values: Any pointer value, or nullptr
00904 * @par Lifetime: Must remain valid if callback uses it
00905 */
00906 void* callback_ctx;
00907
00908 /**
00909 * @brief Enable decoded ASCII output to USB Port 1
00910 * @details
00911 * When true, all received and sent frames are converted to human-readable
00912 * ASCII format and sent to USB Port 1 (/dev/ttyACM1 on RPi5).
00913 *
00914 * **ASCII Format:**
00915 * ```
00916 * [USB] TYPE=cmd FLAGS=0x00 LEN=42 DATA=48656C6C6F...
00917 * [SPI] TYPE=resp FLAGS=0x01 LEN=128 DATA=...
00918 * ```
00919 *
00920 * **Performance Impact:**
00921 * - Adds ~80 us latency per frame (formatting + USB send)
00922 * - Negligible for command/response (< 100 Hz)
00923 * - May impact high-frequency telemetry (> 1 kHz)
00924 *
00925 * **Use Cases:**
00926 * - Development: Enable for debugging and protocol analysis
00927 * - Production: Disable for maximum performance
00928 *
00929 * @par Type: bool
00930 * @par Value: true = enable ASCII output, false = disable
00931 * @par Default: Recommend true for development, false for production
00932 */
00933 bool enable_decoded_output;
00934
00935 /**
00936 * @brief Callback invoked when a transport link status changes
00937 * @details
00938 * Optional. Called when heartbeat monitoring detects a link transition
00939 * (e.g., healthy -> dead). NULL to disable link status notifications.
00940 */
00941 rx_comm_link_status_callback_t link_status_cb;
00942
00943 /**

```

```

00944  * @brief User context for link_status_cb
00945  */
00946  void* link_status_ctx;
00947 } rx_comm_manager_config_t;
00948
00949 /**
00950  * @struct rx_comm_manager_t
00951  * @brief Communication manager handle
00952  *
00953  * @details
00954  * Opaque handle structure containing manager state and resources. Application
00955  * must allocate this structure (typically static or global) and pass to all
00956  * manager functions.
00957  *
00958  * **Initialization:**
00959  * Zero-initialize structure before passing to `rx_comm_manager_init()`:
00960  * @code{.c}
00961  * rx_comm_manager_t mgr = {0}; // Or use memset
00962  * @endcode
00963  *
00964  * **Memory Layout:** See file-level documentation for detailed breakdown.
00965  *
00966  * **Thread Safety:** Not thread-safe - protect with external mutex if accessed
00967  * from multiple threads.
00968  *
00969  * @warning Do not access fields directly - use API functions only
00970  * @warning Structure size ~2.5 KB - consider allocation carefully
00971  *
00972  * @see rx_comm_manager_init() Initialize handle
00973  * @see rx_comm_manager_deinit() Cleanup handle
00974  * @since Version 1.0.0
00975  */
00976 typedef struct {
00977     rx_usb_comm_handle_t* usb_handle; /**< USB comm handle */
00978     rx_spi_comm_handle_t* spi_handle; /**< SPI comm handle */
00979
00980     /**< @brief Optional SPI link layer with HARQ (NULL if disabled)
00981     * @details
00982     * When non-NULL, points to an rx_spi_link_t handle providing FEC
00983     * encoding/decoding, Chase Combining, and ACK/NACK handshake for SPI.
00984     * When NULL, SPI operates in raw mode (frame + CRC-32 only, no FEC/HARQ).
00985     *
00986     * **Ownership and Lifetime:**
00987     * - Pointer copied from config during rx_comm_manager_init()
00988     * - Manager does NOT own the link object - caller must allocate/deallocate
00989     * - Link object must remain valid for entire manager lifetime
00990     * - Link object must NOT be freed until after rx_comm_manager_deinit()
00991     * - If manager is deinitialized, spi_link remains valid but decoupled
00992     *
00993     * **Invariants/Constraints:**
00994     * - If non-NULL: Must point to initialized rx_spi_link_t (spi_link->initialized == true)
00995     * - If non-NULL: spi_link->spi_handle MUST equal manager's spi_handle
00996     * - If non-NULL: spi_link must outlive manager instance
00997     * - NULL is allowed to indicate no SPI HARQ (raw SPI mode)
00998     * - Fields inside spi_link object must remain stable while manager is active
00999     *
01000     * **Valid Ranges/Usage:**
01001     * - Set via config->spi_link during initialization
01002     * - NULL = raw SPI mode (valid configuration for testing or simple use cases)
01003     * - Non-NULL = HARQ mode (production mode with FEC and retransmission)
01004     * - Once set during init, this pointer is immutable (never modified by manager)
01005     *
01006     * **Thread Safety:**
01007     * - Manager does not perform synchronization on spi_link access
01008     * - If multiple threads call rx_comm_manager_poll(), caller must serialize
01009     * - spi_link internal state is protected by its own mechanisms (see rx_spi_link.h)
01010     * - Safe to read this pointer from multiple threads (read-only after init)
01011     *
01012     * @invariant If spi_link != NULL: spi_link->initialized == true
01013     * @invariant If spi_link != NULL: spi_link->spi_handle == this->spi_handle
01014     * @invariant If spi_link != NULL: spi_link must outlive this manager instance
01015     * @invariant Pointer value never changes after rx_comm_manager_init()
01016     *
01017     * @par Type: rx_spi_link_t* (pointer to HARQ link layer handle)
01018     * @par Valid values: Initialized rx_spi_link_t*, or nullptr for raw SPI mode
01019     * @par Lifetime: Must outlive manager instance (caller manages allocation)
01020     * @par Modification: Read-only after init (immutable pointer)
01021     * - Immutable after init (do not change while manager is active)
01022     * - Fields inside rx_spi_link_t must remain stable during use
01023     *
01024     * @see rx_spi_link.h SPI link layer API
01025     * @see rx_comm_manager_config_t::spi_link Configuration parameter
01026     * @since Version 1.0.0
01027     */
01028     rx_spi_link_t* spi_link;
01029
01030     rx_comm_frame_callback_t callback; /**< Frame callback */

```

```

01031 void*                callback_ctx;                /**< Callback context */
01032 char                ascii_buffer[k_comm_manager_ascii_buf_size]; /**< ASCII buffer */
01033 bool                enable_decoded_output;        /**< Enable ASCII output */
01034 bool                initialized;                /**< Initialization flag */
01035
01036 /** @brief Per-transport heartbeat tracking (indexed by rx_comm_channel_t) */
01037 rx_comm_heartbeat_state_t heartbeat[k_comm_channel_count];
01038
01039 /** @brief Link status change callback (optional) */
01040 rx_comm_link_status_callback_t link_status_cb;
01041
01042 /** @brief Link status callback context */
01043 void* link_status_ctx;
01044
01045 /** @brief Static event FIFO queue for reliable command delivery */
01046 rx_comm_event_entry_t event_queue[k_comm_event_queue_depth];
01047
01048 /** @brief Event queue write index (next slot to write) */
01049 uint8_t event_queue_head;
01050
01051 /** @brief Event queue read index (next slot to process) */
01052 uint8_t event_queue_tail;
01053
01054 /** @brief Number of occupied event queue slots */
01055 uint8_t event_queue_count;
01056 } rx_comm_manager_t;
01057
01058 /**
01059  * @struct rx_comm_send_params_t
01060  * @brief Parameters for rx_comm_manager_send
01061  *
01062  * @details
01063  * Wrapper structure providing type safety for send operations. Prevents
01064  * parameter confusion when passing multiple uint8_t/uint32_t values.
01065  *
01066  * **Design Rationale:**
01067  * Without this struct, function signature would be error-prone:
01068  * ```c
01069  * // BAD: Easy to mix up channel, type, flags (all similar types)
01070  * rx_err_t send(mgr, uint8_t ch, uint8_t type, uint8_t flags, ...);
01071  * ```
01072  *
01073  * With struct, parameters are self-documenting:
01074  * ```c
01075  * // GOOD: Clear naming prevents mistakes
01076  * rx_comm_send_params_t params = {
01077  *     .channel = k_comm_channel_usb,
01078  *     .type = k_frame_type_command,
01079  *     .flags = 0x00,
01080  *     // ...
01081  * };
01082  * ```
01083  *
01084  * @par Usage Example:
01085  * @code{.c}
01086  * rx_comm_send_params_t params = {
01087  *     .channel = k_comm_channel_spi,
01088  *     .type = k_frame_type_telemetry,
01089  *     .flags = 0,
01090  *     .payload = telemetry_data,
01091  *     .payload_len = sizeof(telemetry_data),
01092  * };
01093  * rx_comm_manager_send(&mgr, &params);
01094  * @endcode
01095  *
01096  * @see rx_comm_manager_send() Send function using this structure
01097  * @since Version 1.0.0
01098  */
01099 typedef struct {
01100     rx_comm_channel_t channel; /**< Channel to send on (USB or SPI) */
01101     rx_frame_type_t type; /**< Frame type (command, telemetry, etc.) */
01102     uint8_t flags; /**< Frame flags (protocol-specific) */
01103     const uint8_t* payload; /**< Payload data pointer */
01104     uint32_t payload_len; /**< Payload length in bytes */
01105 } rx_comm_send_params_t;
01106
01107 /*
=====
01108  * Lifecycle API
01109  *
=====
01110  */
01111
01112 /**
01113  * @brief Initialize communication manager
01114  *
01115  * @details

```

```

01116 * Initializes the communication manager with the provided configuration.
01117 * Validates configuration parameters and sets up internal state for
01118 * USB and/or SPI channel operation.
01119 *
01120 * @param[out] mgr Pointer to manager handle
01121 * @param[in]  cfg Configuration (NULL for defaults, both channels disabled)
01122 *
01123 * @return k_rx_ok on success
01124 * @return k_rx_err_invalid_arg if mgr is nullptr
01125 * @return k_rx_err_invalid_arg if spi_link is non-NULL but not initialized
01126 * @return k_rx_err_invalid_arg if spi_link->spi_handle != cfg->spi_handle
01127 *
01128 * @pre mgr must not be nullptr
01129 * @pre If cfg->spi_link is non-NULL, spi_link must be initialized via rx_spi_link_init()
01130 * @pre If cfg->spi_link is non-NULL, spi_link->spi_handle must equal cfg->spi_handle
01131 * @pre cfg may be nullptr (results in default configuration with both channels disabled)
01132 *
01133 * @post mgr->usb_handle == cfg->usb_handle (or nullptr if cfg is nullptr)
01134 * @post mgr->spi_handle == cfg->spi_handle (or nullptr if cfg is nullptr)
01135 * @post mgr->spi_link == cfg->spi_link (or nullptr if cfg is nullptr or cfg->spi_link is nullptr)
01136 * @post All internal state initialized (event queue empty, heartbeat timers reset)
01137 *
01138 * @note Thread-safe: May be called from any thread, but concurrent calls with same mgr are prohibited
01139 * @warning Must be called before any other manager operations on this handle
01140 *
01141 * @see rx_spi_link_init() Must call this before passing spi_link to manager
01142 * @see rx_comm_manager_deinit() Cleanup function
01143 *
01144 * @since Version 1.0.0
01145 */
01146 [[nodiscard]] rx_err_t rx_comm_manager_init(rx_comm_manager_t* mgr,
01147                                             const rx_comm_manager_config_t* cfg);
01148
01149 /**
01150 * @brief Deinitialize communication manager
01151 *
01152 * Does not deinitialize the underlying USB/SPI handles.
01153 *
01154 * @param[in,out] mgr Pointer to manager handle
01155 * @return k_rx_ok on success
01156 */
01157 [[nodiscard]] rx_err_t rx_comm_manager_deinit(rx_comm_manager_t* mgr);
01158
01159 /*
01160 * Polling API
01161 */
01162 */
01163 /**
01164 * @brief Poll all enabled channels for incoming frames
01165 *
01166 * Non-blocking check of USB and SPI channels. For each complete frame received:
01167 * 1. If decoded output enabled, format and send ASCII to Port 1
01168 * 2. Invoke user callback with frame data and channel
01169 *
01170 * Call this from main loop or dedicated communication task.
01171 *
01172 * @param[in,out] mgr Pointer to initialized manager
01173 *
01174 * @return k_rx_ok if at least one frame was received
01175 * @return k_rx_err_timeout if no frames available on any channel
01176 * @return k_rx_err_invalid_arg if mgr is nullptr
01177 * @return k_rx_err_invalid_state if not initialized
01178 */
01179 */
01180 [[nodiscard]] rx_err_t rx_comm_manager_poll(rx_comm_manager_t* mgr);
01181
01182 /*
01183 * Send API
01184 */
01185 */
01186 /**
01187 * @brief Send frame on specific channel
01188 *
01189 * Encodes and sends a frame on the specified channel. If decoded output
01190 * is enabled, also sends ASCII representation to Port 1.
01191 *
01192 * @param[in,out] mgr Pointer to initialized manager
01193 * @param[in]     params Send parameters (channel, type, flags, payload, payload_len)
01194 *
01195 * @return k_rx_ok on success
01196 * @return k_rx_err_invalid_arg if mgr or payload is nullptr
01197 * @return k_rx_err_invalid_state if not initialized or channel not enabled
01198 */

```

```

01199 */
01200 [[nodiscard]] rx_err_t rx_comm_manager_send(rx_comm_manager_t* mgr,
01201                                             const rx_comm_send_params_t* params);
01202
01203 /**
01204  * @brief Send response on same channel as received frame
01205  *
01206  * Convenience function for responding to commands. Sends a RESPONSE type
01207  * frame on the specified channel.
01208  *
01209  * @param[in,out] mgr      Pointer to initialized manager
01210  * @param[in]     channel  Channel to respond on (from callback)
01211  * @param[in]     payload  Response payload data
01212  * @param[in]     payload_len Response payload length
01213  *
01214  * @return k_rx_ok on success
01215  */
01216 [[nodiscard]] rx_err_t rx_comm_manager_respond(rx_comm_manager_t* mgr,
01217                                                rx_comm_channel_t channel,
01218                                                const uint8_t* payload,
01219                                                uint32_t payload_len);
01220
01221 /*
=====
01222 * Stream / Event Send API
01223 *
=====
01224 */
01225
01226 /**
01227  * @brief Send data on the stream path (fire-and-forget)
01228  *
01229  * @details
01230  * Stream path is for time-sensitive data like telemetry where stale data
01231  * has no value. Sends immediately with NO retries on any transport.
01232  * If the send fails, the data is simply dropped.
01233  *
01234  * - USB CDC: Direct send, no retries (USB HW provides reliability)
01235  * - SPI: Direct send, no retries (stale telemetry is worthless)
01236  *
01237  * @param[in,out] mgr      Pointer to initialized manager
01238  * @param[in]     params   Send parameters (channel, type, flags, payload, payload_len)
01239  *
01240  * @return k_rx_ok on success
01241  * @return k_rx_err_invalid_arg if mgr or params is nullptr
01242  * @return k_rx_err_invalid_state if not initialized or channel not enabled
01243  *
01244  * @pre mgr must be initialized
01245  * @pre params must be valid
01246  * @post Data sent or silently dropped on failure
01247  *
01248  * @note This is a thin wrapper over rx_comm_manager_send() for semantic clarity
01249  *
01250  * @see rx_comm_manager_event_send() For reliable command delivery
01251  * @since Version 1.0.0
01252  */
01253 [[nodiscard]] rx_err_t rx_comm_manager_stream_send(rx_comm_manager_t* mgr,
01254                                                    const rx_comm_send_params_t* params);
01255
01256 /**
01257  * @brief Queue data on the event path (reliable, SPI retry)
01258  *
01259  * @details
01260  * Event path is for commands and other data requiring reliable delivery.
01261  * The payload is copied into a static FIFO queue and processed by the
01262  * poll loop. Retry behavior depends on transport:
01263  *
01264  * - **SPI**: Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
01265  * - **USB**: Fire-and-forget (USB HW reliability is sufficient)
01266  *
01267  * Queue is processed in FIFO order by rx_comm_manager_poll().
01268  *
01269  * @param[in,out] mgr      Pointer to initialized manager
01270  * @param[in]     params   Send parameters (channel, type, flags, payload, payload_len)
01271  *
01272  * @return k_rx_ok on success (queued)
01273  * @return k_rx_err_invalid_arg if mgr or params is nullptr
01274  * @return k_rx_err_invalid_state if not initialized
01275  * @return k_rx_err_no_mem if event queue is full
01276  *
01277  * @pre mgr must be initialized
01278  * @pre params must be valid
01279  * @pre params->payload_len <= k_frame_max_payload
01280  * @post Payload copied to queue, will be sent by next poll()
01281  *
01282  * @note Payload is COPIED into queue - caller can reuse buffer immediately
01283  * @warning Queue depth is k_comm_event_queue_depth (8). If full, returns error.

```

```

01284 *
01285 * @see rx_comm_manager_stream_send() For fire-and-forget telemetry
01286 * @since Version 1.0.0
01287 */
01288 [[nodiscard]] rx_err_t rx_comm_manager_event_send(rx_comm_manager_t* mgr,
01289                                                    const rx_comm_send_params_t* params);
01290
01291 /*
=====
01292 * Heartbeat API
01293 *
=====
01294 */
01295
01296 /**
01297 * @brief Query link health status for a transport channel
01298 *
01299 * @details
01300 * Returns the current heartbeat-derived link status for the given channel.
01301 * Status is updated by rx_comm_manager_poll() based on the dual-detection
01302 * heartbeat model (200ms implicit timeout).
01303 *
01304 * @param[in] mgr Pointer to initialized manager
01305 * @param[in] channel Channel to query
01306 * @param[out] status Receives current link status
01307 *
01308 * @return k_rx_ok on success
01309 * @return k_rx_err_invalid_arg if mgr or status is nullptr
01310 *
01311 * @since Version 1.0.0
01312 */
01313 [[nodiscard]] rx_err_t rx_comm_manager_link_status(const rx_comm_manager_t* mgr,
01314                                                    rx_comm_channel_t channel,
01315                                                    rx_comm_link_status_t* status);
01316
01317 /*
=====
01318 * Status API
01319 *
=====
01320 */
01321
01322 /**
01323 * @brief Check if channel is ready for communication
01324 *
01325 * @param[in] mgr Pointer to initialized manager
01326 * @param[in] channel Channel to check
01327 * @param[out] ready Set to true if channel is ready
01328 *
01329 * @return k_rx_ok on success
01330 * @return k_rx_err_invalid_arg if mgr or ready is nullptr
01331 */
01332 [[nodiscard]] rx_err_t
01333 rx_comm_manager_channel_ready(rx_comm_manager_t* mgr, rx_comm_channel_t channel, bool* ready);
01334
01335 /**
01336 * @brief Get channel name string
01337 *
01338 * @param[in] channel Channel identifier
01339 * @return Human-readable channel name
01340 */
01341 const char* rx_comm_manager_channel_name(rx_comm_channel_t channel);
01342
01343 /*
=====
01344 * Retransmission Pass-Through
01345 *
=====
01346 */
01347
01348 /**
01349 * @brief Process pending retransmissions on SPI channel
01350 *
01351 * @details
01352 * Thin pass-through to rx_spi_comm_process_retransmits() on the SPI handle.
01353 * Call this from the communication task polling loop.
01354 *
01355 * @param[in,out] mgr Communication manager
01356 * @param[in] current_time_ms Current system time in milliseconds
01357 *
01358 * @return rx_err_t Error code from rx_spi_comm_process_retransmits()
01359 * @retval k_rx_ok No action needed or retransmit sent
01360 * @retval k_rx_err_invalid_arg mgr is nullptr
01361 * @retval k_rx_err_invalid_state mgr not initialized
01362 * @retval k_rx_err_retry_limit Max retries exceeded
01363 *
01364 * @since Version 1.0.0

```

```

01365 */
01366 [[nodiscard]] rx_err_t rx_comm_manager_process_retransmits(rx_comm_manager_t* mgr,
01367                                                             uint32_t          current_time_ms);
01368
01369 /**
01370  * @brief Enable or disable automatic retransmission on SPI channel
01371  *
01372  * @details
01373  * Thin pass-through to rx_spi_comm_set_auto_retransmit() on the SPI handle.
01374  *
01375  * @param[in,out] mgr Communication manager
01376  * @param[in] enabled true to enable, false to disable
01377  * @param[in] config Retransmit config (NULL to keep current/defaults)
01378  *
01379  * @return rx_err_t Error code from rx_spi_comm_set_auto_retransmit()
01380  *
01381  * @since Version 1.0.0
01382  */
01383 [[nodiscard]] rx_err_t
01384 rx_comm_manager_set_auto_retransmit(rx_comm_manager_t* mgr,
01385                                     bool                enabled,
01386                                     const rx_spi_comm_retransmit_config_t* config);
01387
01388 #ifdef __cplusplus
01389 }
01390 #endif

```

4.3 libs/rx_comm_manager/src/rx_comm_manager.c File Reference

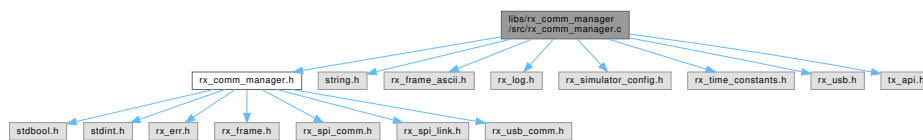
Unified Multi-Channel Communication Manager Implementation.

```

#include "rx_comm_manager.h"
#include <string.h>
#include "rx_frame_ascii.h"
#include "rx_log.h"
#include "rx_simulator_config.h"
#include "rx_time_constants.h"
#include "rx_usb.h"
#include "tx_api.h"

```

Include dependency graph for rx_comm_manager.c:



Enumerations

- enum `rx_comm_sim_time_t`: `uint32_t { k_sim_time_ms_zero = 0 }`
Get current time in milliseconds.
- enum `internal_constants_t`: `uint32_t { k_receive_timeout_ms = 0, k_frame_seq_placeholder = 0, k_frame_crc_placeholder = 0, k_ascii_buffer_first_idx = 0, k_zero_u32 = 0 }`
Internal constants for communication manager implementation.

Functions

- static uint32_t `internal_get_time_ms` (void)
- static void `internal_output_decoded` (rx_comm_manager_t *mgr, const rx_frame_t *frame, bool is_tx)
Output decoded ASCII frame to USB Port 1 for debugging.
- static void `internal_handle_frame` (rx_comm_manager_t *mgr, rx_comm_channel_t channel, const rx_frame_t *frame)
Handle received frame by invoking user callback and optional ASCII output.
- static rx_err_t `internal_poll_usb` (rx_comm_manager_t *mgr)
Poll USB channel for incoming frames (non-blocking).
- static rx_err_t `internal_poll_spi` (rx_comm_manager_t *mgr)
Poll SPI channel for incoming frames (non-blocking).
- static void `internal_process_event_queue` (rx_comm_manager_t *mgr)
Process one event from the FIFO queue.
- static void `internal_check_heartbeat` (rx_comm_manager_t *mgr)
Check heartbeat implicit timeout on all transports.
- rx_err_t `rx_comm_manager_init` (rx_comm_manager_t *mgr, const rx_comm_manager_config_t *cfg)
Initialize communication manager with channel configuration.
- rx_err_t `rx_comm_manager_deinit` (rx_comm_manager_t *mgr)
Deinitialize communication manager and release resources.
- rx_err_t `rx_comm_manager_poll` (rx_comm_manager_t *mgr)
Poll all enabled communication channels for incoming frames (non-blocking).
- rx_err_t `rx_comm_manager_send` (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)
Send frame on specified communication channel.
- rx_err_t `rx_comm_manager_respond` (rx_comm_manager_t *mgr, rx_comm_channel_t channel, const uint8_t *payload, uint32_t payload_len)
Send response frame on specified channel (convenience wrapper).
- rx_err_t `rx_comm_manager_stream_send` (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)
Send data on the stream path (fire-and-forget).
- rx_err_t `rx_comm_manager_event_send` (rx_comm_manager_t *mgr, const rx_comm_send_params_t *params)
Queue data on the event path (reliable, SPI retry).
- rx_err_t `rx_comm_manager_link_status` (const rx_comm_manager_t *mgr, rx_comm_channel_t channel, rx_comm_link_status_t *status)
Query link health status for a transport channel.
- rx_err_t `rx_comm_manager_channel_ready` (rx_comm_manager_t *mgr, rx_comm_channel_t channel, bool *ready)
Query if communication channel is ready for send/receive operations.
- const char * `rx_comm_manager_channel_name` (rx_comm_channel_t channel)
Get human-readable name for communication channel.
- rx_err_t `rx_comm_manager_process_retransmits` (rx_comm_manager_t *mgr, const uint32_t current_time_ms)
Process pending retransmissions on SPI channel.
- rx_err_t `rx_comm_manager_set_auto_retransmit` (rx_comm_manager_t *mgr, const bool enabled, const rx_spi_comm_retransmit_config_t *config)
Enable or disable automatic retransmission on SPI channel.

Variables

- static const char `s_tag` [] = "COMM_MGR"
- static const rx_comm_manager_t `s_zero_mgr` = {}
Zero-initialized comm manager template for stack-safe initialization.

4.3.1 Detailed Description

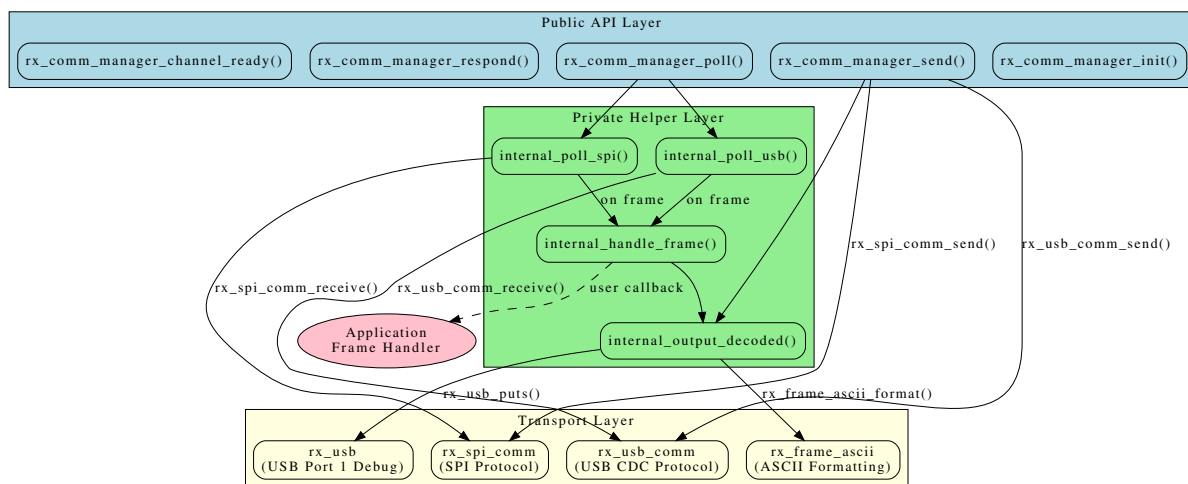
Unified Multi-Channel Communication Manager Implementation.

4.3.1.1 Overview

Production-quality implementation of unified communication manager coordinating multiple simultaneous channels (USB CDC and SPI) with automatic frame protocol handling, non-blocking polling architecture, optional decoded ASCII debugging output, and callback-based event notification.

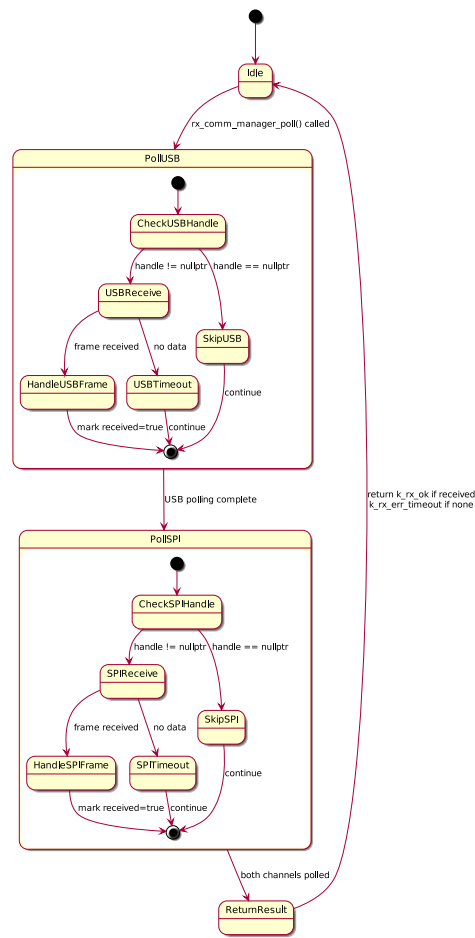
Purpose: Provide centralized, efficient coordination of independent communication channels between RX72N MCU and Raspberry Pi 5 host, abstracting low-level transport details while maintaining high performance and reliability.

4.3.1.2 Implementation Architecture



4.3.1.3 Polling State Machine

The `rx_comm_manager_poll()` function implements a non-blocking state machine that checks all enabled channels sequentially:



4.3.1.4 Performance Characteristics

Operation	Avg Time	Worst Case	Notes
<code>rx_comm_manager_init()</code>	~5 us	~10 us	memset dominates (328 bytes)
<code>rx_comm_manager_poll()</code>	~15-200 us	~500 us	Depends on channels enabled, frames available
<code>rx_comm_manager_send()</code>	~30-150 us	~300 us	Depends on payload size, channel
<code>internal_poll_usb()</code>	~10-100 us	~250 us	USB CDC bulk transfer latency
<code>internal_poll_spi()</code>	~5-50 us	~150 us	SPI hardware transfer latency
<code>internal_handle_frame()</code>	~2 us	~5 us	Callback invocation overhead
<code>internal_output_decoded()</code>	~50-150 us	~300 us	ASCII formatting + USB puts

Timing Analysis:

- Measured @ 240 MHz CPU clock with -O2 optimization
- Poll time dominated by transport layer (USB/SPI receive)
- ASCII debugging adds ~50-150 us per frame (disable in production)
- Callback execution time not included (application-dependent)

4.3.1.5 Memory Usage

Memory Type	Size	Usage
rx_comm_manager_t handle	328 bytes	Main state structure

Memory Type	Size	Usage
Stack (init)	~32 bytes	Function locals
Stack (poll)	~80 bytes	rx_frame_t + locals
Stack (send)	~80 bytes	rx_frame_t + locals
ASCII buffer	256 bytes	Inside handle (for debugging)

Memory Layout (rx_comm_manager_t):

Offset	Size	Field	Purpose
0x00	8	usb_handle	Pointer to USB comm handle
0x08	8	spi_handle	Pointer to SPI comm handle
0x10	8	callback	User frame callback function
0x18	8	callback_ctx	User context pointer
0x20	1	enable_decoded_output	ASCII debug flag
0x21	1	initialized	Initialization flag
0x22	2	(padding)	Alignment
0x24	4	(padding)	Alignment
0x28	256	ascii_buffer	ASCII formatting buffer
Total: 328 bytes (0x148)			

4.3.1.6 Algorithm Details

4.3.1.6.1 Polling Algorithm (rx_comm_manager_poll)

The polling algorithm maximizes throughput by checking all channels without blocking, returning immediately on the first error while continuing to poll remaining channels on timeout (no data available):

Steps:

- Parameter validation** (NULL check, initialized flag)
- Poll USB channel:**
 - If nullptr handle -> skip (k_rx_err_timeout)
 - If data available -> handle frame, mark received=true
 - If timeout -> continue to next channel
 - If error -> return error immediately (critical failure)
- Poll SPI channel:**
 - Same logic as USB
- Return aggregated result:**
 - k_rx_ok if ANY frame received on any channel
 - k_rx_err_timeout if NO frames received (normal, non-error condition)
 - Other error codes propagated immediately

Design Rationale:

- Non-blocking: Never blocks waiting for data
- Fair scheduling: Both channels polled each iteration
- Error prioritization: Critical errors returned immediately
- Timeout tolerance: Timeout is expected, not an error

4.3.1.6.2 Send Algorithm (rx_comm_manager_send)

Routing algorithm with post-send ASCII debugging support:

Steps:

1. **Validate parameters** (NULL checks, payload length $\leq$ k_frame_max_payload)
2. **Route to channel:**
 - USB: Call rx_usb_comm_send()
 - SPI: Call rx_spi_comm_send()
 - Invalid channel: Return k_rx_err_invalid_arg
3. **On success, if ASCII output enabled:**
 - Build temporary rx_frame_t from send parameters
 - Format as ASCII via rx_frame_ascii_format()
 - Send to USB Port 1 via rx_usb_puts()

Performance Note: ASCII formatting adds ~50-150 us overhead per send. Disable in production with `cfg->enable_decoded_output = false`.

4.3.1.7 Thread Safety Analysis

Conditional Thread Safety - Safe if following rules are observed:

Scenario	Thread Safety	Mitigation Required
Single-threaded	[OK] Safe	None
Multi-threaded (different channels)	[OK] Safe	Ensure each thread uses different channel
Multi-threaded (same channel)	[X] Unsafe	External mutex required
Multi-threaded (different handles)	[OK] Safe	None (independent handles)
Init/Deinit from ISR	[X] Unsafe	Call only from task context
Poll/Send from ISR	[WARN] Conditional	OK if callback is ISR-safe

Recommended Architecture:

- Dedicate one thread to poll all channels (e.g., ThreadX communication task)
- Send from any thread/ISR, but use external mutex if same channel
- Ensure user callback is reentrant if called from ISR

NASA Power of 10 Rule 5: All public functions validate preconditions with explicit checks (NULL pointers, initialized flag, parameter ranges).

4.3.1.8 Integration Example

Complete System Setup with ThreadX:

```
// Global communication manager handle
static rx_comm_manager_t g_comm_mgr;
static rx_usb_comm_handle_t g_usb_handle;
static rx_spi_comm_handle_t g_spi_handle;

// User callback for frame processing
static void on_frame_received(rx_comm_channel_t channel,
                             const rx_frame_t* frame,
                             void* ctx)
{
    (void)ctx;

    // Dispatch based on frame type
    switch (frame->header.type) {
        case k_frame_type_command:
            handle_command(channel, frame);
            break;

        case k_frame_type_request:
            handle_request(channel, frame);
            break;

        default:
            // Unknown frame type, ignore
            break;
    }
}

// ThreadX communication task
static void comm_task_entry(ULONG thread_input)
{
    (void)thread_input;

    // Initialize USB CDC communication
    rx_usb_comm_config_t usb_cfg = {
        .port = k_usb_port_proto,
    };
    rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
    if (err != k_rx_ok) {
        // Handle error
        return;
    }

    // Initialize SPI communication
    rx_spi_comm_config_t spi_cfg = {
        .spi_channel = 0,
    };
    err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
    if (err != k_rx_ok) {
        // Handle error
        return;
    }

    // Initialize communication manager
    rx_comm_manager_config_t cfg = {
        .usb_handle = &g_usb_handle,
        .spi_handle = &g_spi_handle,
        .callback = on_frame_received,
        .callback_ctx = nullptr,
        .enable_decoded_output = true, // Enable for development
    };
    err = rx_comm_manager_init(&g_comm_mgr, &cfg);
    if (err != k_rx_ok) {
        // Handle error
        return;
    }

    // Main communication loop (100 Hz polling)
    while (1) {
        // Poll all channels
        err = rx_comm_manager_poll(&g_comm_mgr);

        // Check for critical errors (not timeout)
        if (err != k_rx_ok && err != k_rx_err_timeout) {
            // Log error, attempt recovery
            tx_thread_sleep(100); // 1 second backoff
        }

        // Sleep 10 ticks (10 ms @ 1 kHz tick rate)
        tx_thread_sleep(10);
    }
}
```

```

}

// Send response from application
void send_motor_telemetry(void)
{
    uint8_t telemetry_payload[16];
    // ... fill telemetry data ...

    // Check if USB channel is ready
    bool usb_ready = false;
    rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                                k_comm_channel_usb,
                                                &usb_ready);

    if (err == k_rx_ok && usb_ready) {
        // Send telemetry response
        err = rx_comm_manager_respond(&g_comm_mgr,
                                    k_comm_channel_usb,
                                    telemetry_payload,
                                    sizeof(telemetry_payload));
    }
}

```

4.3.1.9 NASA Power of 10 Compliance

Rule	Compliance	Implementation Notes
Rule 1: Control Flow	[OK]	No goto, setjmp, longjmp, or recursion
Rule 2: Loop Bounds	[OK]	All loops have statically provable bounds
Rule 3: No Dynamic Allocation	[OK]	Zero malloc/free - all static allocation
Rule 4: Function Size	[OK]	All functions < 60 lines
Rule 5: Assertions	[OK]	Minimum 2 checks per function (nullptr, initialized)
Rule 6: Data Scope	[OK]	Variables declared at smallest scope
Rule 7: Check Returns	[OK]	All return values validated or cast to (void)
Rule 8: Preprocessor	[OK]	Minimal macros, typed enums for constants
Rule 9: Pointers	[WARN]	Function pointers used for callback (DIP)
Rule 10: Compiler Warnings	[OK]	-Wall -Wextra -Werror enabled

Rule 9 Deviation: Function pointer used for user callback to enable Dependency Inversion Principle (SOLID) - allows application to register frame handler without tight coupling. This is intentional and documented per project policy.

4.3.1.10 SOLID Principles

Principle	Implementation
Single Responsibility	Module responsible ONLY for channel coordination; frame parsing/transport delegated
Open/Closed	Extensible via callback; new frame types handled by application
Liskov Substitution	Channels are interchangeable (same rx_frame_t interface)
Interface Segregation	Small focused API (7 functions); send/poll/ready separated
Dependency Inversion	Depends on abstract rx_frame_t and channel handles, not concrete implementations

Key Design Decision: The manager does NOT parse frame payloads - it only routes frames to the user callback. This separation allows application-specific command handling without modifying the communication layer.

4.3.1.11 Module Dependencies

Includes:

- [rx_comm_manager.h](#) - Public API definitions
- [rx_frame_ascii.h](#) - ASCII formatting for debugging
- [rx_usb.h](#) - USB port access (Port 1 debug output)
- [string.h](#) - Standard C library (memset, memcpy)

Links Against:

- [rx_usb_comm](#) - USB CDC frame protocol implementation
- [rx_spi_comm](#) - SPI frame protocol implementation
- [rx_frame_ascii](#) - ASCII frame formatting
- [rx_usb](#) - USB hardware abstraction

Date

2026-01-27

Copyright

Copyright (c) 2026 STAR Project

Since

Version 1.0.0

See also

[rx_comm_manager.h](#) Full API documentation
[rx_usb_comm.h](#) USB CDC protocol implementation
[rx_spi_comm.h](#) SPI protocol implementation
[rx_frame_ascii.h](#) ASCII debugging interface

Definition in file [rx_comm_manager.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 internal_constants_t

enum [internal_constants_t](#) : uint32_t

Internal constants for communication manager implementation.

Defines compile-time constants used internally by the communication manager for timeout values, placeholder values, and buffer indexing.

Design Notes:

- Non-blocking timeout (0 ms) ensures poll never blocks
- Placeholder values filled by transport layer (sequence, CRC)
- Explicit array index constant for bounds checking

Since

Version 1.0.0

Enumerator

k_receive_timeout_ms	Non-blocking receive timeout for polling. Set to 0 ms to ensure rx_comm_manager_poll() never blocks waiting for data. Transport layers (USB/SPI) return k_rx_err_timeout immediately if no data. Value: 0 milliseconds
----------------------	---

k_frame_seq_placeholder	Placeholder for frame sequence number (filled by transport layer). Sequence number is managed by transport layer (rx_usb_comm/rx_spi_comm) to detect dropped frames. Manager does not manipulate sequence numbers. Value: 0
k_frame_crc_placeholder	Placeholder for frame CRC-32 (filled by transport layer). CRC-32 calculated and verified by transport layer. Manager builds temp frame with placeholder CRC for ASCII formatting only. Value: 0
k_ascii_buffer_first_idx	First index of ascii_buffer for post-condition validation. Used in rx_comm_manager_init() to verify memset properly cleared buffer. First byte must be '\0' after zero-fill. Value: 0
k_zero_u32	Typed zero constant for uint32_t comparisons. Used for payload length checks and other uint32_t zero comparisons to avoid magic number 0. Provides type safety and self-documenting code. Value: 0

Definition at line 464 of file [rx_comm_manager.c](#).

```

00464         : uint32_t {
00465     /**
00466      * @brief Non-blocking receive timeout for polling
00467      * @details
00468      * Set to 0 ms to ensure rx_comm_manager_poll() never blocks waiting for data.
00469      * Transport layers (USB/SPI) return k_rx_err_timeout immediately if no data.
00470      * @par Value: 0 milliseconds
00471      */
00472     k_receive_timeout_ms = 0,
00473
00474     /**
00475      * @brief Placeholder for frame sequence number (filled by transport layer)
00476      * @details
00477      * Sequence number is managed by transport layer (rx_usb_comm/rx_spi_comm)
00478      * to detect dropped frames. Manager does not manipulate sequence numbers.
00479      * @par Value: 0
00480      */
00481     k_frame_seq_placeholder = 0,
00482
00483     /**
00484      * @brief Placeholder for frame CRC-32 (filled by transport layer)
00485      * @details
00486      * CRC-32 calculated and verified by transport layer. Manager builds temp
00487      * frame with placeholder CRC for ASCII formatting only.
00488      * @par Value: 0
00489      */
00490     k_frame_crc_placeholder = 0,
00491
00492     /**
00493      * @brief First index of ascii_buffer for post-condition validation
00494      * @details
00495      * Used in rx_comm_manager_init() to verify memset properly cleared buffer.
00496      * First byte must be '\0' after zero-fill.
00497      * @par Value: 0
00498      */
00499     k_ascii_buffer_first_idx = 0,
00500
00501     /**
00502      * @brief Typed zero constant for uint32_t comparisons

```

```

00503  * @details
00504  * Used for payload length checks and other uint32_t zero comparisons to avoid
00505  * magic number 0. Provides type safety and self-documenting code.
00506  * @par Value: 0
00507  */
00508  k_zero_u32 = 0,
00509 } internal_constants_t;

```

4.3.2.2 rx_comm_sim_time_t

```
enum rx_comm_sim_time_t : uint32_t
```

Get current time in milliseconds.

Returns the current system time in milliseconds. On RX72N hardware, this uses ThreadX tx_time_get() multiplied by tick period. In simulator builds, returns 0 (timing not available).

Returns

Current time in milliseconds

Since

Version 1.0.0

Simulator time stub value (time is unavailable in simulator)

Enumerator

k_sim_time_ms_zero	Simulator returns zero (no real-time clock)
--------------------	---

Definition at line 431 of file rx_comm_manager.c.

```

00431      : uint32_t {
00432  k_sim_time_ms_zero = 0, /**< Simulator returns zero (no real-time clock) */
00433 } rx_comm_sim_time_t;

```

4.3.3 Function Documentation

4.3.3.1 internal_check_heartbeat()

```
void internal_check_heartbeat (
    rx_comm_manager_t * mgr) [static]
```

Check heartbeat implicit timeout on all transports.

Evaluates the dual-detection heartbeat model. If no valid frame has been received on a transport within 200ms, the link is declared dead and the status callback is invoked.

Parameters

in,out	mgr	Communication manager handle
--------	-----	------------------------------

Precondition

mgr must be non-NULL and initialized

Postcondition

Link status updated for each transport

Since

Version 1.0.0

Definition at line 976 of file [rx_comm_manager.c](#).

```

00977 {
00978     if (mgr == nullptr) {
00979         return;
00980     }
00981
00982     const uint32_t now = internal_get_time_ms();
00983
00984     for (uint8_t ch = 0; ch < k_comm_channel_count; ch++) {
00985         rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[ch];
00986
00987         /* Skip channels that haven't received any frame yet */
00988         if (hb->status == k_link_status_unknown) {
00989             continue;
00990         }
00991
00992         /* Check implicit timeout: 200ms since last valid frame */
00993         if (hb->status == k_link_status_healthy) {
00994             uint32_t elapsed = now - hb->last_rx_ms;
00995             if (elapsed >= k_heartbeat_implicit_timeout_ms) {
00996                 hb->status = k_link_status_dead;
00997                 rx_log_warn_val(s_tag, "Link dead: no frame (ms elapsed)", elapsed);
00998
00999                 if (mgr->link_status_cb != nullptr) {
01000                     mgr->link_status_cb((rx_comm_channel_t)ch, k_link_status_dead, mgr->link_status_ctx);
01001                 }
01002             }
01003         }
01004     }
01005 }

```

References [rx_comm_manager_t::heartbeat](#), [internal_get_time_ms\(\)](#), [k_comm_channel_count](#), [k_heartbeat_implicit_timeout_ms](#), [k_link_status_dead](#), [k_link_status_healthy](#), [k_link_status_unknown](#), [rx_comm_heartbeat_state_t::last_rx_ms](#), [rx_comm_manager_t::link_status_cb](#), [rx_comm_manager_t::link_status_ctx](#), [s_tag](#), and [rx_comm_heartbeat_state_t::status](#).

Referenced by [rx_comm_manager_poll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.2 internal_get_time_ms()

```
uint32_t internal_get_time_ms (
    void ) [static]
```

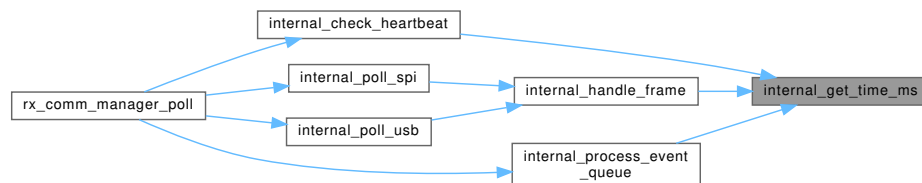
Definition at line 435 of file `rx_comm_manager.c`.

```
00436 {
00437     #if !RX_IS_SIMULATOR
00438         return (uint32_t)tx_time_get() * k_threadx_ms_per_tick;
00439     #else
00440         return k_sim_time_ms_zero;
00441     #endif
00442 }
```

References [k_sim_time_ms_zero](#).

Referenced by [internal_check_heartbeat\(\)](#), [internal_handle_frame\(\)](#), and [internal_process_event_queue\(\)](#).

Here is the caller graph for this function:



4.3.3.3 internal_handle_frame()

```
void internal_handle_frame (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    const rx_frame_t * frame) [static]
```

Handle received frame by invoking user callback and optional ASCII output.

Central frame dispatch point for all channels. Performs optional ASCII debugging output, then invokes user-registered callback with frame data. This function isolates callback invocation logic from channel-specific polling.

Algorithm:

1. Validate parameters (NULL checks, channel range)
2. Output decoded ASCII to Port 1 if enabled (RX direction)
3. Invoke user callback if registered

Design Rationale: Centralizing frame handling logic ensures consistent behavior across USB and SPI channels (single point of dispatch).

Parameters

in,out	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init() • Side effects: May modify <code>ascii_buffer</code> if decoded output enabled
--------	------------	---

in	<i>channel</i>	Channel the frame was received on • Valid range: k_comm_channel_usb or k_comm_channel_spi • Constraints: Must be < k_comm_channel_count • Purpose: Passed to user callback for channel identification
in	<i>frame</i>	Received frame data • Valid range: Non-nullptr to valid, CRC-verified frame • Constraints: Already validated by transport layer • Lifetime: Valid only for duration of this function call

Returns

void (no return value - errors silently handled)

Precondition

mgr must be initialized

frame must be valid and CRC-verified by transport layer

channel must be in valid range [0, k_comm_channel_count)

Postcondition

User callback invoked if registered (callback may be NULL)

ASCII output sent to Port 1 if enabled

Note

Thread safety depends on user callback implementation

Callback errors are not returned (callback should handle internally)

Warning

User callback must not block for extended periods (affects polling rate)

Performance:

- Without decoded output: ~2 us (callback overhead only)
- With decoded output: ~50-150 us (ASCII formatting + USB puts)

See also

[internal_output_decoded\(\)](#) ASCII debugging output

[rx_comm_frame_callback_t](#) User callback type

Since

Version 1.0.0

Definition at line 641 of file [rx_comm_manager.c](#).

```

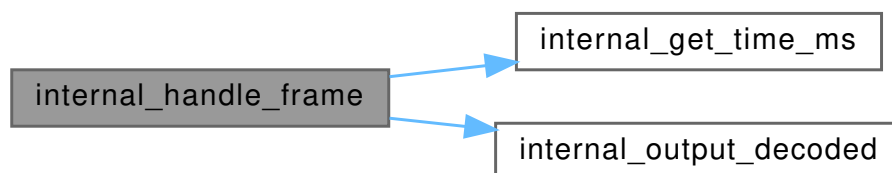
00642 {
00643  /* Rule 5: Pre-condition validation - defensive NULL checks */
00644  if (mgr == nullptr || frame == nullptr) {
00645      return;
00646  }
00647
00648  /* Validate channel is within expected range */
00649  if (channel >= k_comm_channel_count) {
00650      return;
00651  }
00652
00653  /* Update heartbeat: any valid frame resets the implicit timeout timer.
00654   * Per the dual-detection model, this is the primary link health
00655   * mechanism (200ms implicit timeout). */
00656  rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[channel];
00657  hb->last_rx_ms = internal_get_time_ms();
00658  if (hb->status != k_link_status_healthy) {
00659      rx_comm_link_status_t old_status = hb->status;
00660      hb->status = k_link_status_healthy;
00661      if (old_status == k_link_status_dead) {
00662          rx_log_info(s_tag, "Link recovered: frame received");
00663      } else {
00664          rx_log_info(s_tag, "Link established: first frame received");
00665      }
00666      if (mgr->link_status_cb != nullptr && old_status != k_link_status_healthy) {
00667          mgr->link_status_cb(channel, k_link_status_healthy, mgr->link_status_ctx);
00668      }
00669  }
00670
00671  rx_log_debug_val(s_tag, "Frame received, type", (uint8_t)frame->header.type);
00672
00673  /* Output decoded ASCII to Port 1 */
00674  internal_output_decoded(mgr, frame, false);
00675
00676  /* Invoke user callback if set */
00677  if (mgr->callback != nullptr) {
00678      mgr->callback(channel, frame, mgr->callback_ctx);
00679  }
00680 }

```

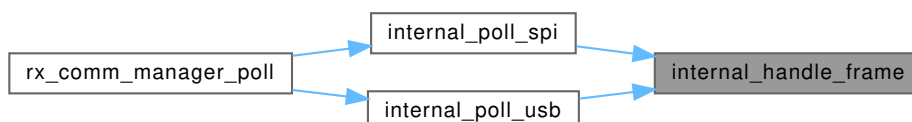
References [rx_comm_manager_t::callback](#), [rx_comm_manager_t::callback_ctx](#), [rx_comm_manager_t::heartbeat](#), [internal_get_time_ms\(\)](#), [internal_output_decoded\(\)](#), [k_comm_channel_count](#), [k_link_status_dead](#), [k_link_status_healthy](#), [rx_comm_heartbeat_state_t::last_rx_ms](#), [rx_comm_manager_t::link_status_cb](#), [rx_comm_manager_t::link_status_ctx](#), [s_tag](#), and [rx_comm_heartbeat_state_t::status](#).

Referenced by [internal_poll_spi\(\)](#), and [internal_poll_usb\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.4 internal_output_decoded()

```
void internal_output_decoded (
    rx_comm_manager_t * mgr,
    const rx_frame_t * frame,
    bool is_tx) [static]
```

Output decoded ASCII frame to USB Port 1 for debugging.

Formats the given frame as human-readable ASCII text and outputs to USB Port 1 (decoded debug port) if decoded output is enabled. Used for development/debugging to monitor frame traffic without binary protocol parsing.

Algorithm:

1. Validate parameters (NULL checks)
2. Check if decoded output is enabled (cfg->enable_decoded_output)
3. Format frame as ASCII via rx_frame_ascii_format()
4. Send ASCII string to USB Port 1 via rx_usb_puts()

Performance: ~50-150 us depending on frame size (ASCII formatting overhead). Disable in production for optimal performance.

Parameters

in,out	<i>mgr</i>	Communication manager handle (uses ascii_buffer) • Valid range: Non-nullptr to initialized handle • Constraints: Must have valid ascii_buffer (256 bytes) • Side effects: Modifies mgr->ascii_buffer content
in	<i>frame</i>	Frame to format and output • Valid range: Non-nullptr to valid rx_frame_t • Constraints: Header and payload must be valid • Units: Binary frame structure
in	<i>is_tx</i>	Direction flag (true = TX, false = RX) • Valid range: true (outgoing frame) or false (incoming frame) • Purpose: Prefix ASCII output with "TX:" or "RX:" for clarity

Returns

void (no return value - errors silently ignored)

Precondition

mgr must be initialized via rx_comm_manager_init()
 frame must be a valid frame structure

Postcondition

mgr->ascii_buffer content is modified (temporary)
 ASCII output sent to USB Port 1 if enabled and successful

Note

Not thread-safe - caller must ensure exclusive access to mgr->ascii_buffer
 Errors are silently ignored (formatting/USB send failures)
 USB Port 1 must be configured for ASCII output to appear

Example Output:

RX: [USB] Type=0x01 Flags=0x00 Len=8 Seq=42 Payload=0102030405060708 CRC=ABCD1234
 TX: [SPI] Type=0x02 Flags=0x80 Len=4 Seq=43 Payload=01020304 CRC=5678CDEF

See also

rx_frame_ascii_format() Frame-to-ASCII conversion
 rx_usb_puts() USB debug output

Since

Version 1.0.0

Definition at line 569 of file rx_comm_manager.c.

```

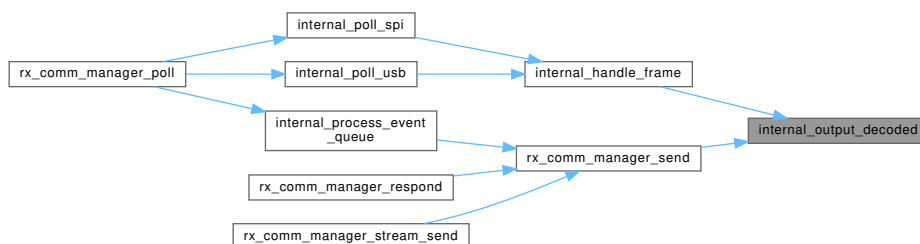
00570 {
00571     /* Rule 5: Pre-condition validation - defensive NULL checks */
00572     if (mgr == nullptr || frame == nullptr) {
00573         return;
00574     }
00575     if (!mgr->enable_decoded_output) {
00576         return;
00577     }
00578 }
00579
00580 uint32_t len = 0;
00581 rx_err_t err =
00582     rx_frame_ascii_format(frame, is_tx, mgr->ascii_buffer, sizeof(mgr->ascii_buffer), &len);
00583 if (err == k_rx_ok && len > 0) {
00584     (void)rx_usb_puts(k_usb_port_decoded, mgr->ascii_buffer);
00585 }
00586 }

```

References [rx_comm_manager_t::ascii_buffer](#), and [rx_comm_manager_t::enable_decoded_output](#).

Referenced by [internal_handle_frame\(\)](#), and [rx_comm_manager_send\(\)](#).

Here is the caller graph for this function:



4.3.3.5 internal_poll_spi()

```
rx_err_t internal_poll_spi (  
    rx_comm_manager_t * mgr) [static]
```

Poll SPI channel for incoming frames (non-blocking).

Attempts to receive one frame from SPI channel with zero timeout (non-blocking). If frame is successfully received, it is dispatched via [internal_handle_frame\(\)](#). Timeout (no data available) is expected and normal - not an error condition.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Skip if SPI handle is nullptr (channel not configured)
3. Call rx_spi_comm_receive() with 0 ms timeout
4. On success: dispatch frame via [internal_handle_frame\(\)](#)
5. On timeout: return k_rx_err_timeout (expected, not error)
6. On other errors: propagate error code

Design Note: Identical logic to [internal_poll_usb\(\)](#) but for SPI channel. Code duplication is intentional (NASA Rule 4: keep functions short and simple).

Parameters

in,out	mgr	Communication manager handle
		• Valid range: Non-nullptr to initialized handle • Constraints: Must have valid spi_handle if SPI channel enabled • Side effects: May invoke callback, modify ascii_buffer

Returns

rx_err_t Error code indicating poll result

Return values

<i>k_rx_ok</i>	Frame received and handled successfully
<i>k_rx_err_timeout</i>	No data available (normal, expected condition)
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_crc</i>	CRC verification failed (corrupted frame)
<i>k_rx_err_invalid_state</i>	SPI peripheral not ready
<i>(other)</i>	Transport layer errors propagated

Precondition

mgr must be non-NULL

If SPI channel enabled, spi_handle must be initialized

Postcondition

On k_rx_ok: callback invoked, ASCII output sent (if enabled)

On timeout: no side effects (no frame received)

Note

Non-blocking - always returns immediately

Timeout is NOT an error - it means no data is available

Warning

Do not call if spi_handle is uninitialized (will return timeout)

Performance:

- No data available: ~5 us (fast path)
- Frame received: ~50-150 us (SPI hardware transfer + callback)

See also

[internal_handle_frame\(\)](#) Frame dispatch

[rx_spi_comm_receive\(\)](#) SPI receive implementation

Since

Version 1.0.0

Definition at line 805 of file [rx_comm_manager.c](#).

```

00806 {
00807     /* Rule 5: Pre-condition validation - prevent NULL dereference */
00808     if (mgr == nullptr) {
00809         return k_rx_err_invalid_arg;
00810     }
00811
00812     if (mgr->spi_handle == nullptr) {
00813         return k_rx_err_timeout;
00814     }
00815
00816     /* If SPI link layer is available, use it for HARQ decoding
00817     * NOTE: Using static buffers to avoid stack overflow (~3KB total).
00818     * Safe because comm manager poll is single-threaded (called from comm_task). */
00819     if (mgr->spi_link != nullptr) {
00820         static rx_spi_link_receive_result_t s_link_result;
00821         rx_err_t err = rx_spi_link_receive(mgr->spi_link, &s_link_result, k_receive_timeout_ms);
00822
00823         if (err == k_rx_ok) {
00824             /* Defensive bounds check: Validate payload length before copy */
00825             if (s_link_result.payload_len > k_harq_max_payload ||
00826                 s_link_result.payload_len > sizeof(((rx_frame_t*)nullptr)->payload)) {
00827                 rx_log_error_val(s_tag, "SPI link receive: payload too large", s_link_result.payload_len);
00828                 return k_rx_err_invalid_size;
00829             }
00830
00831             /* Convert link result to rx_frame_t for internal_handle_frame() */
00832             static rx_frame_t s_frame1;

```

```

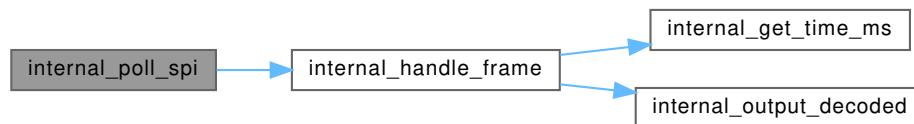
00833     s_frame1                = (rx_frame_t){0};
00834     s_frame1.header.sequence = s_link_result.sequence;
00835     s_frame1.header.length   = (uint16_t)s_link_result.payload_len;
00836     s_frame1.header.type     = s_link_result.frame_type;
00837
00838     /* Preserve retransmit and FEC flags from link layer result */
00839     s_frame1.header.flags = k_frame_flag_none;
00840     if (s_link_result.is_retransmit) {
00841         s_frame1.header.flags |= k_frame_flag_retransmit;
00842     }
00843     if (s_link_result.fec_decoded) {
00844         s_frame1.header.flags |= k_frame_flag_fec_enabled;
00845     }
00846
00847     if (s_link_result.payload_len > k_zero_u32) {
00848         (void)memcpy(s_frame1.payload, s_link_result.payload, s_link_result.payload_len);
00849     }
00850     internal_handle_frame(mgr, k_comm_channel_spi, &s_frame1);
00851     return k_rx_ok;
00852 }
00853
00854 return err;
00855 }
00856
00857 /* Fallback: raw SPI without HARQ */
00858 static rx_frame_t s_frame2;
00859 rx_err_t          err = rx_spi_comm_receive(mgr->spi_handle, &s_frame2, k_receive_timeout_ms);
00860
00861 if (err == k_rx_ok) {
00862     internal_handle_frame(mgr, k_comm_channel_spi, &s_frame2);
00863     return k_rx_ok;
00864 }
00865
00866 return err;
00867 }

```

References [internal_handle_frame\(\)](#), [k_comm_channel_spi](#), [k_receive_timeout_ms](#), [k_zero_u32](#), [s_tag](#), [rx_comm_manager_t::spi_handle](#), and [rx_comm_manager_t::spi_link](#).

Referenced by [rx_comm_manager_poll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.6 internal_poll_usb()

```

rx_err_t internal_poll_usb (
    rx_comm_manager_t * mgr) [static]

```

Poll USB channel for incoming frames (non-blocking).

Attempts to receive one frame from USB CDC channel with zero timeout (non-blocking). If frame is successfully received, it is dispatched via [internal_handle_frame\(\)](#). Timeout (no data available) is expected and normal - not an error condition.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Skip if USB handle is nullptr (channel not configured)
3. Call rx_usb_comm_receive() with 0 ms timeout
4. On success: dispatch frame via [internal_handle_frame\(\)](#)
5. On timeout: return k_rx_err_timeout (expected, not error)
6. On other errors: propagate error code

Design Note: This function treats timeout as non-error. The calling poll function (rx_comm_manager_poll) decides whether overall operation succeeded based on aggregate results from all channels.

Parameters

in,out	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must have valid usb_handle if USB channel enabled • Side effects: May invoke callback, modify ascii_buffer
--------	------------	--

Returns

rx_err_t Error code indicating poll result

Return values

<i>k_rx_ok</i>	Frame received and handled successfully
<i>k_rx_err_timeout</i>	No data available (normal, expected condition)
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_crc</i>	CRC verification failed (corrupted frame)
<i>k_rx_err_invalid_state</i>	USB peripheral not ready
<i>(other)</i>	Transport layer errors propagated

Precondition

mgr must be non-NULL
 If USB channel enabled, usb_handle must be initialized

Postcondition

On k_rx_ok: callback invoked, ASCII output sent (if enabled)
 On timeout: no side effects (no frame received)

Note

Non-blocking - always returns immediately
 Timeout is NOT an error - it means no data is available

Warning

Do not call if usb_handle is uninitialized (will return timeout)

Performance:

- No data available: ~10 us (fast path)
- Frame received: ~100-250 us (USB bulk transfer + callback)

See also

[internal_handle_frame\(\)](#) Frame dispatch

[rx_usb_comm_receive\(\)](#) USB CDC receive implementation

Since

Version 1.0.0

Definition at line 733 of file [rx_comm_manager.c](#).

```

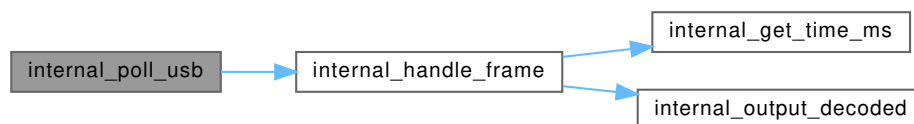
00734 {
00735     /* Rule 5: Pre-condition validation - prevent NULL dereference */
00736     if (mgr == nullptr) {
00737         return k_rx_err_invalid_arg;
00738     }
00739
00740     if (mgr->usb_handle == nullptr) {
00741         return k_rx_err_timeout;
00742     }
00743
00744     rx_frame_t frame;
00745     rx_err_t err = rx_usb_comm_receive(mgr->usb_handle, &frame, k_receive_timeout_ms);
00746
00747     if (err == k_rx_ok) {
00748         internal_handle_frame(mgr, k_comm_channel_usb, &frame);
00749         return k_rx_ok;
00750     }
00751
00752     return err;
00753 }

```

References [internal_handle_frame\(\)](#), [k_comm_channel_usb](#), [k_receive_timeout_ms](#), and [rx_comm_manager_t::usb_handle](#).

Referenced by [rx_comm_manager_poll\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.7 internal_process_event_queue()

```
void internal_process_event_queue (
    rx_comm_manager_t * mgr) [static]
```

Process one event from the FIFO queue.

Dequeues the oldest event and attempts to send it. For SPI events, if the send fails, the event is re-enqueued with incremented retry count (up to `k_event_max_retries`). For USB events, failure is simply dropped (fire-and-forget).

Parameters

in,out	<i>mgr</i>	Communication manager handle
--------	------------	------------------------------

Precondition

`mgr` must be non-NULL and initialized

Postcondition

One event processed (sent or dropped on max retries)

Since

Version 1.0.0

Definition at line 889 of file `rx_comm_manager.c`.

```
00890 {
00891     if (mgr == nullptr || mgr->event_queue_count == 0) {
00892         return;
00893     }
00894     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_tail];
00895     if (!entry->occupied) {
00896         return;
00897     }
00898 }
00899
00900 /* Check backoff: skip if retry time hasn't been reached yet */
00901 const uint32_t now_ms = internal_get_time_ms();
00902 if (entry->retries > 0 && now_ms < entry->next_retry_time_ms) {
00903     return; /* Backoff period not elapsed, try again later */
00904 }
00905
00906 /* Build send params from queued entry */
00907 const rx_comm_send_params_t params = {
00908     .channel = entry->channel,
00909     .type = entry->type,
00910     .flags = entry->flags,
00911     .payload = entry->payload,
00912     .payload_len = entry->payload_len,
00913 };
00914
00915 rx_err_t err = rx_comm_manager_send(mgr, &params);
00916
00917 if (err == k_rx_ok || entry->channel == k_comm_channel_usb) {
00918     /* Success, or USB fire-and-forget: dequeue */
00919     entry->occupied = false;
00920     entry->retries = 0;
00921     entry->next_retry_time_ms = 0;
00922     mgr->event_queue_tail = (mgr->event_queue_tail + 1) % k_comm_event_queue_depth;
00923     mgr->event_queue_count--;
00924 }
00925 if (err == k_rx_ok) {
00926     rx_log_debug_val(s_tag, "Event sent, queue depth", mgr->event_queue_count);
00927 } else {
00928     rx_log_warn_val(s_tag, "USB event dropped, queue depth", mgr->event_queue_count);
00929 }
00930 return;
```

```

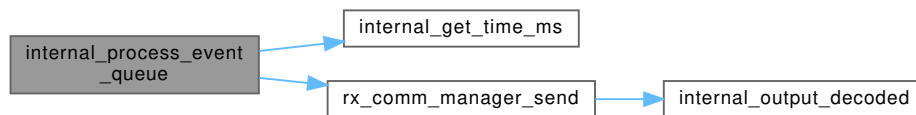
00931 }
00932
00933 /* SPI send failed: retry with exponential backoff */
00934 entry->retries++;
00935 if (entry->retries >= k_event_max_retries) {
00936     rx_log_error_val(s_tag, "SPI event failed after retries", (uint8_t)entry->retries);
00937     entry->occupied = false;
00938     entry->retries = 0;
00939     entry->next_retry_time_ms = 0;
00940     mgr->event_queue_tail = (mgr->event_queue_tail + 1) % k_comm_event_queue_depth;
00941     mgr->event_queue_count--;
00942     return;
00943 }
00944
00945 /* Compute exponential backoff: initial_ms « (retries - 1), capped at max */
00946 uint32_t backoff_ms = (uint32_t)k_event_initial_backoff_ms « (entry->retries - 1);
00947 if (backoff_ms > k_event_max_backoff_ms) {
00948     backoff_ms = k_event_max_backoff_ms;
00949 }
00950 entry->next_retry_time_ms = now_ms + backoff_ms;
00951
00952 rx_log_warn_val(s_tag, "SPI event retry attempt", (uint8_t)entry->retries);
00953 /* Entry stays in queue, will be retried after backoff period */
00954 }

```

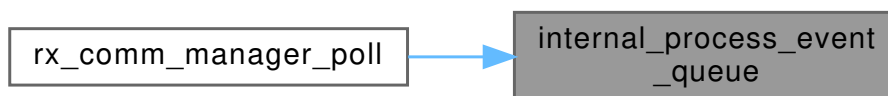
References `rx_comm_event_entry_t::channel`, `rx_comm_manager_t::event_queue`, `rx_comm_manager_t::event_queue_count`, `rx_comm_manager_t::event_queue_tail`, `rx_comm_event_entry_t::flags`, `internal_get_time_ms()`, `k_comm_channel_usb`, `k_comm_event_queue_depth`, `k_event_initial_backoff_ms`, `k_event_max_backoff_ms`, `k_event_max_retries`, `rx_comm_event_entry_t::next_retry_time_ms`, `rx_comm_event_entry_t::occupied`, `rx_comm_event_entry_t::payload`, `rx_comm_event_entry_t::payload_len`, `rx_comm_event_entry_t::retries`, `rx_comm_manager_send()`, `s_tag`, and `rx_comm_event_entry_t::type`.

Referenced by `rx_comm_manager_poll()`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.8 rx_comm_manager_channel_name()

```

const char * rx_comm_manager_channel_name (
    rx_comm_channel_t channel)

```

Get human-readable name for communication channel.

Get channel name string.

Returns static string name for given channel enum value. Used for logging, debugging, and user interface display. Always returns a valid string (never NULL).

Returned Strings:

- `k_comm_channel_usb` -> "USB"
- `k_comm_channel_spi` -> "SPI"
- Invalid/unknown -> "UNKNOWN"

Use Cases:

- Logging channel activity
- Debugging frame traffic
- User interface display
- Error messages

Parameters

in	<i>channel</i>	Communication channel enum value • Valid range: Any <code>rx_comm_channel_t</code> value • Constraints: None (invalid values return "UNKNOWN")
----	----------------	--

Returns

const char* Human-readable channel name (never NULL)

Return values

<i>USB</i>	If <code>channel == k_comm_channel_usb</code>
<i>SPI</i>	If <code>channel == k_comm_channel_spi</code>
<i>UNKNOWN</i>	If channel is invalid or out of range

Precondition

None (accepts any input)

Postcondition

Returns pointer to static string (valid for program lifetime)

Note

Returned string is static - do not free
Thread-safe (returns read-only static strings)
Never returns NULL - always safe to use in printf

Example - Logging:

```
static void on_frame_received(rx_comm_channel_t channel,
                             const rx_frame_t* frame,
                             void* ctx)
{
    printf("Frame received on %s: type=%d len=%d\n",
          rx_comm_manager_channel_name(channel),
          frame->header.type,
          frame->header.length);
}
```

Example - Error Messages:

```
rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err != k_rx_ok) {
    fprintf(stderr, "Failed to send on %s: error %d\n",
          rx_comm_manager_channel_name(params.channel), err);
}
```

See also

[rx_comm_channel_t](#) Channel enumeration

Since

Version 1.0.0

Definition at line 2015 of file [rx_comm_manager.c](#).

```
02016 {
02017     switch (channel) {
02018     case k_comm_channel_usb:
02019         return "USB";
02020     case k_comm_channel_spi:
02021         return "SPI";
02022     default:
02023         return "UNKNOWN";
02024     }
02025 }
```

References [k_comm_channel_spi](#), and [k_comm_channel_usb](#).

4.3.3.9 rx_comm_manager_channel_ready()

```
rx_err_t rx_comm_manager_channel_ready (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    bool * ready) [nodiscard]
```

Query if communication channel is ready for send/receive operations.

Check if channel is ready for communication.

Checks if specified channel is configured and ready for communication. USB channel readiness depends on USB enumeration and configuration by host. SPI channel is ready immediately if handle is configured (no enumeration required).

Readiness Criteria:

- **USB:** usb_handle non-NULL AND USB device enumerated and configured by host

- **SPI:** spi_handle non-NULL (always ready if configured)

Use Cases:

- Check readiness before sending to avoid errors
- Implement fallback logic (try USB, fall back to SPI)
- Log channel availability status

Parameters

in	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init()
----	------------	---

in	<i>channel</i>	Channel to query • Valid range: k_comm_channel_usb or k_comm_channel_spi • Constraints: Must be valid channel ID
out	<i>ready</i>	Pointer to receive ready status • Valid range: Non-nullptr to bool • Output values: true (ready) or false (not ready) • On error: Set to false

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Query successful, ready flag updated
<i>k_rx_err_invalid_arg</i>	mgr or ready is nullptr, or channel invalid
<i>k_rx_err_invalid_state</i>	Manager not initialized

Precondition

mgr must be non-NULL and initialized

ready must be non-NULL

channel must be valid (USB or SPI)

Postcondition

*ready set to true or false based on channel state

On error, *ready set to false (safe default)

Note

USB readiness can change at runtime (host connects/disconnects)

SPI readiness is static (always ready if handle configured)

This function does NOT initiate communication - only queries state

Example - Send with Fallback:

```
uint8_t payload[16] = { ... };

// Try USB first
bool usb_ready = false;
rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                             k_comm_channel_usb,
                                             &usb_ready);
if (err == k_rx_ok && usb_ready) {
    err = rx_comm_manager_respond(&g_comm_mgr,
                                 k_comm_channel_usb,
                                 payload,
                                 sizeof(payload));
} else {
    // Fallback to SPI
    bool spi_ready = false;
    err = rx_comm_manager_channel_ready(&g_comm_mgr,
                                       k_comm_channel_spi,
                                       &spi_ready);
    if (err == k_rx_ok && spi_ready) {
        err = rx_comm_manager_respond(&g_comm_mgr,
                                     k_comm_channel_spi,
                                     payload,
                                     sizeof(payload));
    }
}
```

Example - Log Channel Status:

```
void log_channel_status(void)
{
    bool usb_ready = false, spi_ready = false;

    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_usb, &usb_ready);
    rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_spi, &spi_ready);

    printf("USB: %s, SPI: %s\n",
          usb_ready ? "READY" : "NOT READY",
          spi_ready ? "READY" : "NOT READY");
}
```

See also

[rx_usb_is_configured\(\)](#) USB enumeration status
[rx_comm_manager_send\(\)](#) Send frame on channel

Since

Version 1.0.0

Definition at line 1922 of file [rx_comm_manager.c](#).

```
01923 {
01924     /* Rule 5: Pre-condition validation */
01925     if (mgr == nullptr || ready == nullptr) {
01926         if (ready != nullptr) {
01927             *ready = false;
01928         }
01929         return k_rx_err_invalid_arg;
01930     }
01931     if (!mgr->initialized) {
01932         *ready = false;
01933         return k_rx_err_invalid_state;
01934     }
01935     switch (channel) {
01936     case k_comm_channel_usb:
01937         *ready = (mgr->usb_handle != nullptr) && rx_usb_is_configured(k_usb_port_proto);
01938         break;
01939     case k_comm_channel_spi:
01940         /* SPI is always "ready" if handle is set */
01941         *ready = (mgr->spi_handle != nullptr);
01942         break;
01943     }
```

```
01945
01946     default:
01947         *ready = false;
01948         rx_log_error(s_tag, "Channel ready: invalid channel");
01949         return k_rx_err_invalid_arg;
01950     }
01951
01952     return k_rx_ok;
01953 }
```

References [rx_comm_manager_t::initialized](#), [k_comm_channel_spi](#), [k_comm_channel_usb](#), [s_tag](#), [rx_comm_manager_t::spi_handle](#), and [rx_comm_manager_t::usb_handle](#).

4.3.3.10 rx_comm_manager_deinit()

```
rx_err_t rx_comm_manager_deinit (
    rx_comm_manager_t * mgr) [nodiscard]
```

Deinitialize communication manager and release resources.

Deinitialize communication manager.

Marks communication manager as uninitialized, preventing further operations. This function does NOT deinitialize USB or SPI transport handles - caller must separately call [rx_usb_comm_deinit\(\)](#) and [rx_spi_comm_deinit\(\)](#) if needed.

Algorithm:

1. Validate mgr pointer (NULL check)
2. Validate initialized flag (must be initialized to deinitialize)
3. Clear initialized flag (marks handle as invalid)

Design Note: Minimal deinitialization - no dynamic resources to free (all memory is statically allocated). Function primarily serves as safety guard to prevent use-after-deinit.

Parameters

in,out	<i>mgr</i>	Communication manager handle
		• Valid range: Non-nullptr to initialized handle • Constraints: Must have been initialized via rx_comm_manager_init() • Side effects: Clears mgr->initialized flag

Returns

rx_err_t Error code

Return values

<i>k_rx_ok</i>	Success - manager deinitialized
----------------	---------------------------------

<code>k_rx_err_invalid_arg</code>	mgr is nullptr
<code>k_rx_err_invalid_state</code>	mgr was not initialized

Precondition

mgr must be non-NULL
mgr must be initialized (`mgr->initialized == true`)

Postcondition

`mgr->initialized = false`
Subsequent calls to `rx_comm_manager_poll/send` will return `k_rx_err_invalid_state`

Note

Does NOT free memory (handle is statically allocated)
Does NOT deinitialize transport handles (USB/SPI)
Thread-safe if no concurrent operations on same mgr handle

Example:

```
// Deinitialize manager
rx_err_t err = rx_comm_manager_deinit(&g_comm_mgr);
if (err == k_rx_ok) {
    // Manager no longer usable

    // Separately deinitialize transports if needed
    rx_usb_comm_deinit(&g_usb_handle);
    rx_spi_comm_deinit(&g_spi_handle);
}
```

See also

[rx_comm_manager_init\(\)](#) Initialization
[rx_usb_comm_deinit\(\)](#) USB transport cleanup
[rx_spi_comm_deinit\(\)](#) SPI transport cleanup

Since

Version 1.0.0

Definition at line 1249 of file rx_comm_manager.c.

```
01250 {
01251     /* Rule 5: Pre-condition validation */
01252     if (mgr == nullptr) {
01253         rx_log_error(s_tag, "Deinit failed: NULL mgr");
01254         return k_rx_err_invalid_arg;
01255     }
01256     if (!mgr->initialized) {
01257         rx_log_warn(s_tag, "Deinit called on uninitialized mgr");
01258         return k_rx_err_invalid_state;
01259     }
01260
01261     mgr->initialized = false;
01262
01263     rx_log_info(s_tag, "Comm manager deinitialized");
01264     return k_rx_ok;
01265 }
```

References [rx_comm_manager_t::initialized](#), and [s_tag](#).

4.3.3.11 rx_comm_manager_event_send()

```
rx_err_t rx_comm_manager_event_send (  
    rx_comm_manager_t * mgr,  
    const rx_comm_send_params_t * params)  [nodiscard]
```

Queue data on the event path (reliable, SPI retry).

Event path is for commands and other data requiring reliable delivery. The payload is copied into a static FIFO queue and processed by the poll loop. Retry behavior depends on transport:

- **SPI:** Up to 3 retries with exponential backoff (10ms, 20ms, 40ms)
- **USB:** Fire-and-forget (USB HW reliability is sufficient)

Queue is processed in FIFO order by `rx_comm_manager_poll()`.

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
in	<i>params</i>	Send parameters (channel, type, flags, payload, payload_len)

Returns

k_rx_ok on success (queued)
k_rx_err_invalid_arg if mgr or params is nullptr
k_rx_err_invalid_state if not initialized
k_rx_err_no_mem if event queue is full

Precondition

mgr must be initialized
params must be valid
params->payload_len <= k_frame_max_payload

Postcondition

Payload copied to queue, will be sent by next poll()

Note

Payload is COPIED into queue - caller can reuse buffer immediately

Warning

Queue depth is k_comm_event_queue_depth (8). If full, returns error.

See also

[rx_comm_manager_stream_send\(\)](#) For fire-and-forget telemetry

Since

Version 1.0.0

Definition at line 1751 of file [rx_comm_manager.c](#).

```

01752 {
01753     if (mgr == nullptr || params == nullptr) {
01754         rx_log_error(s_tag, "Event enqueue failed: NULL arg");
01755         return k_rx_err_invalid_arg;
01756     }
01757     if (!mgr->initialized) {
01758         rx_log_error(s_tag, "Event enqueue failed: not initialized");
01759         return k_rx_err_invalid_state;
01760     }
01761     if (params->payload_len > k_frame_max_payload) {
01762         rx_log_error_val(s_tag, "Event enqueue failed: payload too large", params->payload_len);
01763         return k_rx_err_invalid_arg;
01764     }
01765     if (params->payload_len > 0 && params->payload == nullptr) {
01766         rx_log_error(s_tag, "Event enqueue failed: NULL payload with nonzero len");
01767         return k_rx_err_invalid_arg;
01768     }
01769     /* Check queue capacity */
01770     if (mgr->event_queue_count >= k_comm_event_queue_depth) {
01771         rx_log_error_val(s_tag, "Event queue full, depth", (uint8_t)mgr->event_queue_count);
01772         return k_rx_err_no_mem;
01773     }
01774     /* Enqueue: copy payload into static slot */
01775     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_head];
01776     entry->channel = params->channel;
01777     entry->type = params->type;
01778     entry->flags = params->flags;
01779     entry->payload_len = params->payload_len;
01780     entry->retries = 0;
01781     entry->occupied = true;
01782     if (params->payload != nullptr && params->payload_len > 0) {
01783         (void)memcpy(entry->payload, params->payload, params->payload_len);
01784     }
01785     mgr->event_queue_head = (mgr->event_queue_head + 1) % k_comm_event_queue_depth;
01786     mgr->event_queue_count++;
01787     rx_log_debug_val(s_tag, "Event enqueued, queue depth", mgr->event_queue_count);
01788     return k_rx_ok;
01789 }

```

References [rx_comm_event_entry_t::channel](#), [rx_comm_send_params_t::channel](#), [rx_comm_manager_t::event_queue](#), [rx_comm_manager_t::event_queue_count](#), [rx_comm_manager_t::event_queue_head](#), [rx_comm_event_entry_t::flags](#), [rx_comm_send_params_t::flags](#), [rx_comm_manager_t::initialized](#), [k_comm_event_queue_depth](#), [rx_comm_event_entry_t::occupied](#), [rx_comm_event_entry_t::payload](#), [rx_comm_send_params_t::payload](#), [rx_comm_event_entry_t::payload_len](#), [rx_comm_send_params_t::payload_len](#), [rx_comm_event_entry_t::retries](#), [s_tag](#), [rx_comm_event_entry_t::type](#), and [rx_comm_send_params_t::type](#).

4.3.3.12 rx_comm_manager_init()

```

rx_err_t rx_comm_manager_init (
    rx_comm_manager_t * mgr,
    const rx_comm_manager_config_t * cfg) [nodiscard]

```

Initialize communication manager with channel configuration.

Initialize communication manager.

Performs complete initialization of communication manager handle, configuring enabled channels, callback registration, and ASCII debugging output. This function must be called before any other communication manager operations.

Algorithm:

1. **Validate parameters:** Check mgr pointer for nullptr (NASA Rule 5)
2. **Clear handle:** Zero-fill entire structure via memset (328 bytes)
3. **Apply configuration:** Copy config fields if cfg is non-NULL
 - **usb_handle:** Pointer to initialized USB comm handle (may be NULL)
 - **spi_handle:** Pointer to initialized SPI comm handle (may be NULL)
 - **callback:** User frame handler function (may be NULL)
 - **callback_ctx:** User context pointer (may be NULL)
 - **enable_decoded_output:** ASCII debugging flag
4. **Mark initialized:** Set mgr->initialized = true
5. **Post-condition validation:** Verify ascii_buffer properly cleared

Configuration Options:

- **cfg = NULL:** All channels disabled, no callback, ASCII output off
- **usb_handle = NULL:** USB channel disabled (poll skips USB)
- **spi_handle = NULL:** SPI channel disabled (poll skips SPI)
- **callback = NULL:** No callback invoked (silent frame consumption)
- **enable_decoded_output = true:** ASCII debugging enabled (Port 1)

Parameters

out	<i>mgr</i>	Pointer to communication manager handle to initialize • Valid range: Non-nullptr to allocated rx_comm_manager_t (328 bytes) • Constraints: Must be allocated by caller (typically static or global) • Side effects: Entire structure zero-filled, then populated from <i>cfg</i> • Lifetime: Must remain valid until rx_comm_manager_deinit() called
in	<i>cfg</i>	Optional configuration structure • Valid range: NULL or pointer to valid rx_comm_manager_config_t • NULL behavior: All channels disabled, no callback, ASCII off • Lifetime: Not stored - values copied into <i>mgr</i> • Constraints: If non-NULL, all fields must be valid or nullptr as appropriate

Returns

`rx_err_t` Error code indicating initialization success or failure

Return values

<i>k_rx_ok</i>	Success - manager fully initialized and ready for operation
----------------	---

<code>k_rx_err_invalid_arg</code>	mgr is nullptr
<code>k_rx_err_invalid_state</code>	Post-condition validation failed (buffer not cleared)

Precondition

mgr must point to allocated memory (uninitialized content OK)
 USB/SPI handles (if provided) must already be initialized
 Callback function (if provided) must be valid and reentrant

Postcondition

mgr->initialized = true on success
 All mgr fields set to config values or zero
 mgr->ascii_buffer is zero-filled (256 bytes)

Note

This function does NOT initialize USB or SPI transports - caller must initialize rx_usb_comm and rx_spi_comm separately before passing handles to this function
 On success, at least one channel should be enabled (usb_handle or spi_handle non-NULL)
 Thread-safe ONLY if each thread uses a different mgr handle

Performance:

- Execution time: ~5 us @ 240 MHz
- Dominated by memset (328 bytes)

Configuration Example - Both Channels Enabled:

```
static rx_comm_manager_t g_comm_mgr;
static rx_usb_comm_handle_t g_usb_handle;
static rx_spi_comm_handle_t g_spi_handle;

// Initialize transport layers first
rx_usb_comm_config_t usb_cfg = { .port = k_usb_port_proto };
rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
if (err != k_rx_ok) return err;

rx_spi_comm_config_t spi_cfg = { .spi_channel = 0 };
err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
if (err != k_rx_ok) return err;

// Initialize communication manager with both channels
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = &g_spi_handle,
    .callback = on_frame_received,
    .callback_ctx = nullptr,
    .enable_decoded_output = true,
};
err = rx_comm_manager_init(&g_comm_mgr, &cfg);
```

Configuration Example - USB Only, No Callback:

```
rx_comm_manager_config_t cfg = {
    .usb_handle = &g_usb_handle,
    .spi_handle = nullptr,           // SPI disabled
    .callback = nullptr,            // No callback
    .callback_ctx = nullptr,
    .enable_decoded_output = false, // No ASCII debugging
};
rx_err_t err = rx_comm_manager_init(&g_comm_mgr, &cfg);
```

See also

[rx_comm_manager_deinit\(\)](#) Cleanup and resource release

[rx_comm_manager_poll\(\)](#) Poll for incoming frames

[rx_usb_comm_init\(\)](#) Initialize USB CDC transport

[rx_spi_comm_init\(\)](#) Initialize SPI transport

Since

Version 1.0.0

Definition at line 1129 of file [rx_comm_manager.c](#).

```
01130 {
01131     /* Rule 5: Pre-condition validation - nullptr check */
01132     if (mgr == nullptr) {
01133         rx_log_error(s_tag, "Init failed: NULL mgr");
01134         return k_rx_err_invalid_arg;
01135     }
01136
01137     /* Rule 5: Pre-condition validation - SPI link consistency checks */
01138     if (cfg != nullptr && cfg->spi_link != nullptr) {
01139         /* If spi_link is provided, spi_handle must also be provided */
01140         if (cfg->spi_handle == nullptr) {
01141             rx_log_error(s_tag, "Init failed: spi_link requires non-NULL spi_handle");
01142             return k_rx_err_invalid_arg;
01143         }
01144
01145         /* If spi_link is provided, it must be initialized */
01146         if (!cfg->spi_link->initialized) {
01147             rx_log_error(s_tag, "Init failed: spi_link not initialized (call rx_spi_link_init first)");
01148             return k_rx_err_invalid_arg;
01149         }
01150
01151         /* If spi_link is provided, its spi_handle must match config spi_handle */
01152         if (cfg->spi_link->spi_handle != cfg->spi_handle) {
01153             rx_log_error(s_tag, "Init failed: spi_link->spi_handle != cfg->spi_handle");
01154             return k_rx_err_invalid_arg;
01155         }
01156     }
01157
01158     *mgr = s_zero_mgr;
01159
01160     /* Apply configuration */
01161     if (cfg != nullptr) {
01162         mgr->usb_handle = cfg->usb_handle;
01163         mgr->spi_handle = cfg->spi_handle;
01164         mgr->spi_link = cfg->spi_link;
01165         mgr->callback = cfg->callback;
01166         mgr->callback_ctx = cfg->callback_ctx;
01167         mgr->enable_decoded_output = cfg->enable_decoded_output;
01168         mgr->link_status_cb = cfg->link_status_cb;
01169         mgr->link_status_ctx = cfg->link_status_ctx;
01170     }
01171
01172     mgr->initialized = true;
01173
01174     /* Rule 5: Post-condition validation - verify buffer properly cleared */
01175     if (mgr->ascii_buffer[k_ascii_buffer_first_idx] != '\0') {
01176         /* Buffer not properly cleared by memset - potential memory corruption */
01177         mgr->initialized = false;
01178         rx_log_error(s_tag, "Init failed: buffer corruption");
01179         return k_rx_err_invalid_state;
01180     }
01181 }
```

```

01182  /* Rule 5: Post-condition validation - verify handle/link consistency */
01183  if (mgr->spi_link != nullptr && mgr->spi_link != cfg->spi_link) {
01184      /* Postcondition violation: spi_link not properly assigned */
01185      mgr->initialized = false;
01186      rx_log_error(s_tag, "Init failed: spi_link assignment mismatch");
01187      return k_rx_err_invalid_state;
01188  }
01189
01190  rx_log_info(s_tag, "Comm manager initialized");
01191  return k_rx_ok;
01192 }

```

References [rx_comm_manager_t::ascii_buffer](#), [rx_comm_manager_config_t::callback](#), [rx_comm_manager_t::callback](#), [rx_comm_manager_config_t::callback_ctx](#), [rx_comm_manager_t::callback_ctx](#), [rx_comm_manager_config_t::enable_decoded_output](#), [rx_comm_manager_t::enable_decoded_output](#), [rx_comm_manager_t::initialized](#), [k_ascii_buffer_first_idx](#), [rx_comm_manager_config_t::link_status_cb](#), [rx_comm_manager_config_t::link_status_ctx](#), [rx_comm_manager_t::link_status_ctx](#), [s_tag](#), [s_zero_mgr](#), [rx_comm_manager_config_t::spi_handle](#), [rx_comm_manager_t::spi_handle](#), [rx_comm_manager_config_t::spi_link](#), [rx_comm_manager_t::spi_link](#), [rx_comm_manager_config_t::usb_handle](#), and [rx_comm_manager_t::usb_handle](#).

4.3.3.13 rx_comm_manager_link_status()

```

rx_err_t rx_comm_manager_link_status (
    const rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    rx_comm_link_status_t * status) [nodiscard]

```

Query link health status for a transport channel.

Returns the current heartbeat-derived link status for the given channel. Status is updated by [rx_comm_manager_poll\(\)](#) based on the dual-detection heartbeat model (200ms implicit timeout).

Parameters

in	<i>mgr</i>	Pointer to initialized manager
in	<i>channel</i>	Channel to query
out	<i>status</i>	Receives current link status

Returns

[k_rx_ok](#) on success
[k_rx_err_invalid_arg](#) if mgr or status is nullptr

Since

Version 1.0.0

Definition at line 1804 of file [rx_comm_manager.c](#).

```

01807 {
01808     if (mgr == nullptr || status == nullptr) {
01809         rx_log_error(s_tag, "Link status query: NULL arg");
01810         return k_rx_err_invalid_arg;
01811     }
01812
01813     if (!mgr->initialized) {
01814         rx_log_error(s_tag, "Link status query: uninitialized manager");
01815         return k_rx_err_invalid_state;
01816     }
01817
01818     if (channel >= k_comm_channel_count) {
01819         rx_log_error(s_tag, "Link status query: invalid channel");
01820         return k_rx_err_invalid_arg;
01821     }
01822
01823     *status = mgr->heartbeat[channel].status;
01824     return k_rx_ok;
01825 }

```

References [rx_comm_manager_t::heartbeat](#), [rx_comm_manager_t::initialized](#), [k_comm_channel_count](#), [s_tag](#), and [rx_comm_heartbeat_state_t::status](#).

4.3.3.14 rx_comm_manager_poll()

```
rx_err_t rx_comm_manager_poll (  
    rx_comm_manager_t * mgr) [nodiscard]
```

Poll all enabled communication channels for incoming frames (non-blocking).

Poll all enabled channels for incoming frames.

Performs non-blocking poll of all enabled channels (USB and SPI) sequentially, dispatching any received frames to the registered user callback. Returns aggregate result indicating whether any frames were received across all channels.

Algorithm:

1. **Validate parameters:** Check mgr pointer and initialized flag (NASA Rule 5)
2. **Poll USB channel** via [internal_poll_usb\(\)](#):
 - If k_rx_ok: Mark received=true, continue to SPI
 - If k_rx_err_timeout: Continue to SPI (no data is normal)
 - If other error: Return error immediately (critical failure)
3. **Poll SPI channel** via [internal_poll_spi\(\)](#):
 - If k_rx_ok: Mark received=true
 - If k_rx_err_timeout: Continue (no data is normal)
 - If other error: Return error immediately (critical failure)
4. **Return result:**
 - k_rx_ok if ANY channel received frame
 - k_rx_err_timeout if NO channels received (normal condition)
 - Other error codes propagated from transport layers

Performance: 15-200 us typical, depends on:

- Number of enabled channels (USB/SPI/both)
- Whether frames are available (data path vs fast timeout path)
- ASCII debugging enabled (adds ~50-150 us per frame)
- User callback execution time (not included in measurement)

Design Rationale:

- **Non-blocking:** Never blocks waiting for data (0 ms timeout)
- **Fair scheduling:** Both channels polled each iteration
- **Timeout tolerance:** Timeout is expected, not error condition
- **Error prioritization:** Critical errors returned immediately
- **Aggregation:** k_rx_ok if ANY channel received data

Parameters

in,out	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init() • Side effects: Invokes callback, modifies <code>ascii_buffer</code> if frames received
--------	------------	--

Returns

`rx_err_t` Error code indicating aggregate poll result

Return values

<i>k_rx_ok</i>	At least one frame received on any channel (success)
<i>k_rx_err_timeout</i>	No frames received on any channel (normal, not an error)
<i>k_rx_err_invalid_arg</i>	<code>mgr</code> is nullptr
<i>k_rx_err_invalid_state</i>	<code>mgr</code> not initialized
<i>k_rx_err_crc</i>	CRC verification failed on received frame (data corruption)
<i>(other)</i>	Transport layer critical errors propagated

Precondition

`mgr` must be non-NULL

`mgr` must be initialized

At least one channel should be enabled (`usb_handle` or `spi_handle` non-NULL)

Postcondition

User callback invoked for each successfully received frame

ASCII output sent to Port 1 if enabled

Note

Non-blocking - always returns immediately

`k_rx_err_timeout` is NORMAL - means no data available (not an error)

User callback execution time is NOT included in performance measurements

Warning

Call this function regularly (e.g., 100 Hz) to avoid buffer overruns

User callback must not block for extended periods

Performance:

- No frames available: ~15 us (both channels timeout, fast path)
- USB frame received: ~100-250 us (USB bulk transfer + callback)
- SPI frame received: ~50-150 us (SPI hardware transfer + callback)
- Both frames: ~150-400 us (both channels + callbacks)
- With ASCII debugging: Add ~50-150 us per frame

Typical Usage - ThreadX Communication Task:

```
static void comm_task_entry(ULONG thread_input)
{
    (void)thread_input;

    // Initialize communication manager
    // ... (see rx_comm_manager_init() documentation) ...

    // Main polling loop @ 100 Hz
    while (1) {
        // Poll all channels (non-blocking)
        rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);

        // Handle critical errors (not timeout)
        if (err != k_rx_ok && err != k_rx_err_timeout) {
            // Log error, implement recovery strategy
            log_error("Comm poll failed: %d", err);
            tx_thread_sleep(100); // 1 second backoff
            continue;
        }

        // Timeout is normal - just means no data available

        // Sleep 10 ticks (10 ms @ 1 kHz tick rate = 100 Hz polling)
        tx_thread_sleep(10);
    }
}
```

Error Handling - Distinguishing Timeout from Errors:

```
rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
if (err == k_rx_ok) {
    // At least one frame received and processed
} else if (err == k_rx_err_timeout) {
    // No frames available - this is NORMAL, not an error
} else {
    // Critical error - handle appropriately
    handle_comm_error(err);
}
```

See also

[internal_poll_usb\(\)](#) USB channel polling implementation
[internal_poll_spi\(\)](#) SPI channel polling implementation
[internal_handle_frame\(\)](#) Frame dispatch
[rx_comm_manager_init\(\)](#) Initialization with channel configuration

Since

Version 1.0.0

Definition at line 1385 of file `rx_comm_manager.c`.

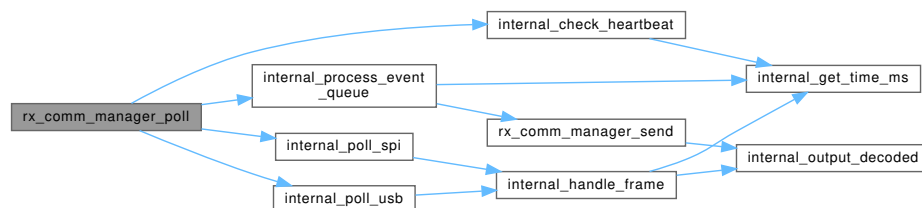
```

01386 {
01387     /* Rule 5: Pre-condition validation */
01388     if (mgr == nullptr) {
01389         return k_rx_err_invalid_arg;
01390     }
01391     if (!mgr->initialized) {
01392         return k_rx_err_invalid_state;
01393     }
01394
01395     bool received = false;
01396
01397     /* Poll USB channel */
01398     rx_err_t usb_err = internal_poll_usb(mgr);
01399     if (usb_err == k_rx_ok) {
01400         received = true;
01401     } else if (usb_err != k_rx_err_timeout) {
01402         /* Propagate non-timeout errors immediately */
01403         rx_log_error_val(s_tag, "USB poll error", usb_err);
01404         return usb_err;
01405     }
01406
01407     /* Poll SPI channel */
01408     rx_err_t spi_err = internal_poll_spi(mgr);
01409     if (spi_err == k_rx_ok) {
01410         received = true;
01411     } else if (spi_err != k_rx_err_timeout) {
01412         /* Propagate non-timeout errors immediately */
01413         rx_log_error_val(s_tag, "SPI poll error", spi_err);
01414         return spi_err;
01415     }
01416
01417     /* Process event queue: send one queued event per poll cycle */
01418     internal_process_event_queue(mgr);
01419
01420     /* Heartbeat: check implicit timeout on each transport */
01421     internal_check_heartbeat(mgr);
01422
01423     return received ? k_rx_ok : k_rx_err_timeout;
01424 }

```

References `rx_comm_manager_t::initialized`, `internal_check_heartbeat()`, `internal_poll_spi()`, `internal_poll_usb()`, `internal_process_event_queue()`, and `s_tag`.

Here is the call graph for this function:



4.3.3.15 rx_comm_manager_process_retransmits()

```

rx_err_t rx_comm_manager_process_retransmits (
    rx_comm_manager_t * mgr,
    uint32_t current_time_ms) [nodiscard]

```

Process pending retransmissions on SPI channel.

Thin pass-through to `rx_spi_comm_process_retransmits()` on the SPI handle. Call this from the communication task polling loop.

Parameters

in,out	<i>mgr</i>	Communication manager
--------	------------	-----------------------

in	<i>current_time_ms</i>	Current system time in milliseconds
----	------------------------	-------------------------------------

Returns

rx_err_t Error code from rx_spi_comm_process_retransmits()

Return values

<i>k_rx_ok</i>	No action needed or retransmit sent
<i>k_rx_err_invalid_arg</i>	mgr is nullptr
<i>k_rx_err_invalid_state</i>	mgr not initialized
<i>k_rx_err_retry_limit</i>	Max retries exceeded

Since

Version 1.0.0

Definition at line 2032 of file [rx_comm_manager.c](#).

```

02033 {
02034     if (mgr == nullptr) {
02035         return k_rx_err_invalid_arg;
02036     }
02037
02038     if (!mgr->initialized) {
02039         return k_rx_err_invalid_state;
02040     }
02041
02042     if (mgr->spi_handle == nullptr) {
02043         return k_rx_ok; /* No SPI channel configured */
02044     }
02045
02046     return rx_spi_comm_process_retransmits(mgr->spi_handle, current_time_ms);
02047 }
```

References [rx_comm_manager_t::initialized](#), and [rx_comm_manager_t::spi_handle](#).

4.3.3.16 rx_comm_manager_respond()

```

rx_err_t rx_comm_manager_respond (
    rx_comm_manager_t * mgr,
    rx_comm_channel_t channel,
    const uint8_t * payload,
    uint32_t payload_len) [nodiscard]
```

Send response frame on specified channel (convenience wrapper).

Send response on same channel as received frame.

Convenience function for sending response frames. Automatically sets frame type to `k_frame_type_response` and flags to `k_frame_flag_none`. Simplifies common use case of responding to commands/requests.

Algorithm:

1. Build [rx_comm_send_params_t](#) with:

- channel: User-specified
 - type: `k_frame_type_response` (fixed)
 - flags: `k_frame_flag_none` (fixed)
 - payload: User-specified
 - payload_len: User-specified
2. Call `rx_comm_manager_send()` with constructed params
 3. Return result from `rx_comm_manager_send()`

When to Use:

- Responding to commands with telemetry data
- Sending acknowledgments
- Replying to requests

When NOT to Use:

- Need custom frame type (use `rx_comm_manager_send` instead)
- Need custom flags (use `rx_comm_manager_send` instead)

Parameters

in	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via <code>rx_comm_manager_init()</code>
in	<i>channel</i>	Channel to send response on • Valid range: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> • Constraints: Channel handle must be configured
in	<i>payload</i>	Response payload data • Valid range: Non-NULL if <code>payload_len > 0</code>, may be NULL if <code>payload_len == 0</code> • Constraints: Must contain valid data for <code>payload_len</code> bytes • Lifetime: Copied immediately, does not need to persist
in	<i>payload_len</i>	Response payload length in bytes • Valid range: <code>[0, k_frame_max_payload]</code> (0-255 bytes) • Units: Bytes

Returns

`rx_err_t` Error code (same as `rx_comm_manager_send`)

Return values

<i>k_rx_ok</i>	Response sent successfully
----------------	----------------------------

<code>k_rx_err_invalid_arg</code>	mgr is nullptr or payload invalid
<code>k_rx_err_invalid_state</code>	Manager not initialized or channel not configured
<i>(other)</i>	Errors propagated from rx_comm_manager_send()

Precondition

Same preconditions as [rx_comm_manager_send\(\)](#)

Postcondition

Response frame transmitted with type=response, flags=none

Note

Equivalent to calling [rx_comm_manager_send\(\)](#) with type=k_frame_type_response

This is a thin wrapper - no additional validation performed

Example - Send Telemetry Response:

```
// Received a telemetry request on USB, send response
static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
{
    uint8_t telemetry[16];

    // Gather telemetry data
    telemetry[0] = get_temperature_celsius();
    telemetry[1] = get_current_ma();
    // ... fill remaining fields ...

    // Send response on same channel request came from
    rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                          channel,
                                          telemetry,
                                          sizeof(telemetry));

    if (err != k_rx_ok) {
        // Handle send error
    }
}
```

Example - Send Empty Acknowledgment:

```
// Send empty ACK response
rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
                                       k_comm_channel_usb,
                                       NULL, // No payload
                                       0);    // Zero length
```

See also

[rx_comm_manager_send\(\)](#) Full send function with all parameters

[rx_comm_send_params_t](#) Send parameters structure

Since

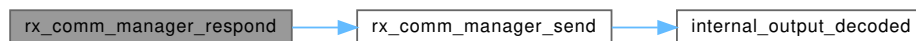
Version 1.0.0

Definition at line 1723 of file [rx_comm_manager.c](#).

```
01727 {
01728     const rx_comm_send_params_t params = {
01729         .channel    = channel,
01730         .type       = k_frame_type_response,
01731         .flags      = k_frame_flag_none,
01732         .payload    = payload,
01733         .payload_len = payload_len,
01734     };
01735     return rx_comm_manager_send(mgr, &params);
01736 }
```

References [rx_comm_manager_send\(\)](#).

Here is the call graph for this function:



4.3.3.17 rx_comm_manager_send()

```
rx_err_t rx_comm_manager_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send frame on specified communication channel.

Send frame on specific channel.

Routes frame transmission to the appropriate transport layer (USB or SPI) based on specified channel. Supports configurable frame type, flags, and variable-length payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.

Algorithm:

1. **Validate parameters** (NASA Rule 5):
 - mgr and params pointers non-NULL
 - Manager initialized flag
 - Payload pointer valid if payload_len > 0
 - payload_len <= k_frame_max_payload (255 bytes)
2. **Route to channel:**
 - USB: Verify usb_handle non-NULL, call rx_usb_comm_send()
 - SPI: Verify spi_handle non-NULL, call rx_spi_comm_send()
 - Invalid channel: Return k_rx_err_invalid_arg
3. **On success, if ASCII debugging enabled:**
 - Build temporary rx_frame_t from params
 - Format as ASCII via [internal_output_decoded\(\)](#)
 - Send to USB Port 1 (TX direction)

Performance:

- USB: ~30-150 us (depends on payload size, USB bulk transfer)
- SPI: ~20-100 us (depends on payload size, SPI hardware transfer)
- ASCII debugging: Add ~50-150 us if enabled

Parameters

in	<i>mgr</i>	Communication manager handle • Valid range: Non-nullptr to initialized handle • Constraints: Must be initialized via rx_comm_manager_init() • Side effects: May modify ascii_buffer if decoded output enabled
----	------------	--

in	<i>params</i>	Send parameters structure • Valid range: Non-nullptr to valid rx_comm_send_params_t • Constraints: All fields must be valid: – channel: <code>k_comm_channel_usb</code> or <code>k_comm_channel_spi</code> – type: Frame type (command, request, response, etc.) – flags: Frame flags (<code>k_frame_flag_none</code> or specific flags) – payload: Non-NULL if <code>payload_len > 0</code>, may be NULL if <code>payload_len == 0</code> – <code>payload_len</code>: <code>[0, k_frame_max_payload]</code> (0-255 bytes) • Lifetime: Not stored - values used immediately
----	---------------	--

Returns

`rx_err_t` Error code indicating send success or failure

Return values

<i>k_rx_ok</i>	Frame sent successfully
<i>k_rx_err_invalid_arg</i>	mgr or params is nullptr, or payload invalid
<i>k_rx_err_invalid_arg</i>	<code>payload_len > k_frame_max_payload</code> (255 bytes)
<i>k_rx_err_invalid_arg</i>	Invalid channel specified
<i>k_rx_err_invalid_arg</i>	Payload is nullptr but <code>payload_len > 0</code>
<i>k_rx_err_invalid_state</i>	Manager not initialized
<i>k_rx_err_invalid_state</i>	Channel not configured (handle is nullptr)
<i>k_rx_err_hw</i>	USB/SPI hardware error (transport layer)
<i>(other)</i>	Transport layer errors propagated

Precondition

mgr must be non-NULL and initialized

params must be non-NULL with all valid fields

Channel handle (USB/SPI) must be initialized

If `payload_len > 0`, payload must point to valid data

Postcondition

Frame transmitted on specified channel if successful

ASCII output sent to Port 1 if enabled and successful

Note

Payload sequence number and CRC-32 managed by transport layer

ASCII debugging adds overhead - disable in production

Warning

Ensure channel is ready before sending (use rx_comm_manager_channel_ready)

Send Parameters Structure:

```
typedef struct {
    rx_comm_channel_t channel;    // USB or SPI
    uint8_t type;                // Frame type
    uint8_t flags;               // Frame flags
    const uint8_t* payload;      // Payload data
    uint32_t payload_len;        // Payload length [0, 255]
} rx_comm_send_params_t;
```

Example - Send Command on USB:

```
uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

rx_comm_send_params_t params = {
    .channel = k_comm_channel_usb,
    .type = k_frame_type_command,
    .flags = k_frame_flag_none,
    .payload = cmd_payload,
    .payload_len = sizeof(cmd_payload),
};

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
if (err == k_rx_ok) {
    // Command sent successfully
}
```

Example - Send Empty Frame (No Payload):

```
rx_comm_send_params_t params = {
    .channel = k_comm_channel_spi,
    .type = k_frame_type_response,
    .flags = k_frame_flag_ack,
    .payload = nullptr,        // No payload
    .payload_len = 0,          // Zero length
};

rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
```

See also

[rx_comm_manager_respond\(\)](#) Convenience wrapper for response frames

[rx_comm_send_params_t](#) Send parameters structure

[rx_usb_comm_send\(\)](#) USB CDC send implementation

[rx_spi_comm_send\(\)](#) SPI send implementation

Since

Version 1.0.0

Definition at line 1540 of file rx_comm_manager.c.

```
01541 {
01542     /* Rule 5: Pre-condition validation */
01543     if (mgr == nullptr || params == nullptr) {
01544         rx_log_error(s_tag, "Send failed: NULL arg");
01545         return k_rx_err_invalid_arg;
01546     }
01547     if (!mgr->initialized) {
01548         rx_log_error(s_tag, "Send failed: not initialized");
01549         return k_rx_err_invalid_state;
01550     }
```

```

01551 if (params->payload == nullptr && params->payload_len > 0) {
01552     rx_log_error(s_tag, "Send failed: NULL payload with nonzero len");
01553     return k_rx_err_invalid_arg;
01554 }
01555 /* Validate payload length is within frame limits */
01556 if (params->payload_len > k_frame_max_payload) {
01557     rx_log_error_val(s_tag, "Send failed: payload too large", params->payload_len);
01558     return k_rx_err_invalid_arg;
01559 }
01560
01561 /* Send on appropriate channel */
01562 rx_err_t err = k_rx_ok;
01563 switch (params->channel) {
01564     case k_comm_channel_usb:
01565         if (mgr->usb_handle == nullptr) {
01566             rx_log_error(s_tag, "Send failed: USB handle NULL");
01567             return k_rx_err_invalid_state;
01568         }
01569         err = rx_usb_comm_send(mgr->usb_handle,
01570                               params->type,
01571                               params->flags,
01572                               params->payload,
01573                               params->payload_len);
01574         break;
01575
01576     case k_comm_channel_spi:
01577         if (mgr->spi_handle == nullptr) {
01578             rx_log_error(s_tag, "Send failed: SPI handle NULL");
01579             return k_rx_err_invalid_state;
01580         }
01581         /* Use HARQ link layer if available, otherwise raw SPI */
01582         if (mgr->spi_link != nullptr) {
01583             /* NOTE: params->flags intentionally NOT forwarded to rx_spi_link_send().
01584              * The link layer manages its own flags internally (k_frame_flag_requires_ack,
01585              * k_frame_flag_fec_enabled, k_frame_flag_retransmit) based on HARQ state.
01586              * Application-level flags in params->flags would be ignored if passed.
01587              * CLANG WARNING: bugprone-branch-clone is a false positive here.
01588              * The two branches call different functions with different signatures:
01589              * - rx_spi_link_send(link, type, payload, len) - no flags parameter
01590              * - rx_spi_comm_send(handle, type, flags, payload, len) - includes flags */
01591             err = rx_spi_link_send(mgr->spi_link, params->type, params->payload, params->payload_len);
01592         } else {
01593             err = rx_spi_comm_send(mgr->spi_handle,
01594                                   params->type,
01595                                   params->flags,
01596                                   params->payload,
01597                                   params->payload_len);
01598         }
01599         break;
01600
01601     default:
01602         rx_log_error(s_tag, "Send failed: invalid channel");
01603         return k_rx_err_invalid_arg;
01604 }
01605
01606 if (err != k_rx_ok) {
01607     rx_log_error_val(s_tag, "Transport send failed", err);
01608 }
01609
01610 /* Output decoded ASCII on success */
01611 if (err == k_rx_ok && mgr->enable_decoded_output) {
01612     /* Build a temporary frame for ASCII formatting */
01613     rx_frame_t frame = {0}; /* Zero-initialize to clear padding/unset fields */
01614     frame.header.sequence = k_frame_seq_placeholder;
01615     frame.header.length = (uint16_t)params->payload_len;
01616     frame.header.type = (uint8_t)params->type;
01617     frame.header.flags = params->flags;
01618     if (params->payload_len > 0 && params->payload_len <= k_frame_max_payload) {
01619         (void)memcpy(frame.payload, params->payload, params->payload_len);
01620     }
01621     frame.crc = k_frame_crc_placeholder;
01622     internal_output_decoded(mgr, &frame, true);
01623 }
01624
01625 return err;
01626 }
01627
01628 }

```

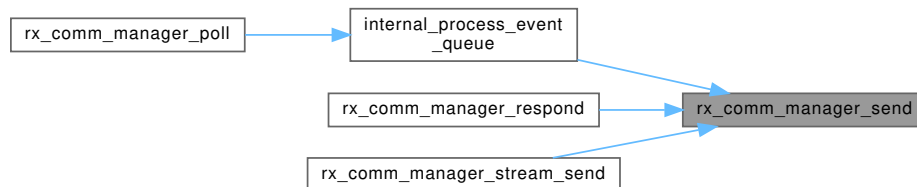
References [rx_comm_send_params_t::channel](#), [rx_comm_manager_t::enable_decoded_output](#), [rx_comm_send_params_t::flags](#), [rx_comm_manager_t::initialized](#), [internal_output_decoded\(\)](#), [k_comm_channel_spi](#), [k_comm_channel_usb](#), [k_frame_crc_placeholder](#), [k_frame_seq_placeholder](#), [rx_comm_send_params_t::payload](#), [rx_comm_send_params_t::payload_len](#), [s_tag](#), [rx_comm_manager_t::spi_handle](#), [rx_comm_manager_t::spi_link](#), [rx_comm_send_params_t::type](#), and [rx_comm_manager_t::usb_handle](#).

Referenced by [internal_process_event_queue\(\)](#), [rx_comm_manager_respond\(\)](#), and [rx_comm_manager_stream_send\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.18 rx_comm_manager_set_auto_retransmit()

```

rx_err_t rx_comm_manager_set_auto_retransmit (
    rx_comm_manager_t * mgr,
    bool enabled,
    const rx_spi_comm_retransmit_config_t * config) [nodiscard]
  
```

Enable or disable automatic retransmission on SPI channel.

Thin pass-through to [rx_spi_comm_set_auto_retransmit\(\)](#) on the SPI handle.

Parameters

in,out	<i>mgr</i>	Communication manager
in	<i>enabled</i>	true to enable, false to disable
in	<i>config</i>	Retransmit config (NULL to keep current/defaults)

Returns

`rx_err_t` Error code from [rx_spi_comm_set_auto_retransmit\(\)](#)

Since

Version 1.0.0

Definition at line 2049 of file [rx_comm_manager.c](#).

```

02052 {
02053     if (mgr == nullptr) {
02054         return k_rx_err_invalid_arg;
02055     }
02056
02057     if (!mgr->initialized) {
02058         return k_rx_err_invalid_state;
02059     }
02060
02061     if (mgr->spi_handle == nullptr) {
02062         return k_rx_err_not_supported;
02063     }
02064
02065     return rx_spi_comm_set_auto_retransmit(mgr->spi_handle, enabled, config);
02066 }
  
```

References [rx_comm_manager_t::initialized](#), and [rx_comm_manager_t::spi_handle](#).

4.3.3.19 rx_comm_manager_stream_send()

```
rx_err_t rx_comm_manager_stream_send (
    rx_comm_manager_t * mgr,
    const rx_comm_send_params_t * params) [nodiscard]
```

Send data on the stream path (fire-and-forget).

Stream path is for time-sensitive data like telemetry where stale data has no value. Sends immediately with NO retries on any transport. If the send fails, the data is simply dropped.

- USB CDC: Direct send, no retries (USB HW provides reliability)
- SPI: Direct send, no retries (stale telemetry is worthless)

Parameters

in,out	<i>mgr</i>	Pointer to initialized manager
in	<i>params</i>	Send parameters (channel, type, flags, payload, payload_len)

Returns

k_rx_ok on success
 k_rx_err_invalid_arg if mgr or params is nullptr
 k_rx_err_invalid_state if not initialized or channel not enabled

Precondition

mgr must be initialized
 params must be valid

Postcondition

Data sent or silently dropped on failure

Note

This is a thin wrapper over [rx_comm_manager_send\(\)](#) for semantic clarity

See also

[rx_comm_manager_event_send\(\)](#) For reliable command delivery

Since

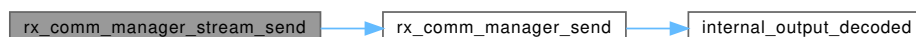
Version 1.0.0

Definition at line 1743 of file [rx_comm_manager.c](#).

```
01744 {
01745     /* Stream path: fire-and-forget, no retries, no queuing.
01746      * Telemetry and other time-sensitive data goes here.
01747      * Stale data has no value, so failure is silently accepted. */
01748     return rx_comm_manager_send(mgr, params);
01749 }
```

References [rx_comm_manager_send\(\)](#).

Here is the call graph for this function:



4.3.4 Variable Documentation

4.3.4.1 s_tag

```
const char s_tag[] = "COMM_MGR" [static]
```

Definition at line 411 of file [rx_comm_manager.c](#).

Referenced by [internal_check_heartbeat\(\)](#), [internal_handle_frame\(\)](#), [internal_poll_spi\(\)](#), [internal_process_event_queue\(\)](#), [rx_comm_manager_channel_ready\(\)](#), [rx_comm_manager_deinit\(\)](#), [rx_comm_manager_event_send\(\)](#), [rx_comm_manager_init\(\)](#), [rx_comm_manager_link_status\(\)](#), [rx_comm_manager_poll\(\)](#), and [rx_comm_manager_send\(\)](#).

4.3.4.2 s_zero_mgr

```
const rx_comm_manager_t s_zero_mgr = {} [static]
```

Zero-initialized comm manager template for stack-safe initialization.

Static const instance used to clear [rx_comm_manager_t](#) without creating a large compound literal on the stack. Lives in .rodata section.

Note

Read-only; never modified after static initialization

Warning

Do not remove - prevents stack overflow in init function

See also

[rx_comm_manager_init\(\)](#) Uses this template to zero-initialize the manager handle

Since

Version 1.0.0

Definition at line 1022 of file [rx_comm_manager.c](#).

```
01022 {};
```

Referenced by [rx_comm_manager_init\(\)](#).

4.4 rx_comm_manager.c

[Go to the documentation of this file.](#)

```
00001 /* lib/rx_comm_manager/src/rx_comm_manager.c */
00002
00003 /**
00004  * @file rx_comm_manager.c
00005  * @brief Unified Multi-Channel Communication Manager Implementation
00006  *
00007  * @details
00008  * ## Overview
00009  * Production-quality implementation of unified communication manager coordinating
00010  * multiple simultaneous channels (USB CDC and SPI) with automatic frame protocol
00011  * handling, non-blocking polling architecture, optional decoded ASCII debugging
00012  * output, and callback-based event notification.
00013  *
00014  * **Purpose**: Provide centralized, efficient coordination of independent communication
00015  * channels between RX72N MCU and Raspberry Pi 5 host, abstracting low-level transport
00016  * details while maintaining high performance and reliability.
00017  *
00018  * ## Implementation Architecture
00019  *
00020  * @dot
00021  * digraph comm_manager_impl {
00022  *   rankdir=TB;
00023  *   node [shape=box, style=rounded];
00024  *
00025  *   subgraph cluster_public {
00026  *     label="Public API Layer";
00027  *     style=filled;
00028  *     fillcolor=lightblue;
00029  *     init [label="rx_comm_manager_init()"];
00030  *     poll [label="rx_comm_manager_poll()"];
00031  *     send [label="rx_comm_manager_send()"];
00032  *     respond [label="rx_comm_manager_respond()"];
00033  *     ready [label="rx_comm_manager_channel_ready()"];
00034  *   }
00035  *
00036  *   subgraph cluster_private {
00037  *     label="Private Helper Layer";
00038  *     style=filled;
00039  *     fillcolor=lightgreen;
00040  *     poll_usb [label="internal_poll_usb()"];
00041  *     poll_spi [label="internal_poll_spi()"];
00042  *     handle_frame [label="internal_handle_frame()"];
00043  *     output_decoded [label="internal_output_decoded()"];
00044  *   }
00045  *
00046  *   subgraph cluster_transport {
00047  *     label="Transport Layer";
00048  *     style=filled;
00049  *     fillcolor=lightyellow;
00050  *     usb_comm [label="rx_usb_comm\n(USB CDC Protocol)"];
00051  *     spi_comm [label="rx_spi_comm\n(SPI Protocol)"];
00052  *     usb_hw [label="rx_usb\n(USB Port 1 Debug)"];
00053  *     frame_ascii [label="rx_frame_ascii\n(ASCII Formatting)"];
00054  *   }
00055  *
00056  *   // Public API to Private Helpers
00057  *   poll -> poll_usb;
00058  *   poll -> poll_spi;
00059  *   send -> output_decoded;
00060  *
00061  *   // Private Helpers to Transport
00062  *   poll_usb -> usb_comm [label="rx_usb_comm_receive()"];
00063  *   poll_spi -> spi_comm [label="rx_spi_comm_receive()"];
00064  *   poll_usb -> handle_frame [label="on frame"];
00065  *   poll_spi -> handle_frame [label="on frame"];
00066  *   handle_frame -> output_decoded;
00067  *   output_decoded -> frame_ascii [label="rx_frame_ascii_format()"];
00068  *   output_decoded -> usb_hw [label="rx_usb_puts()"];
00069  *   send -> usb_comm [label="rx_usb_comm_send()"];
00070  *   send -> spi_comm [label="rx_spi_comm_send()"];
00071  *
00072  *   // User callback
00073  *   handle_frame -> callback [label="user callback", style=dashed];
00074  *   callback [label="Application\nFrame Handler", shape=ellipse, fillcolor=pink, style=filled];
00075  * }
00076  * @enddot
00077  *
00078  * ## Polling State Machine
00079  *
00080  * The rx_comm_manager_poll() function implements a non-blocking state machine
00081  * that checks all enabled channels sequentially:
00082  *
```

```

00083 * @startuml
00084 * [*] --> Idle
00085 * Idle --> PollUSB : rx_comm_manager_poll() called
00086 *
00087 * state PollUSB {
00088 *     [*] --> CheckUSBHandle
00089 *     CheckUSBHandle --> USBReceive : handle != nullptr
00090 *     CheckUSBHandle --> SkipUSB : handle == nullptr
00091 *     USBReceive --> HandleUSBFrame : frame received
00092 *     USBReceive --> USBTimeout : no data
00093 *     HandleUSBFrame --> [*] : mark received=true
00094 *     USBTimeout --> [*] : continue
00095 *     SkipUSB --> [*] : continue
00096 * }
00097 *
00098 * PollUSB --> PollSPI : USB polling complete
00099 *
00100 * state PollSPI {
00101 *     [*] --> CheckSPIHandle
00102 *     CheckSPIHandle --> SPIReceive : handle != nullptr
00103 *     CheckSPIHandle --> SkipSPI : handle == nullptr
00104 *     SPIReceive --> HandleSPIFrame : frame received
00105 *     SPIReceive --> SPITimeout : no data
00106 *     HandleSPIFrame --> [*] : mark received=true
00107 *     SPITimeout --> [*] : continue
00108 *     SkipSPI --> [*] : continue
00109 * }
00110 *
00111 * PollSPI --> ReturnResult : both channels polled
00112 * ReturnResult --> Idle : return k_rx_ok if received\nk_rx_err_timeout if none
00113 * @enduml
00114 *
00115 * ## Performance Characteristics
00116 *
00117 * | Operation | Avg Time | Worst Case | Notes |
00118 * |-----|-----|-----|-----|
00119 * | **rx_comm_manager_init** | ~5 us | ~10 us | memset dominates (328 bytes) |
00120 * | **rx_comm_manager_poll** | ~15-200 us | ~500 us | Depends on channels enabled, frames available |
00121 * | **rx_comm_manager_send** | ~30-150 us | ~300 us | Depends on payload size, channel |
00122 * | **internal_poll_usb** | ~10-100 us | ~250 us | USB CDC bulk transfer latency |
00123 * | **internal_poll_spi** | ~5-50 us | ~150 us | SPI hardware transfer latency |
00124 * | **internal_handle_frame** | ~2 us | ~5 us | Callback invocation overhead |
00125 * | **internal_output_decoded** | ~50-150 us | ~300 us | ASCII formatting + USB puts |
00126 *
00127 * **Timing Analysis**:
00128 * - Measured @ 240 MHz CPU clock with -O2 optimization
00129 * - Poll time dominated by transport layer (USB/SPI receive)
00130 * - ASCII debugging adds ~50-150 us per frame (disable in production)
00131 * - Callback execution time not included (application-dependent)
00132 *
00133 * ## Memory Usage
00134 *
00135 * | Memory Type | Size | Usage |
00136 * |-----|-----|-----|
00137 * | **rx_comm_manager_t handle** | 328 bytes | Main state structure |
00138 * | **Stack (init)** | ~32 bytes | Function locals |
00139 * | **Stack (poll)** | ~80 bytes | rx_frame_t + locals |
00140 * | **Stack (send)** | ~80 bytes | rx_frame_t + locals |
00141 * | **ASCII buffer** | 256 bytes | Inside handle (for debugging) |
00142 *
00143 * **Memory Layout** (rx_comm_manager_t):
00144 * ```
00145 * | Offset | Size | Field | Purpose |
00146 * |-----|-----|-----|-----|
00147 * | 0x00 | 8 | usb_handle | Pointer to USB comm handle |
00148 * | 0x08 | 8 | spi_handle | Pointer to SPI comm handle |
00149 * | 0x10 | 8 | callback | User frame callback function |
00150 * | 0x18 | 8 | callback_ctx | User context pointer |
00151 * | 0x20 | 1 | enable_decoded_output | ASCII debug flag |
00152 * | 0x21 | 1 | initialized | Initialization flag |
00153 * | 0x22 | 2 | (padding) | Alignment |
00154 * | 0x24 | 4 | (padding) | Alignment |
00155 * | 0x28 | 256 | ascii_buffer | ASCII formatting buffer |
00156 * |-----|-----|-----|-----|
00157 * | Total: 328 bytes (0x148) |
00158 * ```
00159 *
00160 * ## Algorithm Details
00161 *
00162 * ### Polling Algorithm (rx_comm_manager_poll)
00163 *
00164 * The polling algorithm maximizes throughput by checking all channels without
00165 * blocking, returning immediately on the first error while continuing to poll
00166 * remaining channels on timeout (no data available):
00167 *
00168 * **Steps**:
00169 * 1. **Parameter validation** (NULL check, initialized flag)

```

```

00170 * 2. **Poll USB channel**:
00171 *   - If nullptr handle -> skip (k_rx_err_timeout)
00172 *   - If data available -> handle frame, mark received=true
00173 *   - If timeout -> continue to next channel
00174 *   - If error -> return error immediately (critical failure)
00175 * 3. **Poll SPI channel**:
00176 *   - Same logic as USB
00177 * 4. **Return aggregated result**:
00178 *   - k_rx_ok if ANY frame received on any channel
00179 *   - k_rx_err_timeout if NO frames received (normal, non-error condition)
00180 *   - Other error codes propagated immediately
00181 *
00182 * **Design Rationale**:
00183 *   - Non-blocking: Never blocks waiting for data
00184 *   - Fair scheduling: Both channels polled each iteration
00185 *   - Error prioritization: Critical errors returned immediately
00186 *   - Timeout tolerance: Timeout is expected, not an error
00187 *
00188 * ### Send Algorithm (rx_comm_manager_send)
00189 *
00190 * Routing algorithm with post-send ASCII debugging support:
00191 *
00192 * **Steps**:
00193 * 1. **Validate parameters** (NULL checks, payload length <= k_frame_max_payload)
00194 * 2. **Route to channel**:
00195 *   - USB: Call rx_usb_comm_send()
00196 *   - SPI: Call rx_spi_comm_send()
00197 *   - Invalid channel: Return k_rx_err_invalid_arg
00198 * 3. **On success, if ASCII output enabled**:
00199 *   - Build temporary rx_frame_t from send parameters
00200 *   - Format as ASCII via rx_frame_ascii_format()
00201 *   - Send to USB Port 1 via rx_usb_puts()
00202 *
00203 * **Performance Note**: ASCII formatting adds ~50-150 us overhead per send.
00204 * Disable in production with `cfg->enable_decoded_output = false`.
00205 *
00206 * ## Thread Safety Analysis
00207 *
00208 * **Conditional Thread Safety** - Safe if following rules are observed:
00209 *
00210 * | Scenario | Thread Safety | Mitigation Required |
00211 * |-----|-----|-----|
00212 * | **Single-threaded** | [OK] Safe | None |
00213 * | **Multi-threaded (different channels)** | [OK] Safe | Ensure each thread uses different channel |
00214 * | **Multi-threaded (same channel)** | [X] Unsafe | External mutex required |
00215 * | **Multi-threaded (different handles)** | [OK] Safe | None (independent handles) |
00216 * | **Init/Deinit from ISR** | [X] Unsafe | Call only from task context |
00217 * | **Poll/Send from ISR** | [WARN] Conditional | OK if callback is ISR-safe |
00218 *
00219 * **Recommended Architecture**:
00220 *   - Dedicate one thread to poll all channels (e.g., ThreadX communication task)
00221 *   - Send from any thread/ISR, but use external mutex if same channel
00222 *   - Ensure user callback is reentrant if called from ISR
00223 *
00224 * **NASA Power of 10 Rule 5**: All public functions validate preconditions with
00225 * explicit checks (NULL pointers, initialized flag, parameter ranges).
00226 *
00227 * ## Integration Example
00228 *
00229 * @par Complete System Setup with ThreadX:
00230 * @code{.c}
00231 * // Global communication manager handle
00232 * static rx_comm_manager_t g_comm_mgr;
00233 * static rx_usb_comm_handle_t g_usb_handle;
00234 * static rx_spi_comm_handle_t g_spi_handle;
00235 *
00236 * // User callback for frame processing
00237 * static void on_frame_received(rx_comm_channel_t channel,
00238 *                             const rx_frame_t* frame,
00239 *                             void* ctx)
00240 * {
00241 *   (void)ctx;
00242 *
00243 *   // Dispatch based on frame type
00244 *   switch (frame->header.type) {
00245 *     case k_frame_type_command:
00246 *       handle_command(channel, frame);
00247 *       break;
00248 *
00249 *     case k_frame_type_request:
00250 *       handle_request(channel, frame);
00251 *       break;
00252 *
00253 *     default:
00254 *       // Unknown frame type, ignore
00255 *       break;
00256 *   }

```

```

00257 * }
00258 *
00259 * // ThreadX communication task
00260 * static void comm_task_entry(ULONG thread_input)
00261 * {
00262 *     (void)thread_input;
00263 *
00264 *     // Initialize USB CDC communication
00265 *     rx_usb_comm_config_t usb_cfg = {
00266 *         .port = k_usb_port_proto,
00267 *     };
00268 *     rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
00269 *     if (err != k_rx_ok) {
00270 *         // Handle error
00271 *         return;
00272 *     }
00273 *
00274 *     // Initialize SPI communication
00275 *     rx_spi_comm_config_t spi_cfg = {
00276 *         .spi_channel = 0,
00277 *     };
00278 *     err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
00279 *     if (err != k_rx_ok) {
00280 *         // Handle error
00281 *         return;
00282 *     }
00283 *
00284 *     // Initialize communication manager
00285 *     rx_comm_manager_config_t cfg = {
00286 *         .usb_handle = &g_usb_handle,
00287 *         .spi_handle = &g_spi_handle,
00288 *         .callback = on_frame_received,
00289 *         .callback_ctx = nullptr,
00290 *         .enable_decoded_output = true, // Enable for development
00291 *     };
00292 *     err = rx_comm_manager_init(&g_comm_mgr, &cfg);
00293 *     if (err != k_rx_ok) {
00294 *         // Handle error
00295 *         return;
00296 *     }
00297 *
00298 *     // Main communication loop (100 Hz polling)
00299 *     while (1) {
00300 *         // Poll all channels
00301 *         err = rx_comm_manager_poll(&g_comm_mgr);
00302 *
00303 *         // Check for critical errors (not timeout)
00304 *         if (err != k_rx_ok && err != k_rx_err_timeout) {
00305 *             // Log error, attempt recovery
00306 *             tx_thread_sleep(100); // 1 second backoff
00307 *         }
00308 *
00309 *         // Sleep 10 ticks (10 ms @ 1 kHz tick rate)
00310 *         tx_thread_sleep(10);
00311 *     }
00312 * }
00313 *
00314 * // Send response from application
00315 * void send_motor_telemetry(void)
00316 * {
00317 *     uint8_t telemetry_payload[16];
00318 *     // ... fill telemetry data ...
00319 *
00320 *     // Check if USB channel is ready
00321 *     bool usb_ready = false;
00322 *     rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
00323 *         k_comm_channel_usb,
00324 *         &usb_ready);
00325 *     if (err == k_rx_ok && usb_ready) {
00326 *         // Send telemetry response
00327 *         err = rx_comm_manager_respond(&g_comm_mgr,
00328 *             k_comm_channel_usb,
00329 *             telemetry_payload,
00330 *             sizeof(telemetry_payload));
00331 *     }
00332 * }
00333 * @endcode
00334 *
00335 * ## NASA Power of 10 Compliance
00336 *
00337 * | Rule | Compliance | Implementation Notes |
00338 * |-----|-----|-----|
00339 * | **Rule 1: Control Flow** | [OK] | No goto, setjmp, longjmp, or recursion |
00340 * | **Rule 2: Loop Bounds** | [OK] | All loops have statically provable bounds |
00341 * | **Rule 3: No Dynamic Allocation** | [OK] | Zero malloc/free - all static allocation |
00342 * | **Rule 4: Function Size** | [OK] | All functions < 60 lines |
00343 * | **Rule 5: Assertions** | [OK] | Minimum 2 checks per function (nullptr, initialized) |

```

```

00344 * | **Rule 6: Data Scope** | [OK] | Variables declared at smallest scope |
00345 * | **Rule 7: Check Returns** | [OK] | All return values validated or cast to (void) |
00346 * | **Rule 8: Preprocessor** | [OK] | Minimal macros, typed enums for constants |
00347 * | **Rule 9: Pointers** | [WARN] | Function pointers used for callback (DIP) |
00348 * | **Rule 10: Compiler Warnings** | [OK] | -Wall -Wextra -Werror enabled |
00349 *
00350 * **Rule 9 Deviation**: Function pointer used for user callback to enable Dependency
00351 * Inversion Principle (SOLID) - allows application to register frame handler without
00352 * tight coupling. This is intentional and documented per project policy.
00353 *
00354 * ## SOLID Principles
00355 *
00356 * | Principle | Implementation |
00357 * |-----|-----|
00358 * | **Single Responsibility** | Module responsible ONLY for channel coordination; frame parsing/transport delegated |
00359 * | **Open/Closed** | Extensible via callback; new frame types handled by application |
00360 * | **Liskov Substitution** | Channels are interchangeable (same rx_frame_t interface) |
00361 * | **Interface Segregation** | Small focused API (7 functions); send/poll/ready separated |
00362 * | **Dependency Inversion** | Depends on abstract rx_frame_t and channel handles, not concrete implementations |
00363 *
00364 * **Key Design Decision**: The manager does NOT parse frame payloads - it only routes
00365 * frames to the user callback. This separation allows application-specific command
00366 * handling without modifying the communication layer.
00367 *
00368 * ## Module Dependencies
00369 *
00370 * **Includes**
00371 * - rx_comm_manager.h - Public API definitions
00372 * - rx_frame_ascii.h - ASCII formatting for debugging
00373 * - rx_usb.h - USB port access (Port 1 debug output)
00374 * - string.h - Standard C library (memset, memcpy)
00375 *
00376 * **Links Against**
00377 * - rx_usb_comm - USB CDC frame protocol implementation
00378 * - rx_spi_comm - SPI frame protocol implementation
00379 * - rx_frame_ascii - ASCII frame formatting
00380 * - rx_usb - USB hardware abstraction
00381 *
00382 * @date 2026-01-27
00383 * @copyright Copyright (c) 2026 STAR Project
00384 * @since Version 1.0.0
00385 *
00386 * @see rx_comm_manager.h Full API documentation
00387 * @see rx_usb_comm.h USB CDC protocol implementation
00388 * @see rx_spi_comm.h SPI protocol implementation
00389 * @see rx_frame_ascii.h ASCII debugging interface
00390 */
00391
00392 #include "rx_comm_manager.h"
00393
00394 #include <string.h>
00395
00396 #include "rx_frame_ascii.h"
00397 #include "rx_log.h"
00398 #include "rx_simulator_config.h"
00399 #include "rx_time_constants.h"
00400 #include "rx_usb.h"
00401
00402 #if !RX_IS_SIMULATOR
00403 #include "tx_api.h"
00404 #endif
00405
00406 /*
=====
00407 * Logging
00408 *
=====
00409 */
00410
00411 static const char s_tag[] = "COMM_MGR";
00412
00413 /*
=====
00414 * Private Helpers
00415 *
=====
00416 */
00417
00418 /**
00419 * @brief Get current time in milliseconds
00420 *
00421 * @details
00422 * Returns the current system time in milliseconds. On RX72N hardware, this
00423 * uses ThreadX tx_time_get() multiplied by tick period. In simulator builds,
00424 * returns 0 (timing not available).
00425 *
00426 * @return Current time in milliseconds

```

```

00427 *
00428 * @since Version 1.0.0
00429 */
00430 /** @brief Simulator time stub value (time is unavailable in simulator) */
00431 typedef enum : uint32_t {
00432     k_sim_time_ms_zero = 0, /**< Simulator returns zero (no real-time clock) */
00433 } rx_comm_sim_time_t;
00434
00435 static uint32_t internal_get_time_ms(void)
00436 {
00437     #if !RX_IS_SIMULATOR
00438         return (uint32_t)tx_time_get() * k_threadx_ms_per_tick;
00439     #else
00440         return k_sim_time_ms_zero;
00441     #endif
00442 }
00443
00444 /*
=====
00445 * Private Constants
00446 *
=====
00447 */
00448
00449 /**
00450 * @enum internal_constants_t
00451 * @brief Internal constants for communication manager implementation
00452 *
00453 * @details
00454 * Defines compile-time constants used internally by the communication manager
00455 * for timeout values, placeholder values, and buffer indexing.
00456 *
00457 * **Design Notes:**
00458 * - Non-blocking timeout (0 ms) ensures poll never blocks
00459 * - Placeholder values filled by transport layer (sequence, CRC)
00460 * - Explicit array index constant for bounds checking
00461 *
00462 * @since Version 1.0.0
00463 */
00464 typedef enum : uint32_t {
00465     /**
00466      * @brief Non-blocking receive timeout for polling
00467      * @details
00468      * Set to 0 ms to ensure rx_comm_manager_poll() never blocks waiting for data.
00469      * Transport layers (USB/SPI) return k_rx_err_timeout immediately if no data.
00470      * @par Value: 0 milliseconds
00471      */
00472     k_receive_timeout_ms = 0,
00473
00474     /**
00475      * @brief Placeholder for frame sequence number (filled by transport layer)
00476      * @details
00477      * Sequence number is managed by transport layer (rx_usb_comm/rx_spi_comm)
00478      * to detect dropped frames. Manager does not manipulate sequence numbers.
00479      * @par Value: 0
00480      */
00481     k_frame_seq_placeholder = 0,
00482
00483     /**
00484      * @brief Placeholder for frame CRC-32 (filled by transport layer)
00485      * @details
00486      * CRC-32 calculated and verified by transport layer. Manager builds temp
00487      * frame with placeholder CRC for ASCII formatting only.
00488      * @par Value: 0
00489      */
00490     k_frame_crc_placeholder = 0,
00491
00492     /**
00493      * @brief First index of ascii_buffer for post-condition validation
00494      * @details
00495      * Used in rx_comm_manager_init() to verify memset properly cleared buffer.
00496      * First byte must be '\0' after zero-fill.
00497      * @par Value: 0
00498      */
00499     k_ascii_buffer_first_idx = 0,
00500
00501     /**
00502      * @brief Typed zero constant for uint32_t comparisons
00503      * @details
00504      * Used for payload length checks and other uint32_t zero comparisons to avoid
00505      * magic number 0. Provides type safety and self-documenting code.
00506      * @par Value: 0
00507      */
00508     k_zero_u32 = 0,
00509 } internal_constants_t;
00510
00511 /*

```

```

=====
00512 * Private Functions
00513 *
=====
00514 */
00515
00516 /**
00517  * @brief Output decoded ASCII frame to USB Port 1 for debugging
00518  *
00519  * @details
00520  * Formats the given frame as human-readable ASCII text and outputs to USB Port 1
00521  * (decoded debug port) if decoded output is enabled. Used for development/debugging
00522  * to monitor frame traffic without binary protocol parsing.
00523  *
00524  * **Algorithm:**
00525  * 1. Validate parameters (NULL checks)
00526  * 2. Check if decoded output is enabled (cfg->enable_decoded_output)
00527  * 3. Format frame as ASCII via rx_frame_ascii_format()
00528  * 4. Send ASCII string to USB Port 1 via rx_usb_puts()
00529  *
00530  * **Performance:** ~50-150 us depending on frame size (ASCII formatting overhead).
00531  * Disable in production for optimal performance.
00532  *
00533  * @param[in,out] mgr Communication manager handle (uses ascii_buffer)
00534  * - **Valid range:** Non-nullptr to initialized handle
00535  * - **Constraints:** Must have valid ascii_buffer (256 bytes)
00536  * - **Side effects:** Modifies mgr->ascii_buffer content
00537  *
00538  * @param[in] frame Frame to format and output
00539  * - **Valid range:** Non-nullptr to valid rx_frame_t
00540  * - **Constraints:** Header and payload must be valid
00541  * - **Units:** Binary frame structure
00542  *
00543  * @param[in] is_tx Direction flag (true = TX, false = RX)
00544  * - **Valid range:** true (outgoing frame) or false (incoming frame)
00545  * - **Purpose:** Prefix ASCII output with "TX:" or "RX:" for clarity
00546  *
00547  * @return void (no return value - errors silently ignored)
00548  *
00549  * @pre mgr must be initialized via rx_comm_manager_init()
00550  * @pre frame must be a valid frame structure
00551  * @post mgr->ascii_buffer content is modified (temporary)
00552  * @post ASCII output sent to USB Port 1 if enabled and successful
00553  *
00554  * @note Not thread-safe - caller must ensure exclusive access to mgr->ascii_buffer
00555  * @note Errors are silently ignored (formatting/USB send failures)
00556  * @note USB Port 1 must be configured for ASCII output to appear
00557  *
00558  * @par Example Output:
00559  * ```
00560  * RX: [USB] Type=0x01 Flags=0x00 Len=8 Seq=42 Payload=0102030405060708 CRC=ABCD1234
00561  * TX: [SPI] Type=0x02 Flags=0x80 Len=4 Seq=43 Payload=01020304 CRC=5678CDEF
00562  * ```
00563  *
00564  * @see rx_frame_ascii_format() Frame-to-ASCII conversion
00565  * @see rx_usb_puts() USB debug output
00566  *
00567  * @since Version 1.0.0
00568  */
00569 static void internal_output_decoded(rx_comm_manager_t* mgr, const rx_frame_t* frame, bool is_tx)
00570 {
00571     /* Rule 5: Pre-condition validation - defensive NULL checks */
00572     if (mgr == nullptr || frame == nullptr) {
00573         return;
00574     }
00575
00576     if (!mgr->enable_decoded_output) {
00577         return;
00578     }
00579
00580     uint32_t len = 0;
00581     rx_err_t err =
00582         rx_frame_ascii_format(frame, is_tx, mgr->ascii_buffer, sizeof(mgr->ascii_buffer), &len);
00583     if (err == k_rx_ok && len > 0) {
00584         (void)rx_usb_puts(k_usb_port_decoded, mgr->ascii_buffer);
00585     }
00586 }
00587
00588 /**
00589  * @brief Handle received frame by invoking user callback and optional ASCII output
00590  *
00591  * @details
00592  * Central frame dispatch point for all channels. Performs optional ASCII debugging
00593  * output, then invokes user-registered callback with frame data. This function
00594  * isolates callback invocation logic from channel-specific polling.
00595  *
00596  * **Algorithm:**

```

```

00597 * 1. Validate parameters (NULL checks, channel range)
00598 * 2. Output decoded ASCII to Port 1 if enabled (RX direction)
00599 * 3. Invoke user callback if registered
00600 *
00601 * **Design Rationale**: Centralizing frame handling logic ensures consistent
00602 * behavior across USB and SPI channels (single point of dispatch).
00603 *
00604 * @param[in,out] mgr Communication manager handle
00605 * - **Valid range**: Non-nullptr to initialized handle
00606 * - **Constraints**: Must be initialized via rx_comm_manager_init()
00607 * - **Side effects**: May modify ascii_buffer if decoded output enabled
00608 *
00609 * @param[in] channel Channel the frame was received on
00610 * - **Valid range**: k_comm_channel_usb or k_comm_channel_spi
00611 * - **Constraints**: Must be < k_comm_channel_count
00612 * - **Purpose**: Passed to user callback for channel identification
00613 *
00614 * @param[in] frame Received frame data
00615 * - **Valid range**: Non-nullptr to valid, CRC-verified frame
00616 * - **Constraints**: Already validated by transport layer
00617 * - **Lifetime**: Valid only for duration of this function call
00618 *
00619 * @return void (no return value - errors silently handled)
00620 *
00621 * @pre mgr must be initialized
00622 * @pre frame must be valid and CRC-verified by transport layer
00623 * @pre channel must be in valid range [0, k_comm_channel_count)
00624 * @post User callback invoked if registered (callback may be NULL)
00625 * @post ASCII output sent to Port 1 if enabled
00626 *
00627 * @note Thread safety depends on user callback implementation
00628 * @note Callback errors are not returned (callback should handle internally)
00629 * @warning User callback must not block for extended periods (affects polling rate)
00630 *
00631 * @par Performance:
00632 * - Without decoded output: ~2 us (callback overhead only)
00633 * - With decoded output: ~50-150 us (ASCII formatting + USB puts)
00634 *
00635 * @see internal_output_decoded() ASCII debugging output
00636 * @see rx_comm_frame_callback_t User callback type
00637 *
00638 * @since Version 1.0.0
00639 */
00640 static void
00641 internal_handle_frame(rx_comm_manager_t* mgr, rx_comm_channel_t channel, const rx_frame_t* frame)
00642 {
00643     /* Rule 5: Pre-condition validation - defensive NULL checks */
00644     if (mgr == nullptr || frame == nullptr) {
00645         return;
00646     }
00647
00648     /* Validate channel is within expected range */
00649     if (channel >= k_comm_channel_count) {
00650         return;
00651     }
00652
00653     /* Update heartbeat: any valid frame resets the implicit timeout timer.
00654      * Per the dual-detection model, this is the primary link health
00655      * mechanism (200ms implicit timeout). */
00656     rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[channel];
00657     hb->last_rx_ms = internal_get_time_ms();
00658     if (hb->status != k_link_status_healthy) {
00659         rx_comm_link_status_t old_status = hb->status;
00660         hb->status = k_link_status_healthy;
00661         if (old_status == k_link_status_dead) {
00662             rx_log_info(s_tag, "Link recovered: frame received");
00663         } else {
00664             rx_log_info(s_tag, "Link established: first frame received");
00665         }
00666         if (mgr->link_status_cb != nullptr && old_status != k_link_status_healthy) {
00667             mgr->link_status_cb(channel, k_link_status_healthy, mgr->link_status_ctx);
00668         }
00669     }
00670
00671     rx_log_debug_val(s_tag, "Frame received, type", (uint8_t)frame->header.type);
00672
00673     /* Output decoded ASCII to Port 1 */
00674     internal_output_decoded(mgr, frame, false);
00675
00676     /* Invoke user callback if set */
00677     if (mgr->callback != nullptr) {
00678         mgr->callback(channel, frame, mgr->callback_ctx);
00679     }
00680 }
00681
00682 /**
00683  * @brief Poll USB channel for incoming frames (non-blocking)

```

```

00684 *
00685 * @details
00686 * Attempts to receive one frame from USB CDC channel with zero timeout (non-blocking).
00687 * If frame is successfully received, it is dispatched via internal_handle_frame().
00688 * Timeout (no data available) is expected and normal - not an error condition.
00689 *
00690 * **Algorithm:**
00691 * 1. Validate mgr pointer (NULL check)
00692 * 2. Skip if USB handle is nullptr (channel not configured)
00693 * 3. Call rx_usb_comm_receive() with 0 ms timeout
00694 * 4. On success: dispatch frame via internal_handle_frame()
00695 * 5. On timeout: return k_rx_err_timeout (expected, not error)
00696 * 6. On other errors: propagate error code
00697 *
00698 * **Design Note**: This function treats timeout as non-error. The calling poll
00699 * function (rx_comm_manager_poll) decides whether overall operation succeeded
00700 * based on aggregate results from all channels.
00701 *
00702 * @param[in,out] mgr Communication manager handle
00703 * - **Valid range**: Non-nullptr to initialized handle
00704 * - **Constraints**: Must have valid usb_handle if USB channel enabled
00705 * - **Side effects**: May invoke callback, modify ascii_buffer
00706 *
00707 * @return rx_err_t Error code indicating poll result
00708 * @retval k_rx_ok Frame received and handled successfully
00709 * @retval k_rx_err_timeout No data available (normal, expected condition)
00710 * @retval k_rx_err_invalid_arg mgr is nullptr
00711 * @retval k_rx_err_crc CRC verification failed (corrupted frame)
00712 * @retval k_rx_err_invalid_state USB peripheral not ready
00713 * @retval (other) Transport layer errors propagated
00714 *
00715 * @pre mgr must be non-NULL
00716 * @pre If USB channel enabled, usb_handle must be initialized
00717 * @post On k_rx_ok: callback invoked, ASCII output sent (if enabled)
00718 * @post On timeout: no side effects (no frame received)
00719 *
00720 * @note Non-blocking - always returns immediately
00721 * @note Timeout is NOT an error - it means no data is available
00722 * @warning Do not call if usb_handle is uninitialized (will return timeout)
00723 *
00724 * @par Performance:
00725 * - No data available: ~10 us (fast path)
00726 * - Frame received: ~100-250 us (USB bulk transfer + callback)
00727 *
00728 * @see internal_handle_frame() Frame dispatch
00729 * @see rx_usb_comm_receive() USB CDC receive implementation
00730 *
00731 * @since Version 1.0.0
00732 */
00733 static rx_err_t internal_poll_usb(rx_comm_manager_t* mgr)
00734 {
00735     /* Rule 5: Pre-condition validation - prevent NULL dereference */
00736     if (mgr == nullptr) {
00737         return k_rx_err_invalid_arg;
00738     }
00739
00740     if (mgr->usb_handle == nullptr) {
00741         return k_rx_err_timeout;
00742     }
00743
00744     rx_frame_t frame;
00745     rx_err_t err = rx_usb_comm_receive(mgr->usb_handle, &frame, k_receive_timeout_ms);
00746
00747     if (err == k_rx_ok) {
00748         internal_handle_frame(mgr, k_comm_channel_usb, &frame);
00749         return k_rx_ok;
00750     }
00751
00752     return err;
00753 }
00754
00755 /**
00756 * @brief Poll SPI channel for incoming frames (non-blocking)
00757 *
00758 * @details
00759 * Attempts to receive one frame from SPI channel with zero timeout (non-blocking).
00760 * If frame is successfully received, it is dispatched via internal_handle_frame().
00761 * Timeout (no data available) is expected and normal - not an error condition.
00762 *
00763 * **Algorithm:**
00764 * 1. Validate mgr pointer (NULL check)
00765 * 2. Skip if SPI handle is nullptr (channel not configured)
00766 * 3. Call rx_spi_comm_receive() with 0 ms timeout
00767 * 4. On success: dispatch frame via internal_handle_frame()
00768 * 5. On timeout: return k_rx_err_timeout (expected, not error)
00769 * 6. On other errors: propagate error code
00770 *

```

```

00771 * **Design Note**: Identical logic to internal_poll_usb() but for SPI channel.
00772 * Code duplication is intentional (NASA Rule 4: keep functions short and simple).
00773 *
00774 * @param[in,out] mgr Communication manager handle
00775 * - **Valid range**: Non-nullptr to initialized handle
00776 * - **Constraints**: Must have valid spi_handle if SPI channel enabled
00777 * - **Side effects**: May invoke callback, modify ascii_buffer
00778 *
00779 * @return rx_err_t Error code indicating poll result
00780 * @retval k_rx_ok Frame received and handled successfully
00781 * @retval k_rx_err_timeout No data available (normal, expected condition)
00782 * @retval k_rx_err_invalid_arg mgr is nullptr
00783 * @retval k_rx_err_crc CRC verification failed (corrupted frame)
00784 * @retval k_rx_err_invalid_state SPI peripheral not ready
00785 * @retval (other) Transport layer errors propagated
00786 *
00787 * @pre mgr must be non-NULL
00788 * @pre If SPI channel enabled, spi_handle must be initialized
00789 * @post On k_rx_ok: callback invoked, ASCII output sent (if enabled)
00790 * @post On timeout: no side effects (no frame received)
00791 *
00792 * @note Non-blocking - always returns immediately
00793 * @note Timeout is NOT an error - it means no data is available
00794 * @warning Do not call if spi_handle is uninitialized (will return timeout)
00795 *
00796 * @par Performance:
00797 * - No data available: ~5 us (fast path)
00798 * - Frame received: ~50-150 us (SPI hardware transfer + callback)
00799 *
00800 * @see internal_handle_frame() Frame dispatch
00801 * @see rx_spi_comm_receive() SPI receive implementation
00802 *
00803 * @since Version 1.0.0
00804 */
00805 static rx_err_t internal_poll_spi(rx_comm_manager_t* mgr)
00806 {
00807     /* Rule 5: Pre-condition validation - prevent NULL dereference */
00808     if (mgr == nullptr) {
00809         return k_rx_err_invalid_arg;
00810     }
00811
00812     if (mgr->spi_handle == nullptr) {
00813         return k_rx_err_timeout;
00814     }
00815
00816     /* If SPI link layer is available, use it for HARQ decoding
00817     * NOTE: Using static buffers to avoid stack overflow (~3KB total).
00818     * Safe because comm_manager_poll is single-threaded (called from comm_task). */
00819     if (mgr->spi_link != nullptr) {
00820         static rx_spi_link_receive_result_t s_link_result;
00821         rx_err_t err = rx_spi_link_receive(mgr->spi_link, &s_link_result, k_receive_timeout_ms);
00822
00823         if (err == k_rx_ok) {
00824             /* Defensive bounds check: Validate payload length before copy */
00825             if (s_link_result.payload_len > k_harq_max_payload ||
00826                 s_link_result.payload_len > sizeof(((rx_frame_t*)nullptr)->payload)) {
00827                 rx_log_error_val(s_tag, "SPI link receive: payload too large", s_link_result.payload_len);
00828                 return k_rx_err_invalid_size;
00829             }
00830
00831             /* Convert link result to rx_frame_t for internal_handle_frame() */
00832             static rx_frame_t s_frame1;
00833             s_frame1 = (rx_frame_t){0};
00834             s_frame1.header.sequence = s_link_result.sequence;
00835             s_frame1.header.length = (uint16_t)s_link_result.payload_len;
00836             s_frame1.header.type = s_link_result.frame_type;
00837
00838             /* Preserve retransmit and FEC flags from link layer result */
00839             s_frame1.header.flags = k_frame_flag_none;
00840             if (s_link_result.is_retransmit) {
00841                 s_frame1.header.flags |= k_frame_flag_retransmit;
00842             }
00843             if (s_link_result.fec_decoded) {
00844                 s_frame1.header.flags |= k_frame_flag_fec_enabled;
00845             }
00846
00847             if (s_link_result.payload_len > k_zero_u32) {
00848                 (void)memcpy(s_frame1.payload, s_link_result.payload, s_link_result.payload_len);
00849             }
00850             internal_handle_frame(mgr, k_comm_channel_spi, &s_frame1);
00851             return k_rx_ok;
00852         }
00853     }
00854     return err;
00855 }
00856
00857 /* Fallback: raw SPI without HARQ */

```

```

00858 static rx_frame_t s_frame2;
00859 rx_err_t err = rx_spi_comm_receive(mgr->spi_handle, &s_frame2, k_receive_timeout_ms);
00860
00861 if (err == k_rx_ok) {
00862     internal_handle_frame(mgr, k_comm_channel_spi, &s_frame2);
00863     return k_rx_ok;
00864 }
00865
00866 return err;
00867 }
00868
00869 /*
=====
00870 * Event Queue Helpers
00871 *
=====
00872 */
00873
00874 /**
00875 * @brief Process one event from the FIFO queue
00876 *
00877 * @details
00878 * Dequeues the oldest event and attempts to send it. For SPI events, if the
00879 * send fails, the event is re-enqueued with incremented retry count (up to
00880 * k_event_max_retries). For USB events, failure is simply dropped (fire-and-forget).
00881 *
00882 * @param[in,out] mgr Communication manager handle
00883 *
00884 * @pre mgr must be non-NULL and initialized
00885 * @post One event processed (sent or dropped on max retries)
00886 *
00887 * @since Version 1.0.0
00888 */
00889 static void internal_process_event_queue(rx_comm_manager_t* mgr)
00890 {
00891     if (mgr == nullptr || mgr->event_queue_count == 0) {
00892         return;
00893     }
00894
00895     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_tail];
00896     if (!entry->occupied) {
00897         return;
00898     }
00899
00900     /* Check backoff: skip if retry time hasn't been reached yet */
00901     const uint32_t now_ms = internal_get_time_ms();
00902     if (entry->retries > 0 && now_ms < entry->next_retry_time_ms) {
00903         return; /* Backoff period not elapsed, try again later */
00904     }
00905
00906     /* Build send params from queued entry */
00907     const rx_comm_send_params_t params = {
00908         .channel = entry->channel,
00909         .type = entry->type,
00910         .flags = entry->flags,
00911         .payload = entry->payload,
00912         .payload_len = entry->payload_len,
00913     };
00914
00915     rx_err_t err = rx_comm_manager_send(mgr, &params);
00916
00917     if (err == k_rx_ok || entry->channel == k_comm_channel_usb) {
00918         /* Success, or USB fire-and-forget: dequeue */
00919         entry->occupied = false;
00920         entry->retries = 0;
00921         entry->next_retry_time_ms = 0;
00922         mgr->event_queue_tail = (mgr->event_queue_tail + 1) % k_comm_event_queue_depth;
00923         mgr->event_queue_count--;
00924
00925         if (err == k_rx_ok) {
00926             rx_log_debug_val(s_tag, "Event sent, queue depth", mgr->event_queue_count);
00927         } else {
00928             rx_log_warn_val(s_tag, "USB event dropped, queue depth", mgr->event_queue_count);
00929         }
00930         return;
00931     }
00932
00933     /* SPI send failed: retry with exponential backoff */
00934     entry->retries++;
00935     if (entry->retries >= k_event_max_retries) {
00936         rx_log_error_val(s_tag, "SPI event failed after retries", (uint8_t)entry->retries);
00937         entry->occupied = false;
00938         entry->retries = 0;
00939         entry->next_retry_time_ms = 0;
00940         mgr->event_queue_tail = (mgr->event_queue_tail + 1) % k_comm_event_queue_depth;
00941         mgr->event_queue_count--;
00942         return;

```

```

00943 }
00944
00945 /* Compute exponential backoff: initial_ms « (retries - 1), capped at max */
00946 uint32_t backoff_ms = (uint32_t)k_event_initial_backoff_ms « (entry->retries - 1);
00947 if (backoff_ms > k_event_max_backoff_ms) {
00948     backoff_ms = k_event_max_backoff_ms;
00949 }
00950 entry->next_retry_time_ms = now_ms + backoff_ms;
00951
00952 rx_log_warn_val(s_tag, "SPI event retry attempt", (uint8_t)entry->retries);
00953 /* Entry stays in queue, will be retried after backoff period */
00954 }
00955
00956 /*
=====
00957 * Heartbeat Helpers
00958 *
=====
00959 */
00960
00961 /**
00962 * @brief Check heartbeat implicit timeout on all transports
00963 *
00964 * @details
00965 * Evaluates the dual-detection heartbeat model.
00966 * If no valid frame has been received on a transport within 200ms, the link
00967 * is declared dead and the status callback is invoked.
00968 *
00969 * @param[in,out] mgr Communication manager handle
00970 *
00971 * @pre mgr must be non-NULL and initialized
00972 * @post Link status updated for each transport
00973 *
00974 * @since Version 1.0.0
00975 */
00976 static void internal_check_heartbeat(rx_comm_manager_t* mgr)
00977 {
00978     if (mgr == nullptr) {
00979         return;
00980     }
00981
00982     const uint32_t now = internal_get_time_ms();
00983
00984     for (uint8_t ch = 0; ch < k_comm_channel_count; ch++) {
00985         rx_comm_heartbeat_state_t* hb = &mgr->heartbeat[ch];
00986
00987         /* Skip channels that haven't received any frame yet */
00988         if (hb->status == k_link_status_unknown) {
00989             continue;
00990         }
00991
00992         /* Check implicit timeout: 200ms since last valid frame */
00993         if (hb->status == k_link_status_healthy) {
00994             uint32_t elapsed = now - hb->last_rx_ms;
00995             if (elapsed >= k_heartbeat_implicit_timeout_ms) {
00996                 hb->status = k_link_status_dead;
00997                 rx_log_warn_val(s_tag, "Link dead: no frame (ms elapsed)", elapsed);
00998
00999                 if (mgr->link_status_cb != nullptr) {
01000                     mgr->link_status_cb((rx_comm_channel_t)ch, k_link_status_dead, mgr->link_status_ctx);
01001                 }
01002             }
01003         }
01004     }
01005 }
01006
01007 /*
=====
01008 * Public Functions
01009 *
=====
01010 */
01011
01012 /**
01013 * @var s_zero_mgr
01014 * @brief Zero-initialized comm manager template for stack-safe initialization
01015 * @details Static const instance used to clear rx_comm_manager_t without creating
01016 *         a large compound literal on the stack. Lives in .rodata section.
01017 * @note Read-only; never modified after static initialization
01018 * @warning Do not remove - prevents stack overflow in init function
01019 * @see rx_comm_manager_init() Uses this template to zero-initialize the manager handle
01020 * @since Version 1.0.0
01021 */
01022 static const rx_comm_manager_t s_zero_mgr = {};
01023
01024 /**
01025 * @brief Initialize communication manager with channel configuration

```

```

01026 *
01027 * @details
01028 * Performs complete initialization of communication manager handle, configuring
01029 * enabled channels, callback registration, and ASCII debugging output. This function
01030 * must be called before any other communication manager operations.
01031 *
01032 * **Algorithm**:
01033 * 1. **Validate parameters**: Check mgr pointer for nullptr (NASA Rule 5)
01034 * 2. **Clear handle**: Zero-fill entire structure via memset (328 bytes)
01035 * 3. **Apply configuration**: Copy config fields if cfg is non-NULL
01036 *   - usb_handle: Pointer to initialized USB comm handle (may be NULL)
01037 *   - spi_handle: Pointer to initialized SPI comm handle (may be NULL)
01038 *   - callback: User frame handler function (may be NULL)
01039 *   - callback_ctx: User context pointer (may be NULL)
01040 *   - enable_decoded_output: ASCII debugging flag
01041 * 4. **Mark initialized**: Set mgr->initialized = true
01042 * 5. **Post-condition validation**: Verify ascii_buffer properly cleared
01043 *
01044 * **Configuration Options**:
01045 * - **cfg = NULL**: All channels disabled, no callback, ASCII output off
01046 * - **usb_handle = NULL**: USB channel disabled (poll skips USB)
01047 * - **spi_handle = NULL**: SPI channel disabled (poll skips SPI)
01048 * - **callback = NULL**: No callback invoked (silent frame consumption)
01049 * - **enable_decoded_output = true**: ASCII debugging enabled (Port 1)
01050 *
01051 * @param[out] mgr Pointer to communication manager handle to initialize
01052 * - **Valid range**: Non-nullptr to allocated rx_comm_manager_t (328 bytes)
01053 * - **Constraints**: Must be allocated by caller (typically static or global)
01054 * - **Side effects**: Entire structure zero-filled, then populated from cfg
01055 * - **Lifetime**: Must remain valid until rx_comm_manager_deinit() called
01056 *
01057 * @param[in] cfg Optional configuration structure
01058 * - **Valid range**: NULL or pointer to valid rx_comm_manager_config_t
01059 * - **NULL behavior**: All channels disabled, no callback, ASCII off
01060 * - **Lifetime**: Not stored - values copied into mgr
01061 * - **Constraints**: If non-NULL, all fields must be valid or nullptr as appropriate
01062 *
01063 * @return rx_err_t Error code indicating initialization success or failure
01064 * - **k_rx_ok**: Success - manager fully initialized and ready for operation
01065 * - **k_rx_err_invalid_arg**: mgr is nullptr
01066 * - **k_rx_err_invalid_state**: Post-condition validation failed (buffer not cleared)
01067 *
01068 * @pre mgr must point to allocated memory (uninitialized content OK)
01069 * @pre USB/SPI handles (if provided) must already be initialized
01070 * @pre Callback function (if provided) must be valid and reentrant
01071 * @post mgr->initialized = true on success
01072 * @post All mgr fields set to config values or zero
01073 * @post mgr->ascii_buffer is zero-filled (256 bytes)
01074 *
01075 * @note This function does NOT initialize USB or SPI transports - caller must initialize
01076 *       rx_usb_comm and rx_spi_comm separately before passing handles to this function
01077 * @note On success, at least one channel should be enabled (usb_handle or spi_handle non-NULL)
01078 * @note Thread-safe ONLY if each thread uses a different mgr handle
01079 *
01080 * @par Performance:
01081 * - Execution time: ~5 us @ 240 MHz
01082 * - Dominated by memset (328 bytes)
01083 *
01084 * @par Configuration Example - Both Channels Enabled:
01085 * @code{.c}
01086 * static rx_comm_manager_t g_comm_mgr;
01087 * static rx_usb_comm_handle_t g_usb_handle;
01088 * static rx_spi_comm_handle_t g_spi_handle;
01089 *
01090 * // Initialize transport layers first
01091 * rx_usb_comm_config_t usb_cfg = { .port = k_usb_port_proto };
01092 * rx_err_t err = rx_usb_comm_init(&g_usb_handle, &usb_cfg);
01093 * if (err != k_rx_ok) return err;
01094 *
01095 * rx_spi_comm_config_t spi_cfg = { .spi_channel = 0 };
01096 * err = rx_spi_comm_init(&g_spi_handle, &spi_cfg);
01097 * if (err != k_rx_ok) return err;
01098 *
01099 * // Initialize communication manager with both channels
01100 * rx_comm_manager_config_t cfg = {
01101 *   .usb_handle = &g_usb_handle,
01102 *   .spi_handle = &g_spi_handle,
01103 *   .callback = on_frame_received,
01104 *   .callback_ctx = nullptr,
01105 *   .enable_decoded_output = true,
01106 * };
01107 * err = rx_comm_manager_init(&g_comm_mgr, &cfg);
01108 * @endcode
01109 *
01110 * @par Configuration Example - USB Only, No Callback:
01111 * @code{.c}
01112 * rx_comm_manager_config_t cfg = {

```

```

01113 * .usb_handle = &g_usb_handle,
01114 * .spi_handle = nullptr,           // SPI disabled
01115 * .callback = nullptr,           // No callback
01116 * .callback_ctx = nullptr,
01117 * .enable_decoded_output = false, // No ASCII debugging
01118 * };
01119 * rx_err_t err = rx_comm_manager_init(&g_comm_mgr, &cfg);
01120 * @endcode
01121 *
01122 * @see rx_comm_manager_deinit() Cleanup and resource release
01123 * @see rx_comm_manager_poll() Poll for incoming frames
01124 * @see rx_usb_comm_init() Initialize USB CDC transport
01125 * @see rx_spi_comm_init() Initialize SPI transport
01126 *
01127 * @since Version 1.0.0
01128 */
01129 rx_err_t rx_comm_manager_init(rx_comm_manager_t* mgr, const rx_comm_manager_config_t* cfg)
01130 {
01131     /* Rule 5: Pre-condition validation - nullptr check */
01132     if (mgr == nullptr) {
01133         rx_log_error(s_tag, "Init failed: NULL mgr");
01134         return k_rx_err_invalid_arg;
01135     }
01136
01137     /* Rule 5: Pre-condition validation - SPI link consistency checks */
01138     if (cfg != nullptr && cfg->spi_link != nullptr) {
01139         /* If spi_link is provided, spi_handle must also be provided */
01140         if (cfg->spi_handle == nullptr) {
01141             rx_log_error(s_tag, "Init failed: spi_link requires non-NULL spi_handle");
01142             return k_rx_err_invalid_arg;
01143         }
01144
01145         /* If spi_link is provided, it must be initialized */
01146         if (!cfg->spi_link->initialized) {
01147             rx_log_error(s_tag, "Init failed: spi_link not initialized (call rx_spi_link_init first)");
01148             return k_rx_err_invalid_arg;
01149         }
01150
01151         /* If spi_link is provided, its spi_handle must match config spi_handle */
01152         if (cfg->spi_link->spi_handle != cfg->spi_handle) {
01153             rx_log_error(s_tag, "Init failed: spi_link->spi_handle != cfg->spi_handle");
01154             return k_rx_err_invalid_arg;
01155         }
01156     }
01157
01158     *mgr = s_zero_mgr;
01159
01160     /* Apply configuration */
01161     if (cfg != nullptr) {
01162         mgr->usb_handle = cfg->usb_handle;
01163         mgr->spi_handle = cfg->spi_handle;
01164         mgr->spi_link = cfg->spi_link;
01165         mgr->callback = cfg->callback;
01166         mgr->callback_ctx = cfg->callback_ctx;
01167         mgr->enable_decoded_output = cfg->enable_decoded_output;
01168         mgr->link_status_cb = cfg->link_status_cb;
01169         mgr->link_status_ctx = cfg->link_status_ctx;
01170     }
01171
01172     mgr->initialized = true;
01173
01174     /* Rule 5: Post-condition validation - verify buffer properly cleared */
01175     if (mgr->ascii_buffer[k_ascii_buffer_first_idx] != '\0') {
01176         /* Buffer not properly cleared by memset - potential memory corruption */
01177         mgr->initialized = false;
01178         rx_log_error(s_tag, "Init failed: buffer corruption");
01179         return k_rx_err_invalid_state;
01180     }
01181
01182     /* Rule 5: Post-condition validation - verify handle/link consistency */
01183     if (mgr->spi_link != nullptr && mgr->spi_link != cfg->spi_link) {
01184         /* Postcondition violation: spi_link not properly assigned */
01185         mgr->initialized = false;
01186         rx_log_error(s_tag, "Init failed: spi_link assignment mismatch");
01187         return k_rx_err_invalid_state;
01188     }
01189
01190     rx_log_info(s_tag, "Comm manager initialized");
01191     return k_rx_ok;
01192 }
01193
01194 /**
01195 * @brief Deinitialize communication manager and release resources
01196 *
01197 * @details
01198 * Marks communication manager as uninitialized, preventing further operations.
01199 * This function does NOT deinitialize USB or SPI transport handles - caller

```

```

01200 * must separately call rx_usb_comm_deinit() and rx_spi_comm_deinit() if needed.
01201 *
01202 * **Algorithm:**
01203 * 1. Validate mgr pointer (NULL check)
01204 * 2. Validate initialized flag (must be initialized to deinitialize)
01205 * 3. Clear initialized flag (marks handle as invalid)
01206 *
01207 * **Design Note**: Minimal deinitialization - no dynamic resources to free
01208 * (all memory is statically allocated). Function primarily serves as safety
01209 * guard to prevent use-after-deinit.
01210 *
01211 * @param[in,out] mgr Communication manager handle
01212 * - **Valid range**: Non-nullptr to initialized handle
01213 * - **Constraints**: Must have been initialized via rx_comm_manager_init()
01214 * - **Side effects**: Clears mgr->initialized flag
01215 *
01216 * @return rx_err_t Error code
01217 * @retval k_rx_ok Success - manager deinitialized
01218 * @retval k_rx_err_invalid_arg mgr is nullptr
01219 * @retval k_rx_err_invalid_state mgr was not initialized
01220 *
01221 * @pre mgr must be non-NULL
01222 * @pre mgr must be initialized (mgr->initialized == true)
01223 * @post mgr->initialized = false
01224 * @post Subsequent calls to rx_comm_manager_poll/send will return k_rx_err_invalid_state
01225 *
01226 * @note Does NOT free memory (handle is statically allocated)
01227 * @note Does NOT deinitialize transport handles (USB/SPI)
01228 * @note Thread-safe if no concurrent operations on same mgr handle
01229 *
01230 * @par Example:
01231 * @code{.c}
01232 * // Deinitialize manager
01233 * rx_err_t err = rx_comm_manager_deinit(&g_comm_mgr);
01234 * if (err == k_rx_ok) {
01235 *     // Manager no longer usable
01236 * }
01237 * // Separately deinitialize transports if needed
01238 * rx_usb_comm_deinit(&g_usb_handle);
01239 * rx_spi_comm_deinit(&g_spi_handle);
01240 * }
01241 * @endcode
01242 *
01243 * @see rx_comm_manager_init() Initialization
01244 * @see rx_usb_comm_deinit() USB transport cleanup
01245 * @see rx_spi_comm_deinit() SPI transport cleanup
01246 *
01247 * @since Version 1.0.0
01248 */
01249 rx_err_t rx_comm_manager_deinit(rx_comm_manager_t* mgr)
01250 {
01251     /* Rule 5: Pre-condition validation */
01252     if (mgr == nullptr) {
01253         rx_log_error(s_tag, "Deinit failed: NULL mgr");
01254         return k_rx_err_invalid_arg;
01255     }
01256     if (!mgr->initialized) {
01257         rx_log_warn(s_tag, "Deinit called on uninitialized mgr");
01258         return k_rx_err_invalid_state;
01259     }
01260     mgr->initialized = false;
01261     rx_log_info(s_tag, "Comm manager deinitialized");
01262     return k_rx_ok;
01263 }
01264
01265 /**
01266 * @brief Poll all enabled communication channels for incoming frames (non-blocking)
01267 *
01268 * @details
01269 * Performs non-blocking poll of all enabled channels (USB and SPI) sequentially,
01270 * dispatching any received frames to the registered user callback. Returns aggregate
01271 * result indicating whether any frames were received across all channels.
01272 *
01273 * **Algorithm:**
01274 * 1. **Validate parameters**: Check mgr pointer and initialized flag (NASA Rule 5)
01275 * 2. **Poll USB channel** via internal_poll_usb():
01276 *   - If k_rx_ok: Mark received=true, continue to SPI
01277 *   - If k_rx_err_timeout: Continue to SPI (no data is normal)
01278 *   - If other error: Return error immediately (critical failure)
01279 * 3. **Poll SPI channel** via internal_poll_spi():
01280 *   - If k_rx_ok: Mark received=true
01281 *   - If k_rx_err_timeout: Continue (no data is normal)
01282 *   - If other error: Return error immediately (critical failure)
01283 * 4. **Return result**:
01284 *   - k_rx_ok if ANY channel received frame

```

```

01287 * - k_rx_err_timeout if NO channels received (normal condition)
01288 * - Other error codes propagated from transport layers
01289 *
01290 * **Performance**: 15-200 us typical, depends on:
01291 * - Number of enabled channels (USB/SPI/both)
01292 * - Whether frames are available (data path vs fast timeout path)
01293 * - ASCII debugging enabled (adds ~50-150 us per frame)
01294 * - User callback execution time (not included in measurement)
01295 *
01296 * **Design Rationale**:
01297 * - **Non-blocking**: Never blocks waiting for data (0 ms timeout)
01298 * - **Fair scheduling**: Both channels polled each iteration
01299 * - **Timeout tolerance**: Timeout is expected, not error condition
01300 * - **Error prioritization**: Critical errors returned immediately
01301 * - **Aggregation**: k_rx_ok if ANY channel received data
01302 *
01303 * @param[in,out] mgr Communication manager handle
01304 * - **Valid range**: Non-nullptr to initialized handle
01305 * - **Constraints**: Must be initialized via rx_comm_manager_init()
01306 * - **Side effects**: Invokes callback, modifies ascii_buffer if frames received
01307 *
01308 * @return rx_err_t Error code indicating aggregate poll result
01309 * @retval k_rx_ok At least one frame received on any channel (success)
01310 * @retval k_rx_err_timeout No frames received on any channel (normal, not an error)
01311 * @retval k_rx_err_invalid_arg mgr is nullptr
01312 * @retval k_rx_err_invalid_state mgr not initialized
01313 * @retval k_rx_err_crc CRC verification failed on received frame (data corruption)
01314 * @retval (other) Transport layer critical errors propagated
01315 *
01316 * @pre mgr must be non-NULL
01317 * @pre mgr must be initialized
01318 * @pre At least one channel should be enabled (usb_handle or spi_handle non-NULL)
01319 * @post User callback invoked for each successfully received frame
01320 * @post ASCII output sent to Port 1 if enabled
01321 *
01322 * @note Non-blocking - always returns immediately
01323 * @note k_rx_err_timeout is NORMAL - means no data available (not an error)
01324 * @note User callback execution time is NOT included in performance measurements
01325 * @warning Call this function regularly (e.g., 100 Hz) to avoid buffer overruns
01326 * @warning User callback must not block for extended periods
01327 *
01328 * @par Performance:
01329 * - No frames available: ~15 us (both channels timeout, fast path)
01330 * - USB frame received: ~100-250 us (USB bulk transfer + callback)
01331 * - SPI frame received: ~50-150 us (SPI hardware transfer + callback)
01332 * - Both frames: ~150-400 us (both channels + callbacks)
01333 * - With ASCII debugging: Add ~50-150 us per frame
01334 *
01335 * @par Typical Usage - ThreadX Communication Task:
01336 * @code{.c}
01337 * static void comm_task_entry(ULONG thread_input)
01338 * {
01339 *     (void)thread_input;
01340 *
01341 *     // Initialize communication manager
01342 *     // ... (see rx_comm_manager_init() documentation) ...
01343 *
01344 *     // Main polling loop @ 100 Hz
01345 *     while (1) {
01346 *         // Poll all channels (non-blocking)
01347 *         rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
01348 *
01349 *         // Handle critical errors (not timeout)
01350 *         if (err != k_rx_ok && err != k_rx_err_timeout) {
01351 *             // Log error, implement recovery strategy
01352 *             log_error("Comm poll failed: %d", err);
01353 *             tx_thread_sleep(100); // 1 second backoff
01354 *             continue;
01355 *         }
01356 *
01357 *         // Timeout is normal - just means no data available
01358 *
01359 *         // Sleep 10 ticks (10 ms @ 1 kHz tick rate = 100 Hz polling)
01360 *         tx_thread_sleep(10);
01361 *     }
01362 * }
01363 * @endcode
01364 *
01365 * @par Error Handling - Distinguishing Timeout from Errors:
01366 * @code{.c}
01367 * rx_err_t err = rx_comm_manager_poll(&g_comm_mgr);
01368 * if (err == k_rx_ok) {
01369 *     // At least one frame received and processed
01370 * } else if (err == k_rx_err_timeout) {
01371 *     // No frames available - this is NORMAL, not an error
01372 * } else {
01373 *     // Critical error - handle appropriately

```

```

01374 *   handle_comm_error(err);
01375 * }
01376 * @endcode
01377 *
01378 * @see internal_poll_usb() USB channel polling implementation
01379 * @see internal_poll_spi() SPI channel polling implementation
01380 * @see internal_handle_frame() Frame dispatch
01381 * @see rx_comm_manager_init() Initialization with channel configuration
01382 *
01383 * @since Version 1.0.0
01384 */
01385 rx_err_t rx_comm_manager_poll(rx_comm_manager_t* mgr)
01386 {
01387     /* Rule 5: Pre-condition validation */
01388     if (mgr == nullptr) {
01389         return k_rx_err_invalid_arg;
01390     }
01391     if (!mgr->initialized) {
01392         return k_rx_err_invalid_state;
01393     }
01394
01395     bool received = false;
01396
01397     /* Poll USB channel */
01398     rx_err_t usb_err = internal_poll_usb(mgr);
01399     if (usb_err == k_rx_ok) {
01400         received = true;
01401     } else if (usb_err != k_rx_err_timeout) {
01402         /* Propagate non-timeout errors immediately */
01403         rx_log_error_val(s_tag, "USB poll error", usb_err);
01404         return usb_err;
01405     }
01406
01407     /* Poll SPI channel */
01408     rx_err_t spi_err = internal_poll_spi(mgr);
01409     if (spi_err == k_rx_ok) {
01410         received = true;
01411     } else if (spi_err != k_rx_err_timeout) {
01412         /* Propagate non-timeout errors immediately */
01413         rx_log_error_val(s_tag, "SPI poll error", spi_err);
01414         return spi_err;
01415     }
01416
01417     /* Process event queue: send one queued event per poll cycle */
01418     internal_process_event_queue(mgr);
01419
01420     /* Heartbeat: check implicit timeout on each transport */
01421     internal_check_heartbeat(mgr);
01422
01423     return received ? k_rx_ok : k_rx_err_timeout;
01424 }
01425
01426 /**
01427  * @brief Send frame on specified communication channel
01428  *
01429  * @details
01430  * Routes frame transmission to the appropriate transport layer (USB or SPI) based
01431  * on specified channel. Supports configurable frame type, flags, and variable-length
01432  * payload. Optionally outputs ASCII-formatted frame to USB Port 1 if debugging enabled.
01433  *
01434  * **Algorithm:**
01435  * 1. **Validate parameters** (NASA Rule 5):
01436  *   - mgr and params pointers non-NULL
01437  *   - Manager initialized flag
01438  *   - Payload pointer valid if payload_len > 0
01439  *   - payload_len <= k_frame_max_payload (255 bytes)
01440  * 2. **Route to channel**
01441  *   - USB: Verify usb_handle non-NULL, call rx_usb_comm_send()
01442  *   - SPI: Verify spi_handle non-NULL, call rx_spi_comm_send()
01443  *   - Invalid channel: Return k_rx_err_invalid_arg
01444  * 3. **On success, if ASCII debugging enabled**
01445  *   - Build temporary rx_frame_t from params
01446  *   - Format as ASCII via internal_output_decoded()
01447  *   - Send to USB Port 1 (TX direction)
01448  *
01449  * **Performance**
01450  * - USB: ~30-150 us (depends on payload size, USB bulk transfer)
01451  * - SPI: ~20-100 us (depends on payload size, SPI hardware transfer)
01452  * - ASCII debugging: Add ~50-150 us if enabled
01453  *
01454  * @param[in] mgr Communication manager handle
01455  * - **Valid range**: Non-nullptr to initialized handle
01456  * - **Constraints**: Must be initialized via rx_comm_manager_init()
01457  * - **Side effects**: May modify ascii_buffer if decoded output enabled
01458  *
01459  * @param[in] params Send parameters structure
01460  * - **Valid range**: Non-nullptr to valid rx_comm_send_params_t

```

```

01461 * - **Constraints**: All fields must be valid:
01462 * - channel: k_comm_channel_usb or k_comm_channel_spi
01463 * - type: Frame type (command, request, response, etc.)
01464 * - flags: Frame flags (k_frame_flag_none or specific flags)
01465 * - payload: Non-NULL if payload_len > 0, may be NULL if payload_len == 0
01466 * - payload_len: [0, k_frame_max_payload] (0-255 bytes)
01467 * - **Lifetime**: Not stored - values used immediately
01468 *
01469 * @return rx_err_t Error code indicating send success or failure
01470 * @retval k_rx_ok Frame sent successfully
01471 * @retval k_rx_err_invalid_arg mgr or params is nullptr, or payload invalid
01472 * @retval k_rx_err_invalid_arg payload_len > k_frame_max_payload (255 bytes)
01473 * @retval k_rx_err_invalid_arg Invalid channel specified
01474 * @retval k_rx_err_invalid_arg Payload is nullptr but payload_len > 0
01475 * @retval k_rx_err_invalid_state Manager not initialized
01476 * @retval k_rx_err_invalid_state Channel not configured (handle is nullptr)
01477 * @retval k_rx_err_hw USB/SPI hardware error (transport layer)
01478 * @retval (other) Transport layer errors propagated
01479 *
01480 * @pre mgr must be non-NULL and initialized
01481 * @pre params must be non-NULL with all valid fields
01482 * @pre Channel handle (USB/SPI) must be initialized
01483 * @pre If payload_len > 0, payload must point to valid data
01484 * @post Frame transmitted on specified channel if successful
01485 * @post ASCII output sent to Port 1 if enabled and successful
01486 *
01487 * @note Payload sequence number and CRC-32 managed by transport layer
01488 * @note ASCII debugging adds overhead - disable in production
01489 * @warning Ensure channel is ready before sending (use rx_comm_manager_channel_ready)
01490 *
01491 * @par Send Parameters Structure:
01492 * @code{.c}
01493 * typedef struct {
01494 *     rx_comm_channel_t channel;    // USB or SPI
01495 *     uint8_t type;                // Frame type
01496 *     uint8_t flags;               // Frame flags
01497 *     const uint8_t* payload;      // Payload data
01498 *     uint32_t payload_len;        // Payload length [0, 255]
01499 * } rx_comm_send_params_t;
01500 * @endcode
01501 *
01502 * @par Example - Send Command on USB:
01503 * @code{.c}
01504 * uint8_t cmd_payload[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
01505 *
01506 * rx_comm_send_params_t params = {
01507 *     .channel = k_comm_channel_usb,
01508 *     .type = k_frame_type_command,
01509 *     .flags = k_frame_flag_none,
01510 *     .payload = cmd_payload,
01511 *     .payload_len = sizeof(cmd_payload),
01512 * };
01513 *
01514 * rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
01515 * if (err == k_rx_ok) {
01516 *     // Command sent successfully
01517 * }
01518 * @endcode
01519 *
01520 * @par Example - Send Empty Frame (No Payload):
01521 * @code{.c}
01522 * rx_comm_send_params_t params = {
01523 *     .channel = k_comm_channel_spi,
01524 *     .type = k_frame_type_response,
01525 *     .flags = k_frame_flag_ack,
01526 *     .payload = nullptr,          // No payload
01527 *     .payload_len = 0,           // Zero length
01528 * };
01529 *
01530 * rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
01531 * @endcode
01532 *
01533 * @see rx_comm_manager_respond() Convenience wrapper for response frames
01534 * @see rx_comm_send_params_t Send parameters structure
01535 * @see rx_usb_comm_send() USB CDC send implementation
01536 * @see rx_spi_comm_send() SPI send implementation
01537 *
01538 * @since Version 1.0.0
01539 */
01540 rx_err_t rx_comm_manager_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01541 {
01542     /* Rule 5: Pre-condition validation */
01543     if (mgr == nullptr || params == nullptr) {
01544         rx_log_error(s_tag, "Send failed: NULL arg");
01545         return k_rx_err_invalid_arg;
01546     }
01547     if (!mgr->initialized) {

```

```

01548     rx_log_error(s_tag, "Send failed: not initialized");
01549     return k_rx_err_invalid_state;
01550 }
01551 if (params->payload == nullptr && params->payload_len > 0) {
01552     rx_log_error(s_tag, "Send failed: NULL payload with nonzero len");
01553     return k_rx_err_invalid_arg;
01554 }
01555 /* Validate payload length is within frame limits */
01556 if (params->payload_len > k_frame_max_payload) {
01557     rx_log_error_val(s_tag, "Send failed: payload too large", params->payload_len);
01558     return k_rx_err_invalid_arg;
01559 }
01560
01561 /* Send on appropriate channel */
01562 rx_err_t err = k_rx_ok;
01563 switch (params->channel) {
01564     case k_comm_channel_usb:
01565         if (mgr->usb_handle == nullptr) {
01566             rx_log_error(s_tag, "Send failed: USB handle NULL");
01567             return k_rx_err_invalid_state;
01568         }
01569         err = rx_usb_comm_send(mgr->usb_handle,
01570                               params->type,
01571                               params->flags,
01572                               params->payload,
01573                               params->payload_len);
01574         break;
01575
01576     case k_comm_channel_spi:
01577         if (mgr->spi_handle == nullptr) {
01578             rx_log_error(s_tag, "Send failed: SPI handle NULL");
01579             return k_rx_err_invalid_state;
01580         }
01581         /* Use HARQ link layer if available, otherwise raw SPI */
01582         if (mgr->spi_link != nullptr) {
01583             /* NOTE: params->flags intentionally NOT forwarded to rx_spi_link_send().
01584              * The link layer manages its own flags internally (k_frame_flag_requires_ack,
01585              * k_frame_flag_fec_enabled, k_frame_flag_retransmit) based on HARQ state.
01586              * Application-level flags in params->flags would be ignored if passed.
01587              * CLANG WARNING: bugprone-branch-clone is a false positive here.
01588              * The two branches call different functions with different signatures:
01589              * - rx_spi_link_send(link, type, payload, len) - no flags parameter
01590              * - rx_spi_comm_send(handle, type, flags, payload, len) - includes flags */
01591             err = rx_spi_link_send(mgr->spi_link, params->type, params->payload, params->payload_len);
01592         } else {
01593             err = rx_spi_comm_send(mgr->spi_handle,
01594                                   params->type,
01595                                   params->flags,
01596                                   params->payload,
01597                                   params->payload_len);
01598         }
01599         break;
01600
01601     default:
01602         rx_log_error(s_tag, "Send failed: invalid channel");
01603         return k_rx_err_invalid_arg;
01604 }
01605
01606 if (err != k_rx_ok) {
01607     rx_log_error_val(s_tag, "Transport send failed", err);
01608 }
01609
01610 /* Output decoded ASCII on success */
01611 if (err == k_rx_ok && mgr->enable_decoded_output) {
01612     /* Build a temporary frame for ASCII formatting */
01613     rx_frame_t frame = {0}; /* Zero-initialize to clear padding/unset fields */
01614     frame.header.sequence = k_frame_seq_placeholder;
01615     frame.header.length = (uint16_t)params->payload_len;
01616     frame.header.type = (uint8_t)params->type;
01617     frame.header.flags = params->flags;
01618     if (params->payload_len > 0 && params->payload_len <= k_frame_max_payload) {
01619         (void)memcpy(frame.payload, params->payload, params->payload_len);
01620     }
01621     frame.crc = k_frame_crc_placeholder;
01622
01623     internal_output_decoded(mgr, &frame, true);
01624 }
01625
01626 return err;
01627 }
01628
01629 /**
01630  * @brief Send response frame on specified channel (convenience wrapper)
01631  *
01632  * @details
01633  * Convenience function for sending response frames. Automatically sets frame type

```

```

01635 * to k_frame_type_response and flags to k_frame_flag_none. Simplifies common use
01636 * case of responding to commands/requests.
01637 *
01638 * **Algorithm:**
01639 * 1. Build rx_comm_send_params_t with:
01640 *   - channel: User-specified
01641 *   - type: k_frame_type_response (fixed)
01642 *   - flags: k_frame_flag_none (fixed)
01643 *   - payload: User-specified
01644 *   - payload_len: User-specified
01645 * 2. Call rx_comm_manager_send() with constructed params
01646 * 3. Return result from rx_comm_manager_send()
01647 *
01648 * **When to Use**:
01649 * - Responding to commands with telemetry data
01650 * - Sending acknowledgments
01651 * - Replying to requests
01652 *
01653 * **When NOT to Use**:
01654 * - Need custom frame type (use rx_comm_manager_send instead)
01655 * - Need custom flags (use rx_comm_manager_send instead)
01656 *
01657 * @param[in] mgr Communication manager handle
01658 *   - **Valid range**: Non-nullptr to initialized handle
01659 *   - **Constraints**: Must be initialized via rx_comm_manager_init()
01660 *
01661 * @param[in] channel Channel to send response on
01662 *   - **Valid range**: k_comm_channel_usb or k_comm_channel_spi
01663 *   - **Constraints**: Channel handle must be configured
01664 *
01665 * @param[in] payload Response payload data
01666 *   - **Valid range**: Non-NULL if payload_len > 0, may be NULL if payload_len == 0
01667 *   - **Constraints**: Must contain valid data for payload_len bytes
01668 *   - **Lifetime**: Copied immediately, does not need to persist
01669 *
01670 * @param[in] payload_len Response payload length in bytes
01671 *   - **Valid range**: [0, k_frame_max_payload] (0-255 bytes)
01672 *   - **Units**: Bytes
01673 *
01674 * @return rx_err_t Error code (same as rx_comm_manager_send)
01675 * @retval k_rx_ok Response sent successfully
01676 * @retval k_rx_err_invalid_arg mgr is nullptr or payload invalid
01677 * @retval k_rx_err_invalid_state Manager not initialized or channel not configured
01678 * @retval (other) Errors propagated from rx_comm_manager_send()
01679 *
01680 * @pre Same preconditions as rx_comm_manager_send()
01681 * @post Response frame transmitted with type=response, flags=none
01682 *
01683 * @note Equivalent to calling rx_comm_manager_send() with type=k_frame_type_response
01684 * @note This is a thin wrapper - no additional validation performed
01685 *
01686 * @par Example - Send Telemetry Response:
01687 * @code{.c}
01688 * // Received a telemetry request on USB, send response
01689 * static void on_telemetry_request(rx_comm_channel_t channel, const rx_frame_t* frame)
01690 * {
01691 *     uint8_t telemetry[16];
01692 *
01693 *     // Gather telemetry data
01694 *     telemetry[0] = get_temperature_celsius();
01695 *     telemetry[1] = get_current_ma();
01696 *     // ... fill remaining fields ...
01697 *
01698 *     // Send response on same channel request came from
01699 *     rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
01700 *                                           channel,
01701 *                                           telemetry,
01702 *                                           sizeof(telemetry));
01703 *     if (err != k_rx_ok) {
01704 *         // Handle send error
01705 *     }
01706 * }
01707 * @endcode
01708 *
01709 * @par Example - Send Empty Acknowledgment:
01710 * @code{.c}
01711 * // Send empty ACK response
01712 * rx_err_t err = rx_comm_manager_respond(&g_comm_mgr,
01713 *                                       k_comm_channel_usb,
01714 *                                       NULL, // No payload
01715 *                                       0); // Zero length
01716 * @endcode
01717 *
01718 * @see rx_comm_manager_send() Full send function with all parameters
01719 * @see rx_comm_send_params_t Send parameters structure
01720 *
01721 * @since Version 1.0.0

```

```

01722 */
01723 rx_err_t rx_comm_manager_respond(rx_comm_manager_t* mgr,
01724                                 rx_comm_channel_t channel,
01725                                 const uint8_t* payload,
01726                                 uint32_t payload_len)
01727 {
01728     const rx_comm_send_params_t params = {
01729         .channel = channel,
01730         .type = k_frame_type_response,
01731         .flags = k_frame_flag_none,
01732         .payload = payload,
01733         .payload_len = payload_len,
01734     };
01735     return rx_comm_manager_send(mgr, &params);
01736 }
01737
01738 /*
=====
01739 * Stream / Event Send API
01740 *
=====
01741 */
01742
01743 rx_err_t rx_comm_manager_stream_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01744 {
01745     /* Stream path: fire-and-forget, no retries, no queuing.
01746      * Telemetry and other time-sensitive data goes here.
01747      * Stale data has no value, so failure is silently accepted. */
01748     return rx_comm_manager_send(mgr, params);
01749 }
01750
01751 rx_err_t rx_comm_manager_event_send(rx_comm_manager_t* mgr, const rx_comm_send_params_t* params)
01752 {
01753     if (mgr == nullptr || params == nullptr) {
01754         rx_log_error(s_tag, "Event enqueue failed: NULL arg");
01755         return k_rx_err_invalid_arg;
01756     }
01757
01758     if (!mgr->initialized) {
01759         rx_log_error(s_tag, "Event enqueue failed: not initialized");
01760         return k_rx_err_invalid_state;
01761     }
01762
01763     if (params->payload_len > k_frame_max_payload) {
01764         rx_log_error_val(s_tag, "Event enqueue failed: payload too large", params->payload_len);
01765         return k_rx_err_invalid_arg;
01766     }
01767
01768     if (params->payload_len > 0 && params->payload == nullptr) {
01769         rx_log_error(s_tag, "Event enqueue failed: NULL payload with nonzero len");
01770         return k_rx_err_invalid_arg;
01771     }
01772
01773     /* Check queue capacity */
01774     if (mgr->event_queue_count >= k_comm_event_queue_depth) {
01775         rx_log_error_val(s_tag, "Event queue full, depth", (uint8_t)mgr->event_queue_count);
01776         return k_rx_err_no_mem;
01777     }
01778
01779     /* Enqueue: copy payload into static slot */
01780     rx_comm_event_entry_t* entry = &mgr->event_queue[mgr->event_queue_head];
01781     entry->channel = params->channel;
01782     entry->type = params->type;
01783     entry->flags = params->flags;
01784     entry->payload_len = params->payload_len;
01785     entry->retries = 0;
01786     entry->occupied = true;
01787
01788     if (params->payload != nullptr && params->payload_len > 0) {
01789         (void)memcpy(entry->payload, params->payload, params->payload_len);
01790     }
01791
01792     mgr->event_queue_head = (mgr->event_queue_head + 1) % k_comm_event_queue_depth;
01793     mgr->event_queue_count++;
01794
01795     rx_log_debug_val(s_tag, "Event enqueued, queue depth", mgr->event_queue_count);
01796     return k_rx_ok;
01797 }
01798
01799 /*
=====
01800 * Heartbeat API
01801 *
=====
01802 */
01803
01804 rx_err_t rx_comm_manager_link_status(const rx_comm_manager_t* mgr,

```

```

01805         rx_comm_channel_t    channel,
01806         rx_comm_link_status_t* status)
01807 {
01808     if (mgr == nullptr || status == nullptr) {
01809         rx_log_error(s_tag, "Link status query: NULL arg");
01810         return k_rx_err_invalid_arg;
01811     }
01812
01813     if (!mgr->initialized) {
01814         rx_log_error(s_tag, "Link status query: uninitialized manager");
01815         return k_rx_err_invalid_state;
01816     }
01817
01818     if (channel >= k_comm_channel_count) {
01819         rx_log_error(s_tag, "Link status query: invalid channel");
01820         return k_rx_err_invalid_arg;
01821     }
01822
01823     *status = mgr->heartbeat[channel].status;
01824     return k_rx_ok;
01825 }
01826
01827 /**
01828  * @brief Query if communication channel is ready for send/receive operations
01829  *
01830  * @details
01831  * Checks if specified channel is configured and ready for communication. USB channel
01832  * readiness depends on USB enumeration and configuration by host. SPI channel is
01833  * ready immediately if handle is configured (no enumeration required).
01834  *
01835  * **Readiness Criteria**:
01836  * - **USB**: usb_handle non-NULL AND USB device enumerated and configured by host
01837  * - **SPI**: spi_handle non-NULL (always ready if configured)
01838  *
01839  * **Use Cases**:
01840  * - Check readiness before sending to avoid errors
01841  * - Implement fallback logic (try USB, fall back to SPI)
01842  * - Log channel availability status
01843  *
01844  * @param[in] mgr Communication manager handle
01845  * - **Valid range**: Non-nullptr to initialized handle
01846  * - **Constraints**: Must be initialized via rx_comm_manager_init()
01847  *
01848  * @param[in] channel Channel to query
01849  * - **Valid range**: k_comm_channel_usb or k_comm_channel_spi
01850  * - **Constraints**: Must be valid channel ID
01851  *
01852  * @param[out] ready Pointer to receive ready status
01853  * - **Valid range**: Non-nullptr to bool
01854  * - **Output values**: true (ready) or false (not ready)
01855  * - **On error**: Set to false
01856  *
01857  * @return rx_err_t Error code
01858  * @retval k_rx_ok Query successful, ready flag updated
01859  * @retval k_rx_err_invalid_arg mgr or ready is nullptr, or channel invalid
01860  * @retval k_rx_err_invalid_state Manager not initialized
01861  *
01862  * @pre mgr must be non-NULL and initialized
01863  * @pre ready must be non-NULL
01864  * @pre channel must be valid (USB or SPI)
01865  * @post *ready set to true or false based on channel state
01866  * @post On error, *ready set to false (safe default)
01867  *
01868  * @note USB readiness can change at runtime (host connects/disconnects)
01869  * @note SPI readiness is static (always ready if handle configured)
01870  * @note This function does NOT initiate communication - only queries state
01871  *
01872  * @par Example - Send with Fallback:
01873  * @code{.c}
01874  * uint8_t payload[16] = { ... };
01875  *
01876  * // Try USB first
01877  * bool usb_ready = false;
01878  * rx_err_t err = rx_comm_manager_channel_ready(&g_comm_mgr,
01879  *                                             k_comm_channel_usb,
01880  *                                             &usb_ready);
01881  * if (err == k_rx_ok && usb_ready) {
01882  *     err = rx_comm_manager_respond(&g_comm_mgr,
01883  *                                 k_comm_channel_usb,
01884  *                                 payload,
01885  *                                 sizeof(payload));
01886  * } else {
01887  *     // Fallback to SPI
01888  *     bool spi_ready = false;
01889  *     err = rx_comm_manager_channel_ready(&g_comm_mgr,
01890  *                                       k_comm_channel_spi,
01891  *                                       &spi_ready);

```

```

01892 *   if (err == k_rx_ok && spi_ready) {
01893 *       err = rx_comm_manager_respond(&g_comm_mgr,
01894 *                                   k_comm_channel_spi,
01895 *                                   payload,
01896 *                                   sizeof(payload));
01897 *   }
01898 * }
01899 * @endcode
01900 *
01901 * @par Example - Log Channel Status:
01902 * @code{.c}
01903 * void log_channel_status(void)
01904 * {
01905 *     bool usb_ready = false, spi_ready = false;
01906 *
01907 *     rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_usb, &usb_ready);
01908 *     rx_comm_manager_channel_ready(&g_comm_mgr, k_comm_channel_spi, &spi_ready);
01909 *
01910 *     printf("USB: %s, SPI: %s\n",
01911 *           usb_ready ? "READY" : "NOT READY",
01912 *           spi_ready ? "READY" : "NOT READY");
01913 * }
01914 * @endcode
01915 *
01916 * @see rx_usb_is_configured() USB enumeration status
01917 * @see rx_comm_manager_send() Send frame on channel
01918 *
01919 * @since Version 1.0.0
01920 */
01921 rx_err_t
01922 rx_comm_manager_channel_ready(rx_comm_manager_t* mgr, rx_comm_channel_t channel, bool* ready)
01923 {
01924     /* Rule 5: Pre-condition validation */
01925     if (mgr == nullptr || ready == nullptr) {
01926         if (ready != nullptr) {
01927             *ready = false;
01928         }
01929         return k_rx_err_invalid_arg;
01930     }
01931     if (!mgr->initialized) {
01932         *ready = false;
01933         return k_rx_err_invalid_state;
01934     }
01935     switch (channel) {
01936     case k_comm_channel_usb:
01937         *ready = (mgr->usb_handle != nullptr) && rx_usb_is_configured(k_usb_port_proto);
01938         break;
01939     case k_comm_channel_spi:
01940         /* SPI is always "ready" if handle is set */
01941         *ready = (mgr->spi_handle != nullptr);
01942         break;
01943     default:
01944         *ready = false;
01945         rx_log_error(s_tag, "Channel ready: invalid channel");
01946         return k_rx_err_invalid_arg;
01947     }
01948     return k_rx_ok;
01949 }
01950
01951 /**
01952 * @brief Get human-readable name for communication channel
01953 *
01954 * @details
01955 * Returns static string name for given channel enum value. Used for logging,
01956 * debugging, and user interface display. Always returns a valid string (never NULL).
01957 *
01958 * **Returned Strings**:
01959 * - k_comm_channel_usb -> "USB"
01960 * - k_comm_channel_spi -> "SPI"
01961 * - Invalid/unknown -> "UNKNOWN"
01962 *
01963 * **Use Cases**:
01964 * - Logging channel activity
01965 * - Debugging frame traffic
01966 * - User interface display
01967 * - Error messages
01968 *
01969 * @param[in] channel Communication channel enum value
01970 * - **Valid range**: Any rx_comm_channel_t value
01971 * - **Constraints**: None (invalid values return "UNKNOWN")
01972 *
01973 * @return const char* Human-readable channel name (never NULL)
01974 * @retval "USB" If channel == k_comm_channel_usb

```

```

01979 * @retval "SPI" If channel == k_comm_channel_spi
01980 * @retval "UNKNOWN" If channel is invalid or out of range
01981 *
01982 * @pre None (accepts any input)
01983 * @post Returns pointer to static string (valid for program lifetime)
01984 *
01985 * @note Returned string is static - do not free
01986 * @note Thread-safe (returns read-only static strings)
01987 * @note Never returns NULL - always safe to use in printf
01988 *
01989 * @par Example - Logging:
01990 * @code{.c}
01991 * static void on_frame_received(rx_comm_channel_t channel,
01992 *                             const rx_frame_t* frame,
01993 *                             void* ctx)
01994 * {
01995 *     printf("Frame received on %s: type=%d len=%d\n",
01996 *          rx_comm_manager_channel_name(channel),
01997 *          frame->header.type,
01998 *          frame->header.length);
01999 * }
02000 * @endcode
02001 *
02002 * @par Example - Error Messages:
02003 * @code{.c}
02004 * rx_err_t err = rx_comm_manager_send(&g_comm_mgr, &params);
02005 * if (err != k_rx_ok) {
02006 *     fprintf(stderr, "Failed to send on %s: error %d\n",
02007 *          rx_comm_manager_channel_name(params.channel), err);
02008 * }
02009 * @endcode
02010 *
02011 * @see rx_comm_channel_t Channel enumeration
02012 *
02013 * @since Version 1.0.0
02014 */
02015 const char* rx_comm_manager_channel_name(rx_comm_channel_t channel)
02016 {
02017     switch (channel) {
02018     case k_comm_channel_usb:
02019         return "USB";
02020     case k_comm_channel_spi:
02021         return "SPI";
02022     default:
02023         return "UNKNOWN";
02024     }
02025 }
02026
02027 /*
=====
02028 * Retransmission Pass-Through
02029 *
=====
02030 */
02031
02032 rx_err_t rx_comm_manager_process_retransmits(rx_comm_manager_t* mgr, const uint32_t current_time_ms)
02033 {
02034     if (mgr == nullptr) {
02035         return k_rx_err_invalid_arg;
02036     }
02037
02038     if (!mgr->initialized) {
02039         return k_rx_err_invalid_state;
02040     }
02041
02042     if (mgr->spi_handle == nullptr) {
02043         return k_rx_ok; /* No SPI channel configured */
02044     }
02045
02046     return rx_spi_comm_process_retransmits(mgr->spi_handle, current_time_ms);
02047 }
02048
02049 rx_err_t rx_comm_manager_set_auto_retransmit(rx_comm_manager_t* mgr,
02050                                             const bool enabled,
02051                                             const rx_spi_comm_retransmit_config_t* config)
02052 {
02053     if (mgr == nullptr) {
02054         return k_rx_err_invalid_arg;
02055     }
02056
02057     if (!mgr->initialized) {
02058         return k_rx_err_invalid_state;
02059     }
02060
02061     if (mgr->spi_handle == nullptr) {
02062         return k_rx_err_not_supported;
02063     }

```

```
02064  
02065  return rx_spi_comm_set_auto_retransmit(mgr->spi_handle, enabled, config);  
02066 }
```